

Trie





Trie



Trie

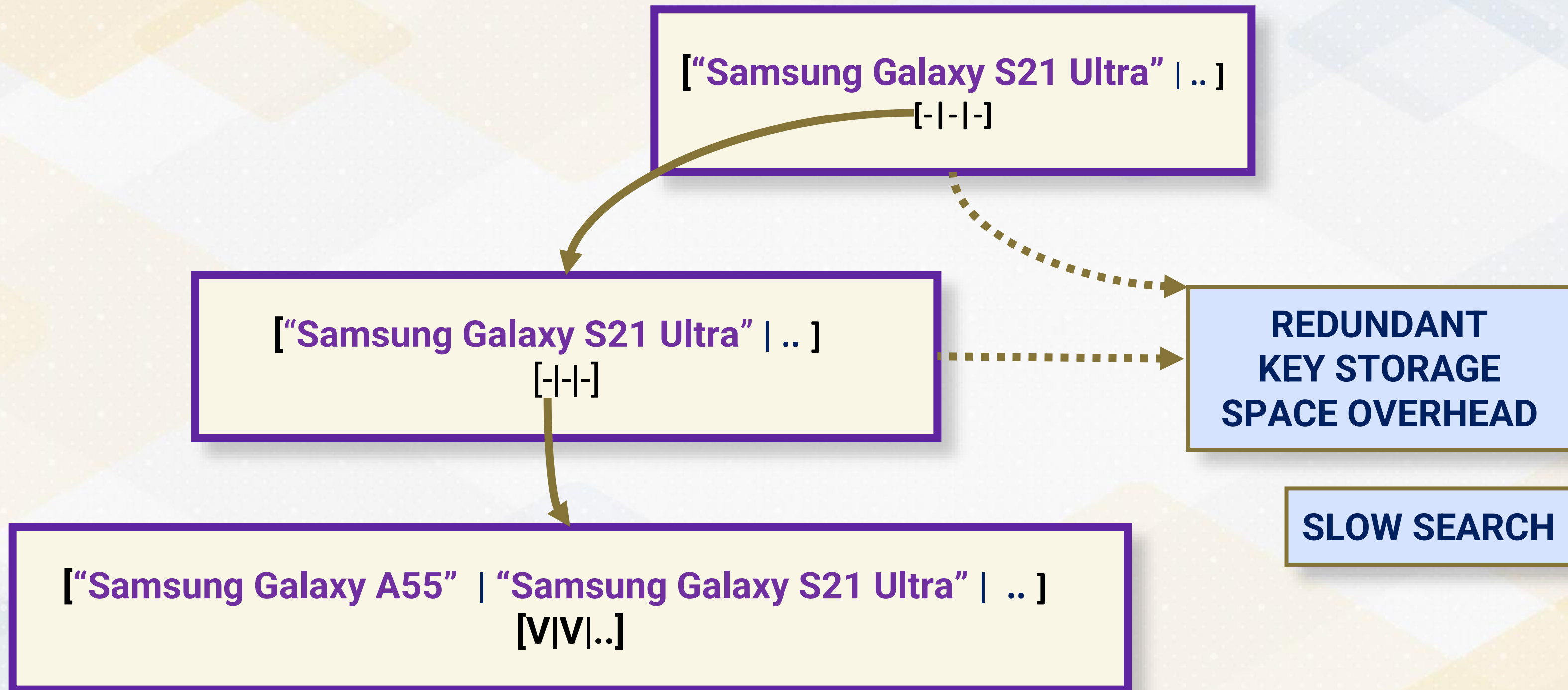
**Patricia
Trie**

Product Catalog Search

Product Name	Relevant or not
Samsung Galaxy S21 Ultra	Yes
Apple iPhone XS Max	No
Samsung Galaxy A55	Yes
Apple iPhone 12	No
Samsung Galaxy S27	Yes

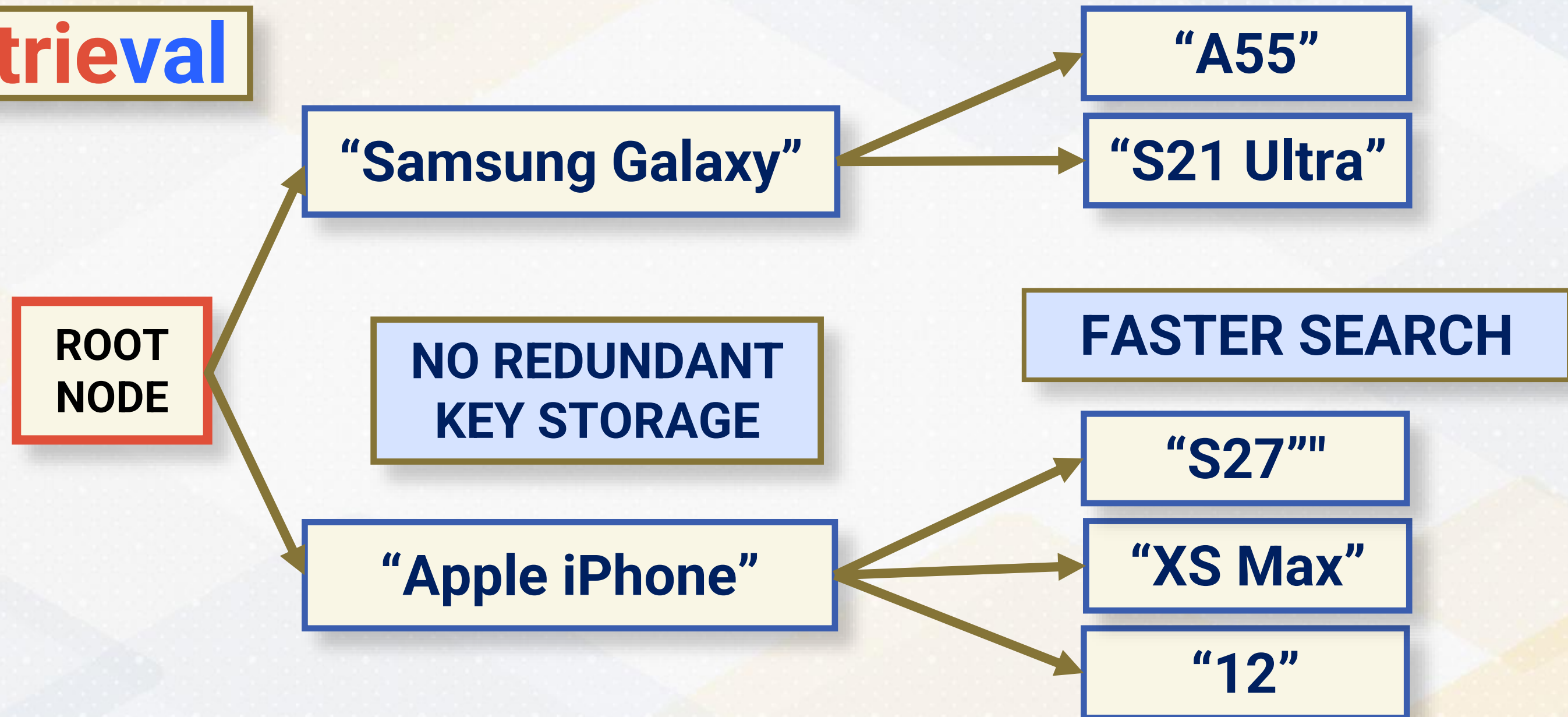
```
// Find all Samsung Galaxy phones in the catalog
std::vector<std::string> galaxyPhones =
    productCatalog.startsWith("Samsung Galaxy");
```

Limitations of B+Tree



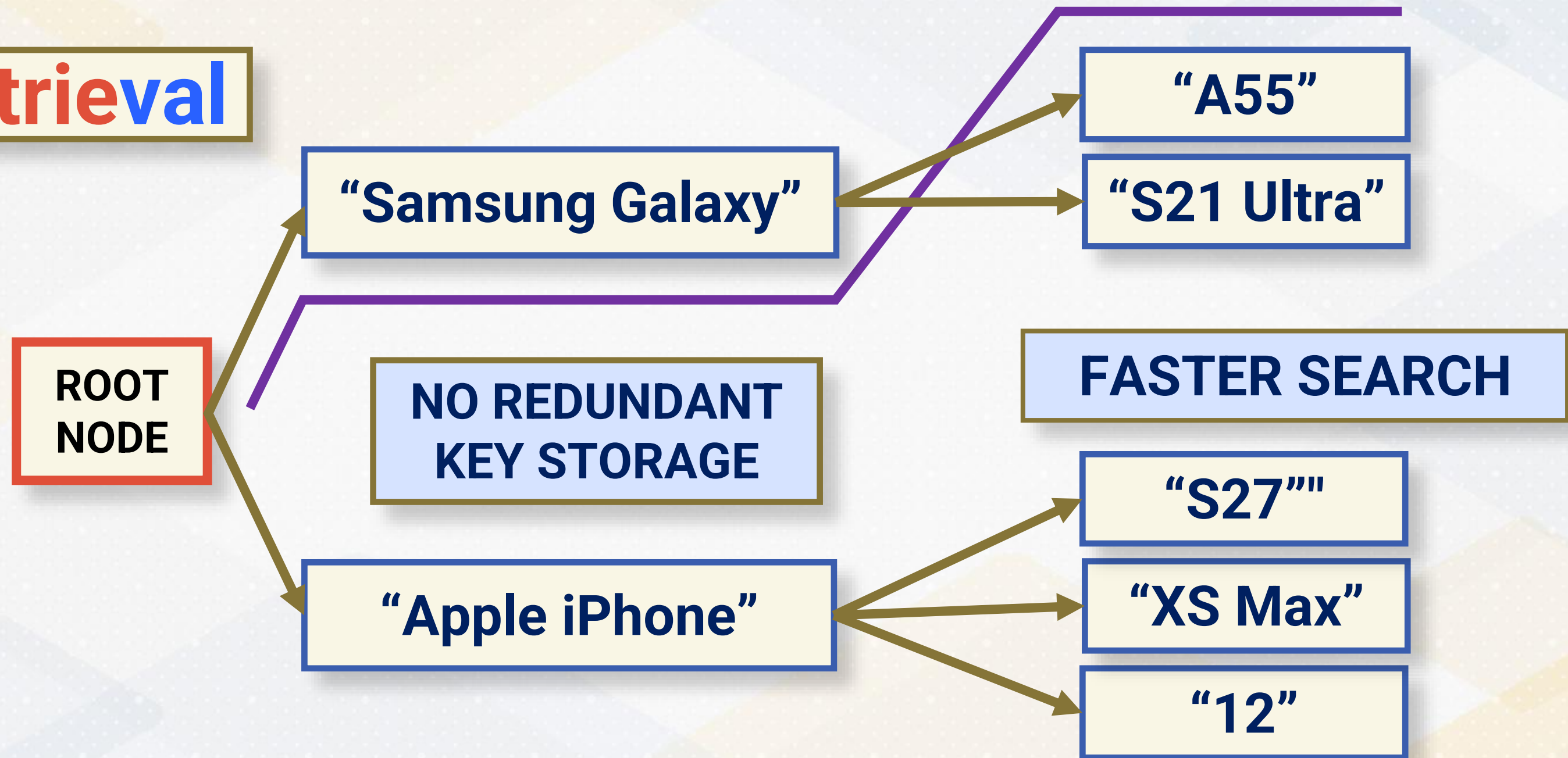
Trie

Retrieval



Trie

Retrieval



Patricia Trie

Trie

Patricia Trie

Trie

Merge Single
Character Nodes

Into Single Node

Or Whole Strings

Patricia Trie

Trie

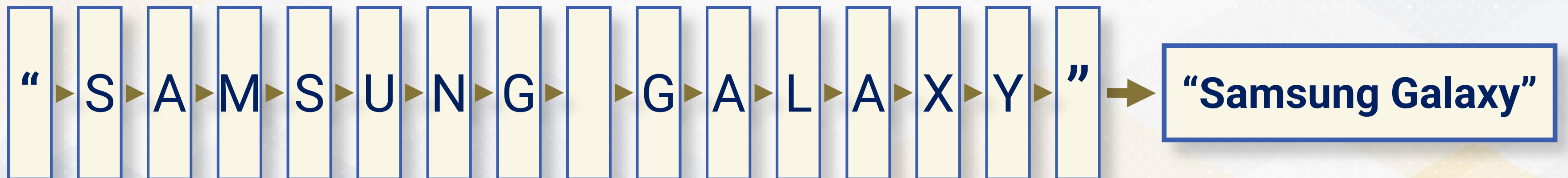
Merge Single
Character Nodes

Into Single Node

Or Whole Strings

Basic Trie

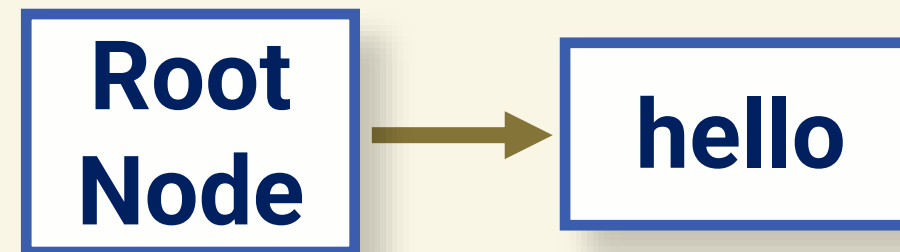
Patricia Trie



Insertion Example: “hello”

**Root
Node**

Insertion Example: “hello”

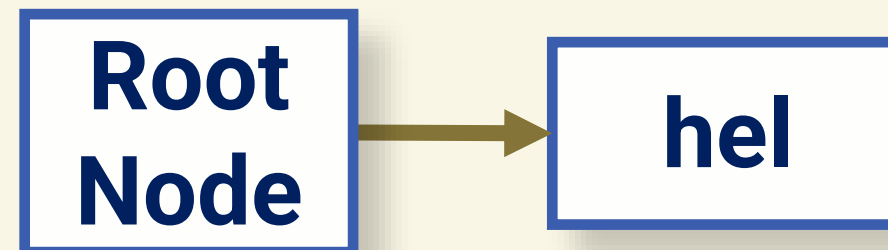


Insertion Example: “helium”

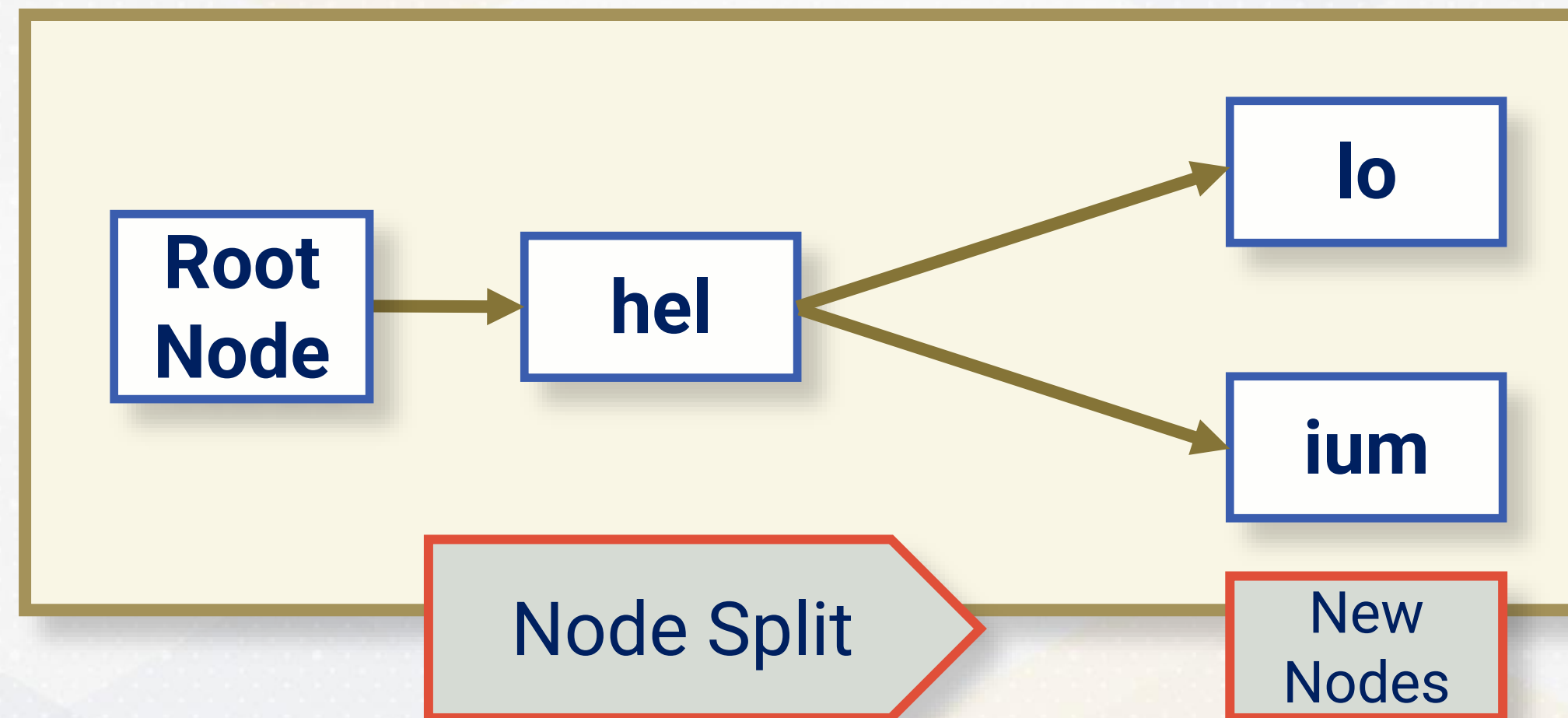
**Root
Node**



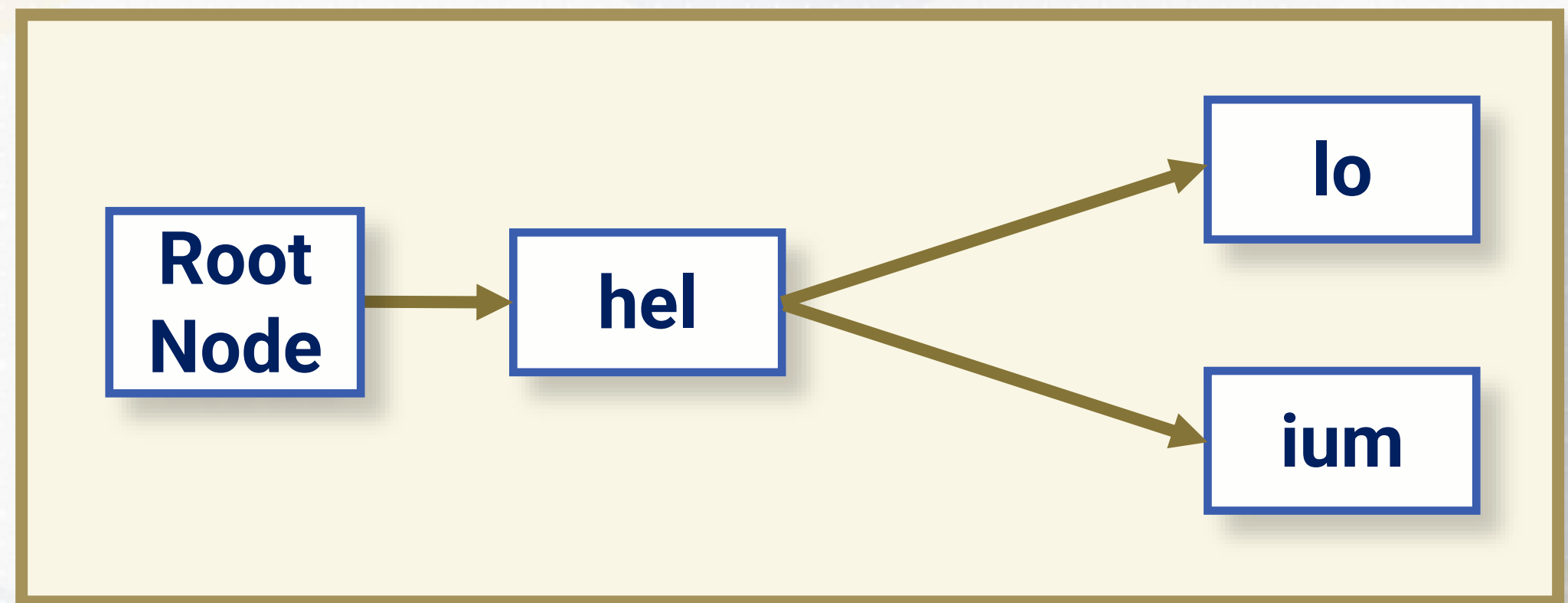
Insertion Example: “helium”



Insertion Example: “helium”

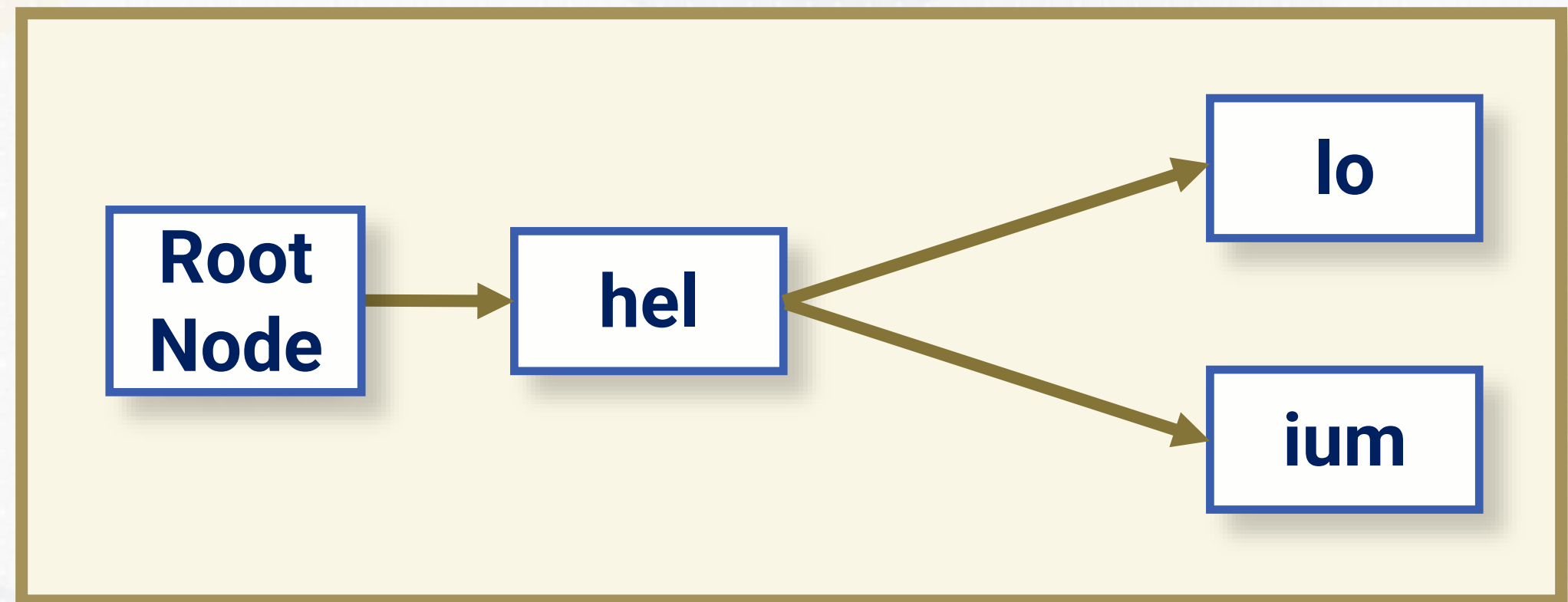


Retrieval: “hello” and “helicopter”



Retrieval: “hello” and “helicopter”

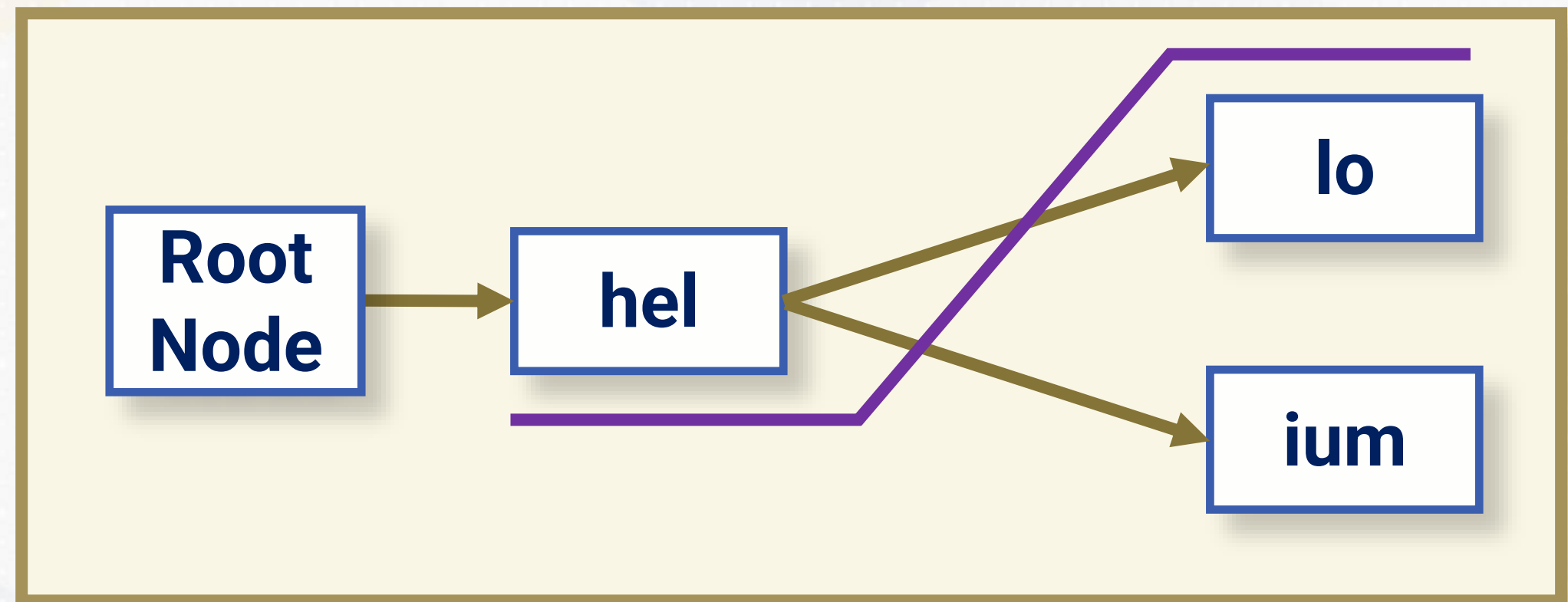
Navigate Root Node to
Relevant Leaf Node



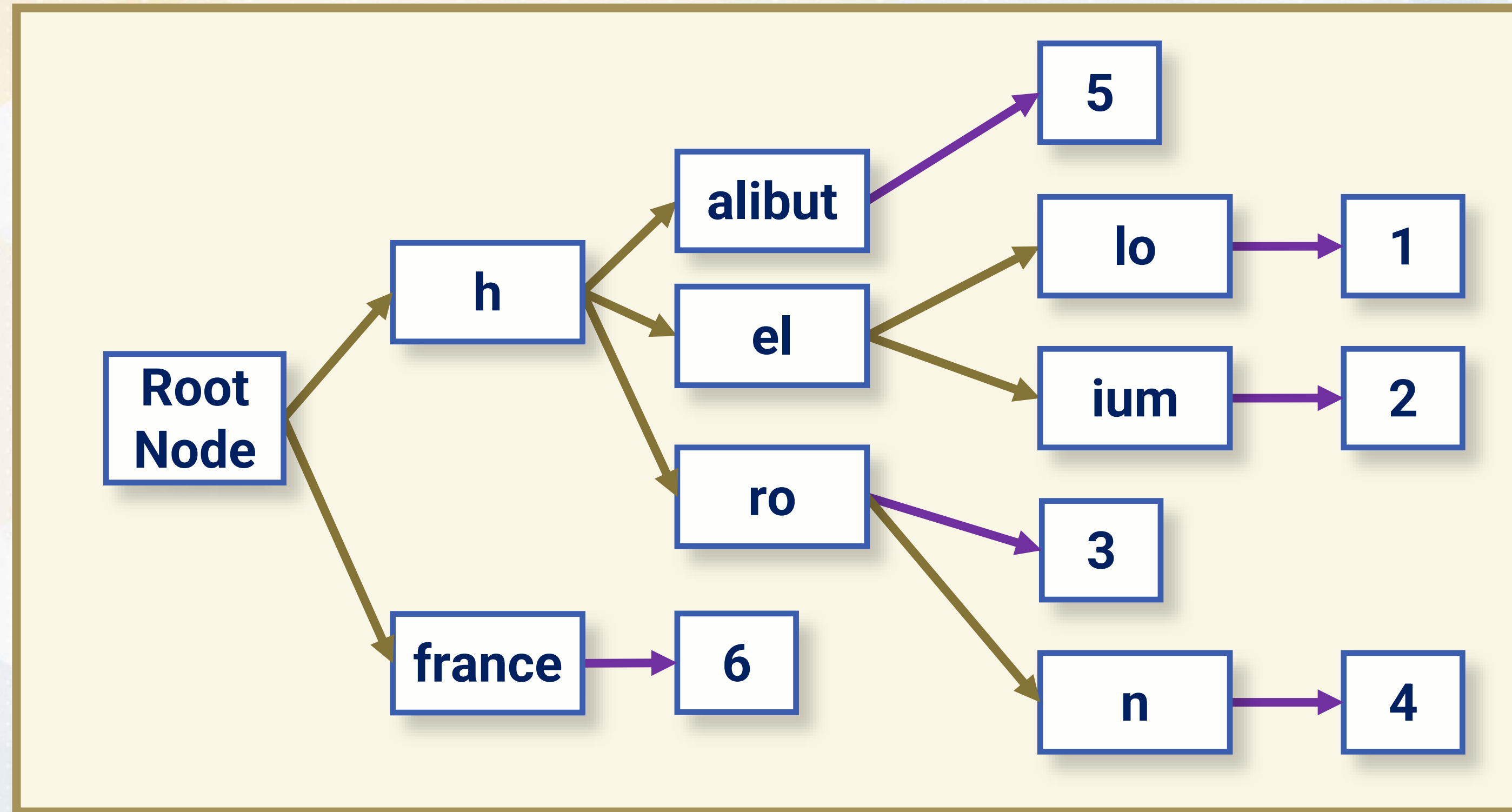
Retrieval: “hello” and “helicopter”

Navigate Root Node to
Relevant Leaf Node

Each Character
Narrows Down Search



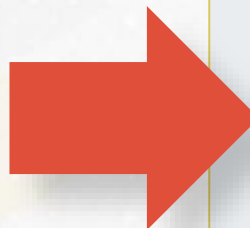
More Complex Trie with Key-Value Pairs



Patricia Trie

```
class PatriciaNode {  
public:  
    bool isEndOfWord;  
    int value;  
    std::map<std::string, PatriciaNode *> children;  
  
    PatriciaNode() : isEndOfWord(false) {}  
};
```

Patricia Trie



```
class PatriciaNode {  
public:  
    bool isEndOfWord;  
    int value;  
    std::map<std::string, PatriciaNode *> children;  
  
    PatriciaNode() : isEndOfWord(false) {}  
};
```


Insertion

```
void insertHelper(PatriciaNode *node, const std::string &word, size_t index) {  
    // Termination case: If the whole word has been processed  
    if (index == word.length()) {  
        node->isEndOfWord = true;  
        return;  
    }  
    // Prepare the substring from the current index to the end  
    std::string remaining = word.substr(index);  
    ...  
}
```

Insertion

Termination Case

```
void insertHelper(PatriciaNode *node, const std::string &word, size_t index) {  
    // Termination case: If the whole word has been processed  
    if (index == word.length()) {  
        node->isEndOfWord = true;  
        return;  
    }  
    // Prepare the substring from the current index to the end  
    std::string remaining = word.substr(index);  
    ...  
}
```

Insertion

Termination Case

Stop Recursion

```
void insertHelper(PatriciaNode *node, const std::string &word, size_t index) {  
    // Termination case: If the whole word has been processed  
    if (index == word.length()) {  
        node->isEndOfWord = true;  
        return;  
    }  
    // Prepare the substring from the current index to the end  
    std::string remaining = word.substr(index);  
    ...  
}
```


Insertion

Termination Case

Stop Recursion

Find Substring

```
void insertHelper(PatriciaNode *node, const std::string &word, size_t index) {  
    // Termination case: If the whole word has been processed  
    if (index == word.length()) {  
        node->isEndOfWord = true;  
        return;  
    }  
    // Prepare the substring from the current index to the end  
    std::string remaining = word.substr(index);  
    ...  
}
```

Insertion: Identifying Common Prefixes

Trie

```
for (auto &child : node->children) {  
    // Calculate common prefix length  
    const std::string &key = child.first;  
    PatriciaNode *childNode = child.second;  
    size_t commonLength = 0;  
    while (commonLength < key.length() && commonLength < remaining.length() &&  
           key[commonLength] == remaining[commonLength]) {  
        commonLength++;  
    }  
    ...  
}
```

Insertion: Identifying Common Prefixes

Trie

Identifies
Common Prefixes

Splits Nodes
When Necessary

Common Prefix
Length

```
for (auto &child : node->children) {  
    // Calculate common prefix length  
    const std::string &key = child.first;  
    PatriciaNode *childNode = child.second;  
    size_t commonLength = 0;  
    while (commonLength < key.length() && commonLength < remaining.length() &&  
           key[commonLength] == remaining[commonLength]) {  
        commonLength++;  
    }  
    ...  
}
```


Insertion: Handling No Common Prefix

```
// If no child matches the remaining part of the word  
node->children[remaining] = new PatriciaNode();  
  
// Recursively insert the remainder – e.g. “hello”  
insertHelper(node->children[remaining], word, word.length(), value);
```

Node Splitting

```
if (commonLength > 0) {  
    if (commonLength < key.length()) {  
        // Split the child's key  
        std::string newKey = key.substr(0, commonLength); // "hel"  
        std::string oldKey = key.substr(commonLength); // "lo"  
        PatriciaNode *newChildNode = new PatriciaNode();  
        newChildNode->children[oldKey] = childNode;  
        node->children[newKey] = newChildNode;  
        node->children.erase(key);  
        childNode = newChildNode;  
    }  
    // remainder: "ium"  
    insertHelper(childNode, word, index + commonLength, value);  
    return;  
}
```

Complexity Comparison of Trie and B+Tree

L: Average length of the key

N: Number of keys

	Patricia Trie	B+Tree
Insertion Time Complexity	$O(L)$	$O(\log N)$
Search Time Complexity	$O(L)$	$O(\log N)$

Complexity Comparison of Trie and B+Tree

L: Average length of the key

N: Number of keys

	Patricia Trie	B+Tree
Insertion Time Complexity	$O(L)$	$O(\log N)$
Search Time Complexity	$O(L)$	$O(\log N)$
Space Complexity	$O(N * L)$	$O(N)$

Complexity Comparison of Trie and B+Tree

	Patricia Trie	B+Tree
Target Use Case	Fast prefix queries	Fast range queries

Complexity Comparison of Trie and B+Tree

	Patricia Trie	B+Tree
Target Use Case	Fast prefix queries	Fast range queries
Memory Efficiency	Less efficient with dissimilar keys	More efficient, stores many keys per node

Complexity Comparison of Trie and B+Tree

	Patricia Trie	B+Tree
Target Use Case	Fast prefix queries	Fast range queries
Memory Efficiency	Less efficient with dissimilar keys	More efficient, stores many keys per node
Range Queries	Supported but not efficient	Highly efficient





Trie





Trie

**Patricia
Trie**

CMYK-Based Tertiaries



IMPACT
PURPLE
(dark purple)
PMS: 267 C
RGB: 95, 36, 159
HEX: #5F249F
CMYK: 81, 99, 0, 0



ELECTRIC
BLUE
(light green-blue)
PMS: 325 C
RGB: 100 204 201
HEX: #64CCC9
CMYK: 54, 0, 20, 0



RAT CAP
(warm yellow)
PMS: 116 C
RGB: 255, 205, 0
HEX: #FFCD00
CMYK: 0, 10, 98, 0



BOLD BLUE
(medium blue)
PMS: 7455 C
RGB: 58, 93, 174
HEX: #3A5DAE
CMYK: 86, 66, 0, 0



CANOPY LIME
(light warm green)
PMS: 2299 C
RGB: 164, 210, 51
HEX: #A4D233
CMYK: 38, 0, 94, 0



NEW HORIZON
(orange-red)
PMS: 7417 C
RGB: 224, 79, 57
HEX: #E04F39
CMYK: 0, 82, 82, 0



OLYMPIC
TEAL
(medium green-blue)
PMS: 321 C
RGB: 0, 140, 149
HEX: #008C95
CMYK: 100, 0, 37, 10

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.



TECH MEDIUM GOLD
HEX: #A4925A
RGB: 164, 146, 90
For use in large headline text in any digital media.



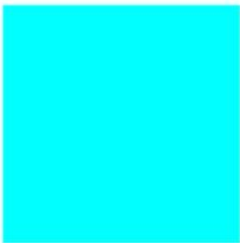
TECH DARK GOLD
HEX: #857437
RGB: 133, 116, 55
For use in small bold text in any digital media.

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.



BRIGHT
PURPLE
(violet)
RGB: 120, 0, 255
HEX: #7800FF



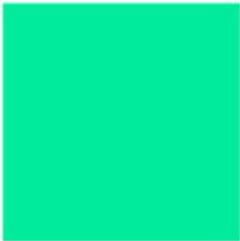
BRIGHT
ELECTRIC
(light teal blue)
RGB: 0, 255, 255
HEX: #00FFFF



BRIGHT BUZZ
(warm yellow)
RGB: 255, 204, 0
HEX: #FFCC00



BRIGHT BLUE
(medium blue)
RGB: 41, 97, 255
HEX: #2961FF



BRIGHT
CANOPY
(cool green)
RGB: 0, 236, 156
HEX: #00EC9C



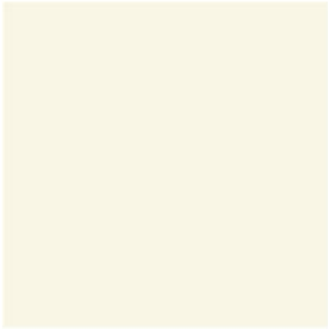
BRIGHT
HORIZON
(red-orange)
RGB: 255, 100, 15
HEX: #FF640F



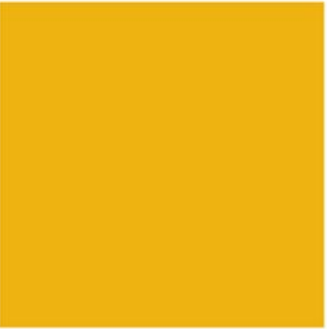
GRAY MATTER
(neutral dark gray)
PMS: 425
HEX: #54585A
RGB: 84, 88, 90
HSL: 200, 3, 34
HSB: 200, 7, 35
CMYK: 48, 29, 26, 76



PI MILE
(light warm gray)
PMS: 427
HEX: #D6DBD4
RGB: 214, 219, 212
HSL: 103, 9, 85
HSB: 103, 3, 86
CMYK: 9, 4, 10, 8



DIPLOMA
(light ivory yellow)
PMS: 7499
HEX: #F9F6E5
RGB: 249, 246, 229
HSL: 51, 63, 94
HSB: 51, 8, 98
CMYK: 1, 1, 10, 1



BUZZ GOLD
PMS: 124
HEX: #EAAA00
RGB: 234, 170, 0
HSL: 44, 100, 46
HSB: 44, 100, 92
CMYK: 0, 30, 100, 0



Inverted Index





Inverted Index



**Inverted
Index**

**Web-Scale
Search**

Inverted Index

Map words to the positions in which they appear in a document.

Word	Positions
russia	10, 20
napolean	25

Proximity Search

distance("russia", "napoleon") less than ten words

Word	Positions
russia	10, 20
napoleon	25

Library Search

Map words to the documents in which they appear.

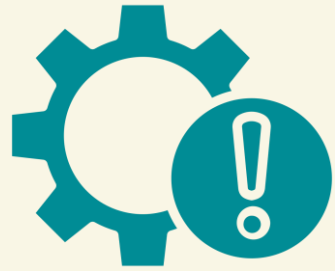
Word	Document ID, Position
russia	(1, 10), (1, 20)
napoleon	(1 , 25), (2 , 30), (3 , 50)

Why Inverted Index?

**Inverted
Index**



Internals



Why Inverted Index?

**Inverted
Index**



**Text
Data**

**Word
Search**

**Proximity
Search**

Internals



Why Inverted Index?

Inverted Index

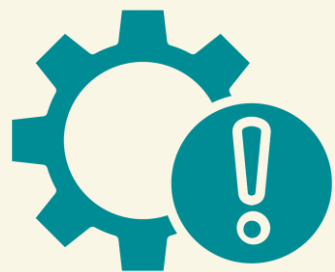


**Text
Data**

**Word
Search**

**Proximity
Search**

Internals



**Hash Table
Foundation**

**Keys are
Words**

**Values are
Doc IDs,
Positions**

Structure of an Inverted Index

Key

“russia”

Value

{1: {5, 20}, 2: {10}}

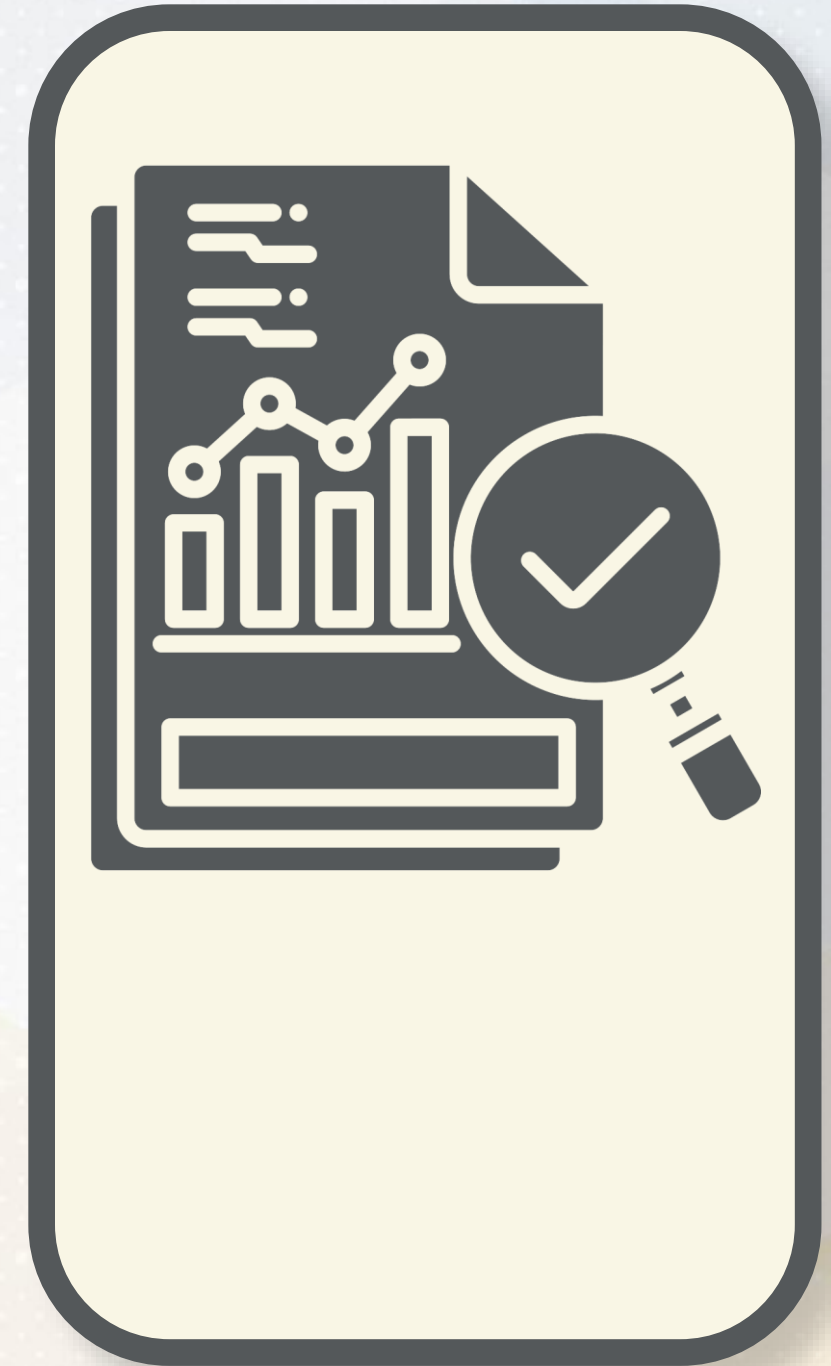
```
std::unordered_map<  
    std::string,  
    std::unordered_map<int, std::vector<int>>  
>
```



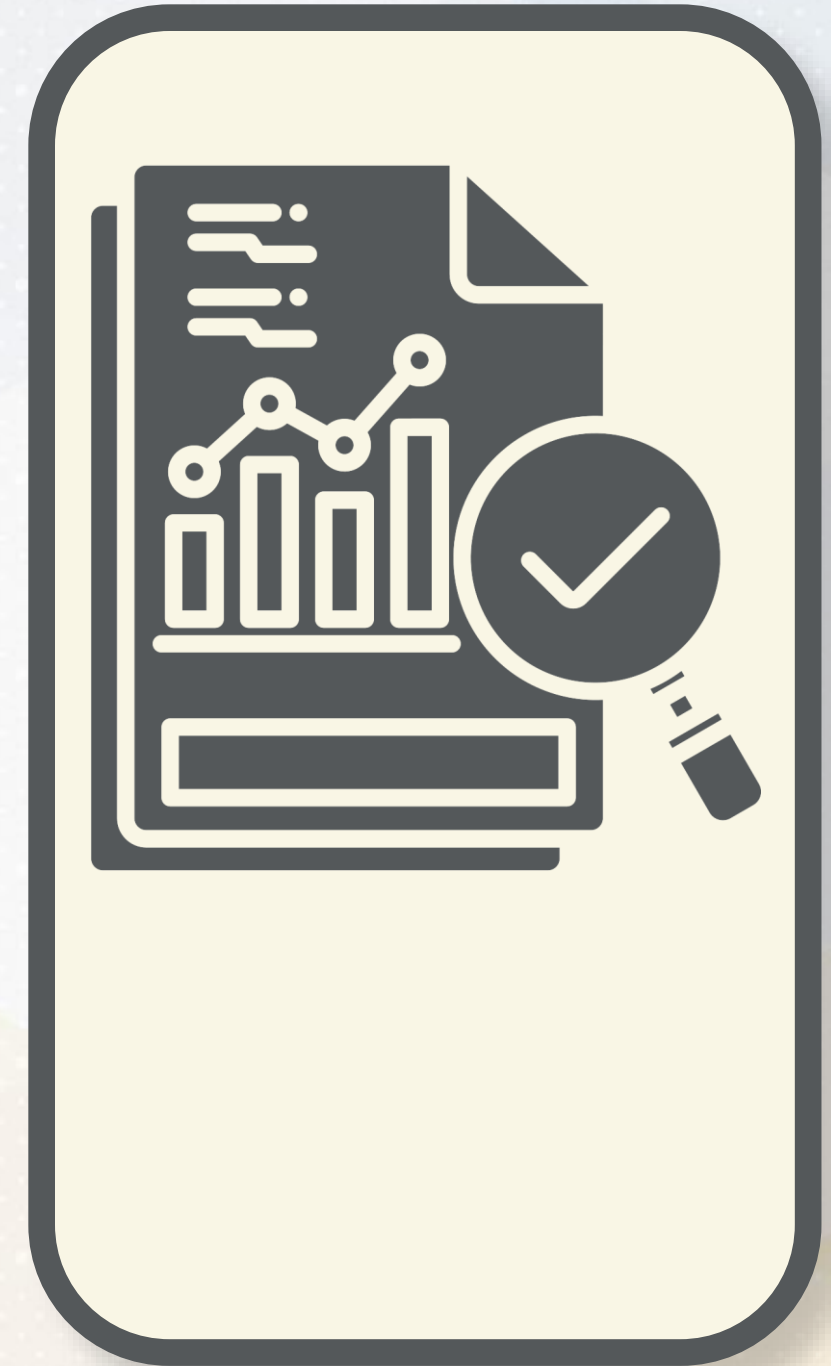
Core Functions



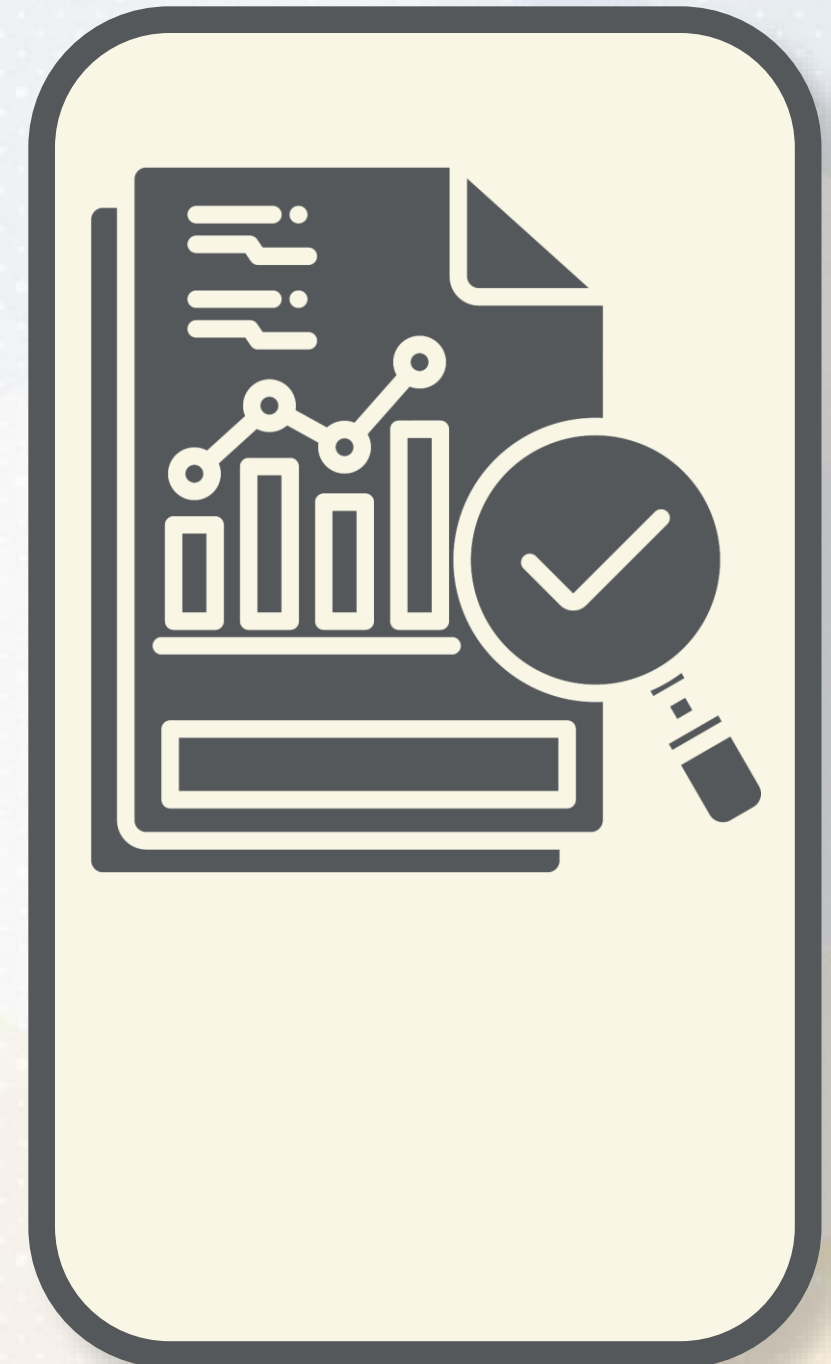
Core Functions



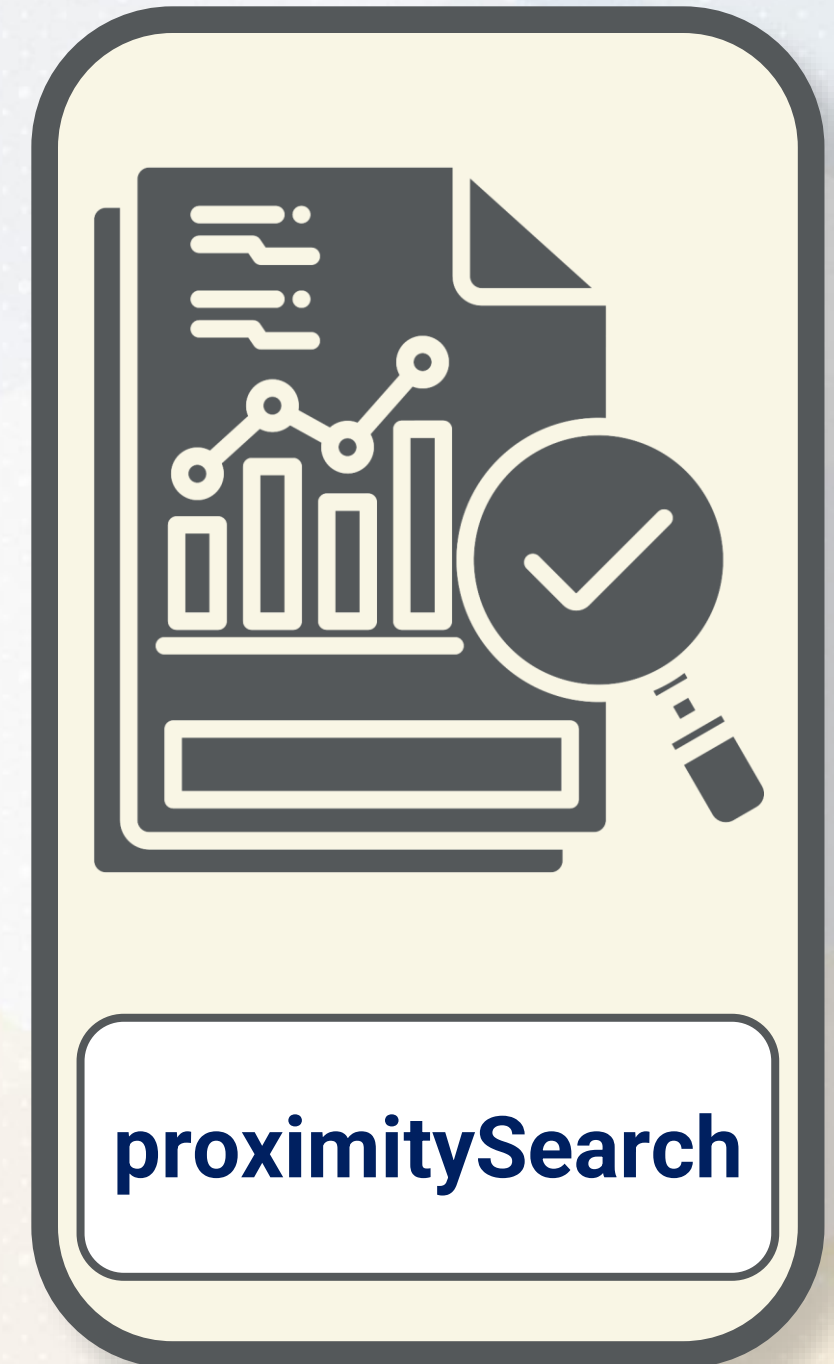
Core Functions



Core Functions



Core Functions



Adding Documents to the Inverted Index



```
void addDocument(int docID, const std::string &content) {  
    documents.push_back(content);  
    std::vector<std::string> words = split(toLower(content));  
    for (size_t i = 0; i < words.size(); ++i) {  
        index[words[i]][docID].push_back(i);  
    }  
}
```

Adding Documents to the Inverted Index



Add Word to
unordered_map

Map to Doc ID
and Word
Position

Convert word to
lowercase if
needed

```
void addDocument(int docID, const std::string &content) {  
    documents.push_back(content);  
    std::vector<std::string> words = split(toLower(content));  
    for (size_t i = 0; i < words.size(); ++i) {  
        index[words[i]][docID].push_back(i);  
    }  
}
```


Retrieving Documents Containing a Term



```
std::unordered_set<int> getDocuments(  
    const std::string &word) {  
    std::string lowerWord = toLower(word);  
    if (index.find(lowerWord) != index.end()) {  
        std::unordered_set<int> docIDs;  
        for (const auto &entry : index[lowerWord]) {  
            docIDs.insert(entry.first);  
        }  
        return docIDs;  
    }  
    return {};  
}
```


Retrieving Documents Containing a Term



Converts input
word to lowercase



```
std::unordered_set<int> getDocuments(  
    const std::string &word) {  
    std::string lowerWord = toLower(word);  
    if (index.find(lowerWord) != index.end()) {  
        std::unordered_set<int> docIDs;  
        for (const auto &entry : index[lowerWord]) {  
            docIDs.insert(entry.first);  
        }  
        return docIDs;  
    }  
    return {};  
}
```

Retrieving Documents Containing a Term



Converts input
word to lowercase



Returns list of
document IDs

```
std::unordered_set<int> getDocuments(  
    const std::string &word) {  
    std::string lowerWord = toLower(word);  
    if (index.find(lowerWord) != index.end()) {  
        std::unordered_set<int> docIDs;  
        for (const auto &entry : index[lowerWord]) {  
            docIDs.insert(entry.first);  
        }  
        return docIDs;  
    }  
    return {};  
}
```

Proximity Search: Finding Words Near Each Other

```
std::unordered_set<int> proximitySearch(const std::string &word1,
                                       const std::string &word2, int k)
{
    ...
    if (index.find(lowerWord1) != index.end() && index.find(lowerWord2) != index.end()) {
        for (const auto &entry1 : index[lowerWord1]) {
            int docID = entry1.first;
            if (index[lowerWord2].find(docID) != index[lowerWord2].end()) {
                const auto &positions1 = entry1.second;
                const auto &positions2 = index[lowerWord2].at(docID);
                for (int pos1 : positions1) { for (int pos2 : positions2) {
                    if (std::abs(pos1 - pos2) <= k) { result.insert(docID); break; }
                }
            }
        }
        ...
    }
}
```


Benefits of Inverted Index



Benefits of Inverted Index



Efficient

Fast lookup for individual words

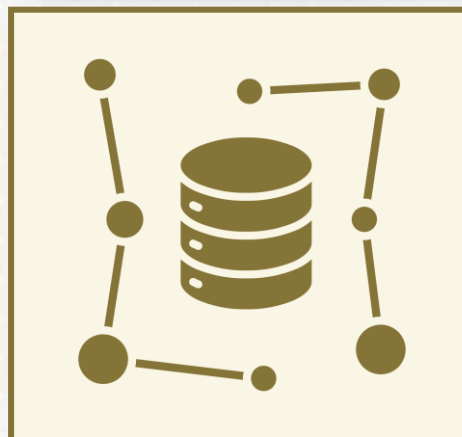


Benefits of Inverted Index



Efficient

Fast lookup for individual words



Complexity

Proximity search and Boolean queries

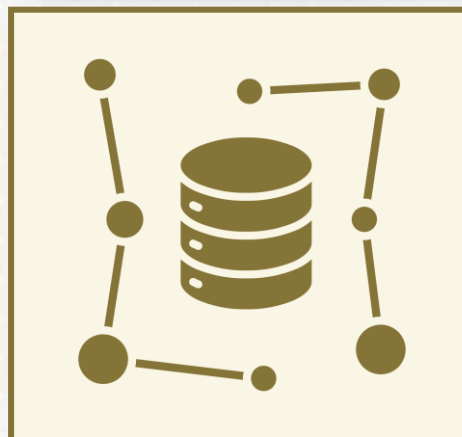


Benefits of Inverted Index



Efficient

Fast lookup for individual words



Complexity

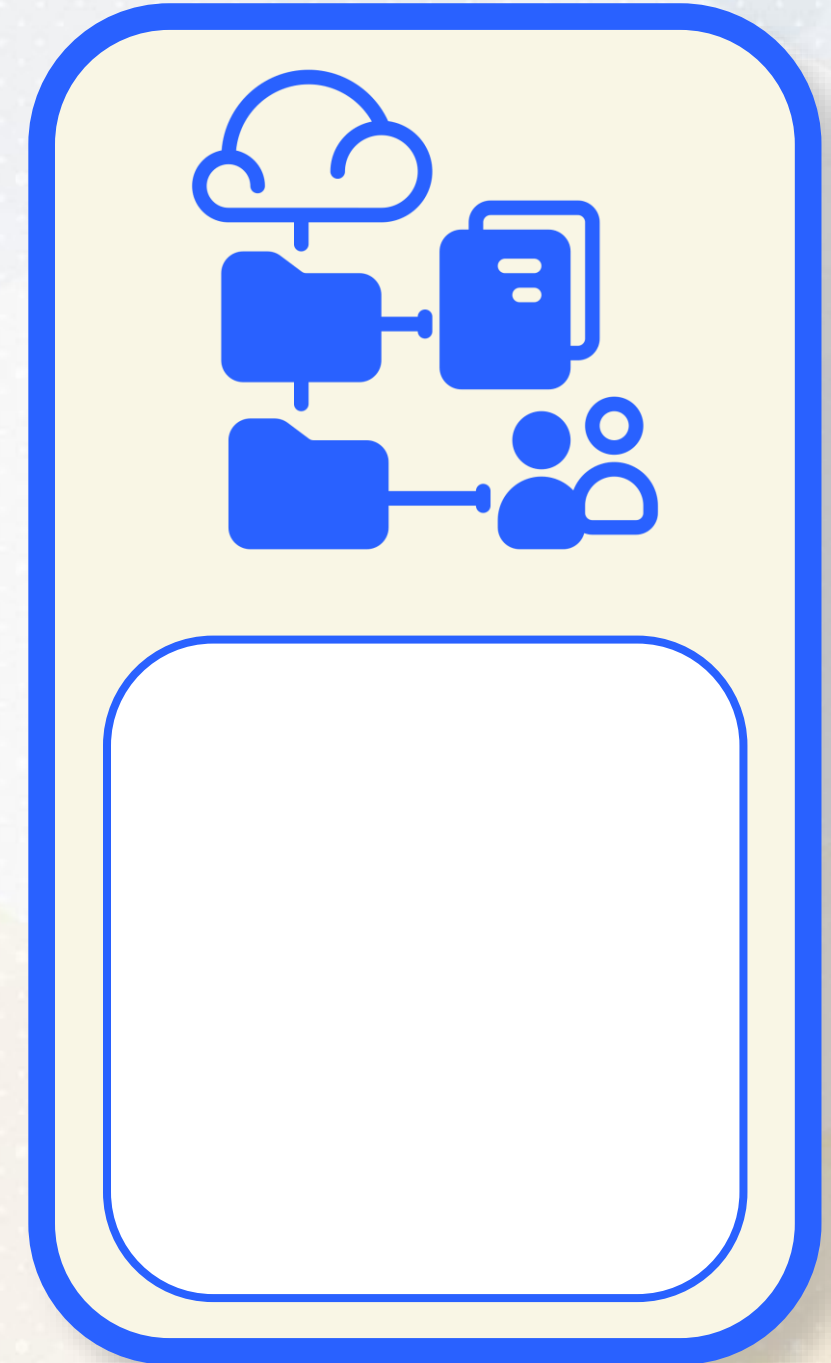
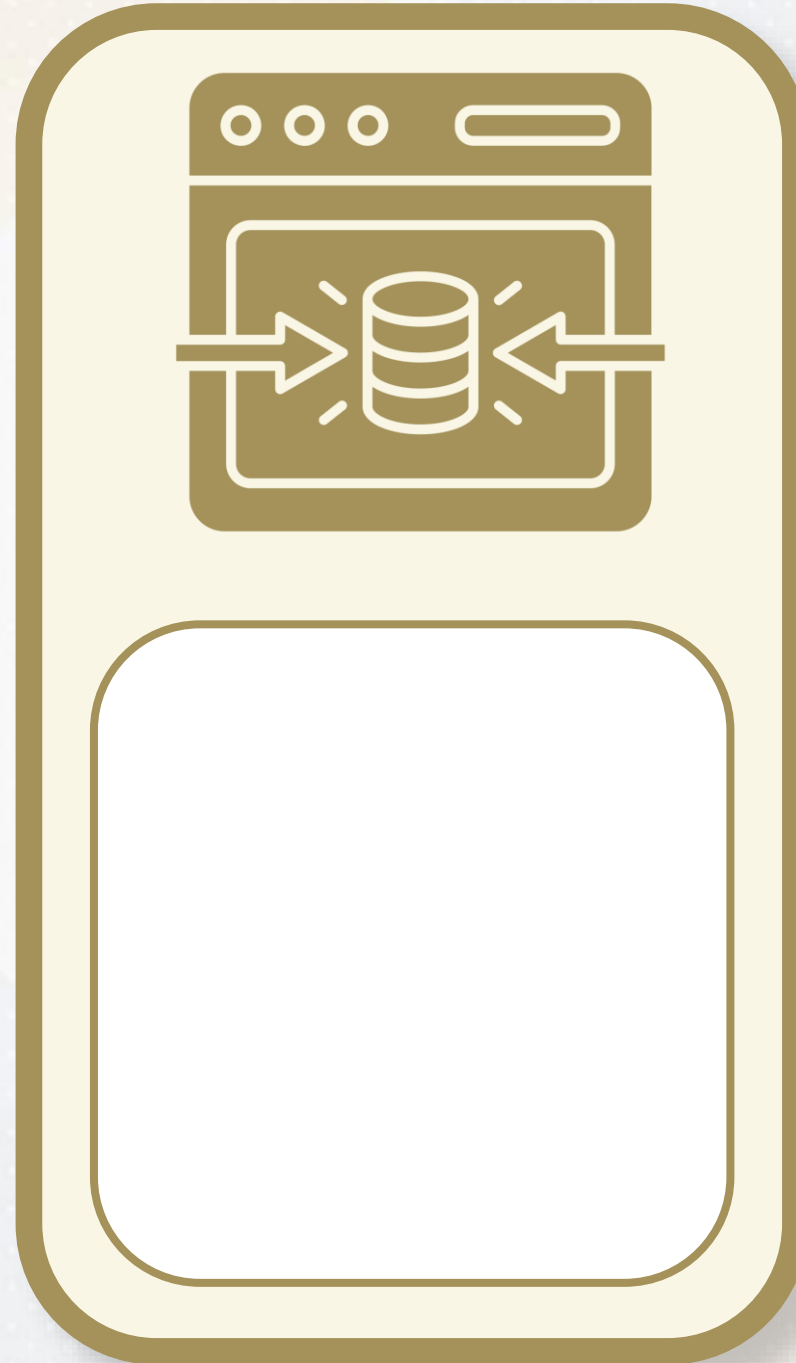
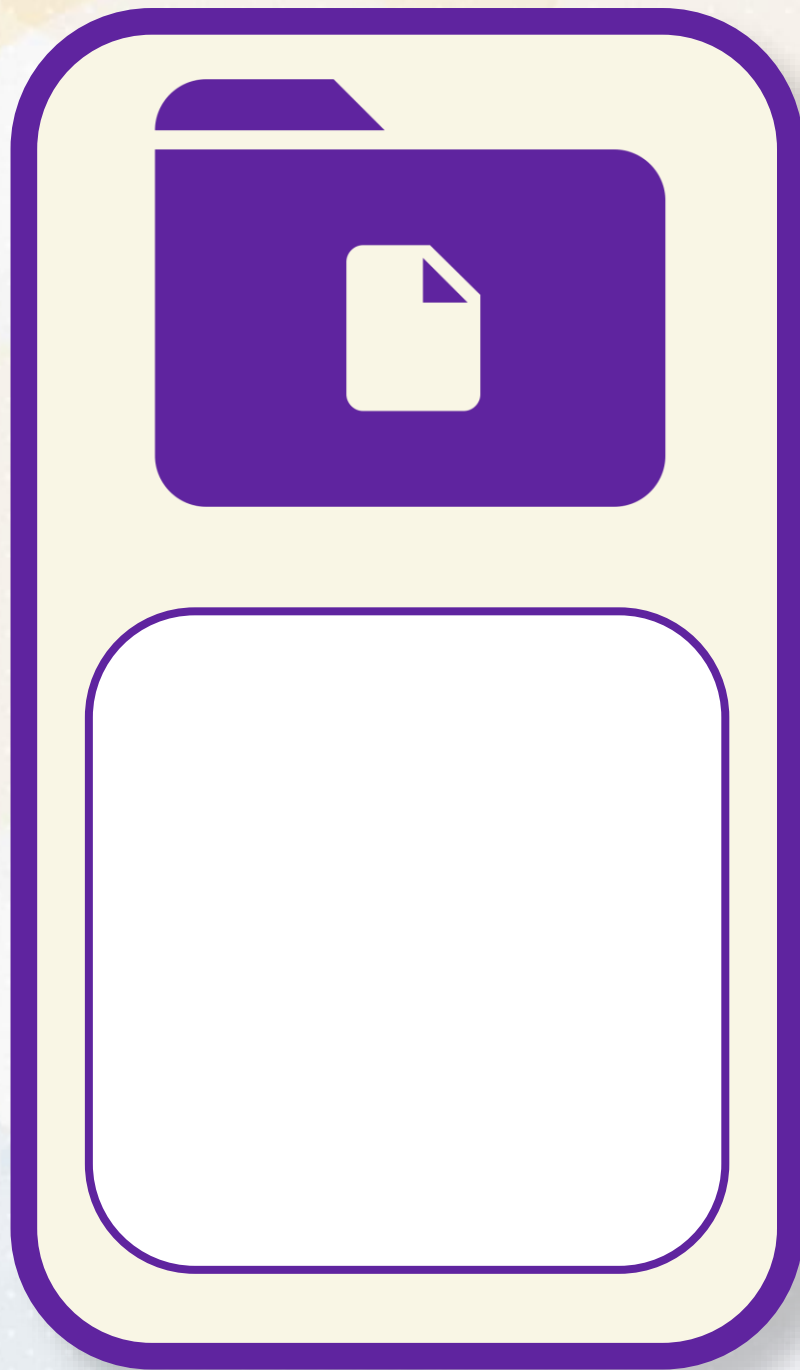
Proximity search and Boolean queries



Scalable

Scales to large document collections

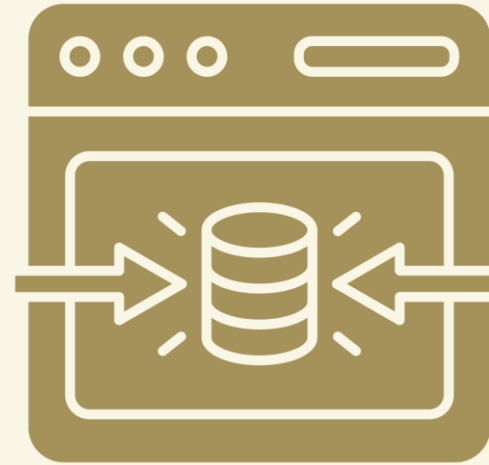
Challenges with Inverted Index



Challenges with Inverted Index



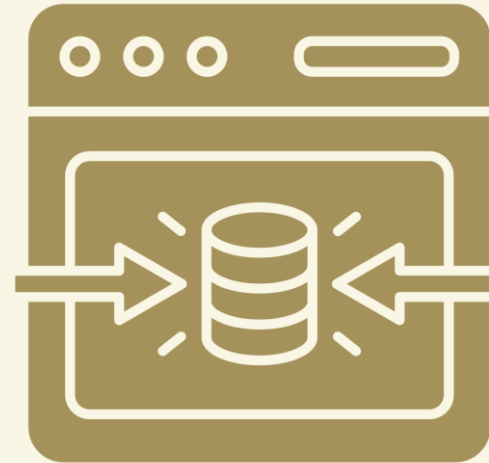
Large
Storage
Footprint



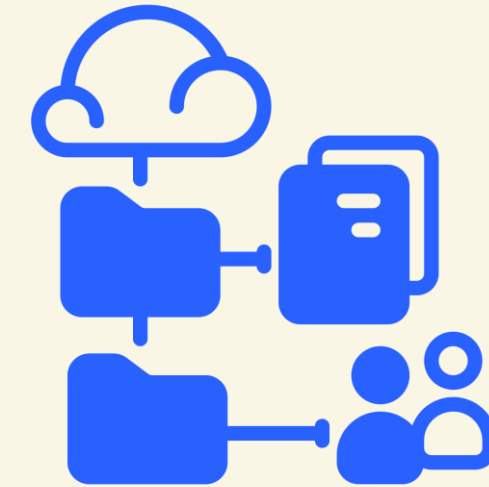
Challenges with Inverted Index



Large
Storage
Footprint



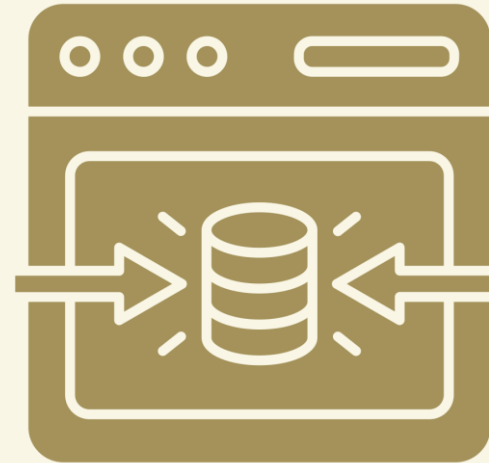
Compression:
Delta
Encoding



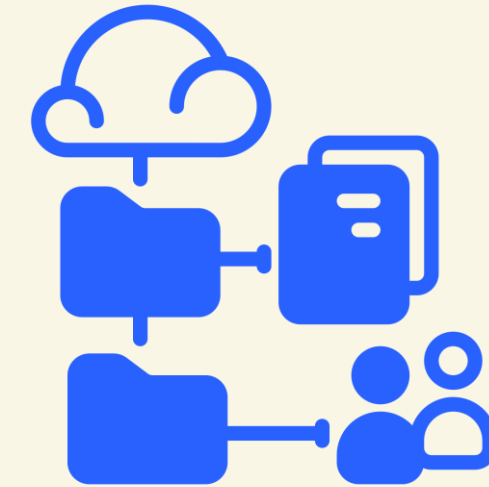
Challenges with Inverted Index



Large
Storage
Footprint



Compression:
Delta
Encoding



Disk-Based
Storage
Management



Web-Scale Search



Stanford Digital Library Project (1995)

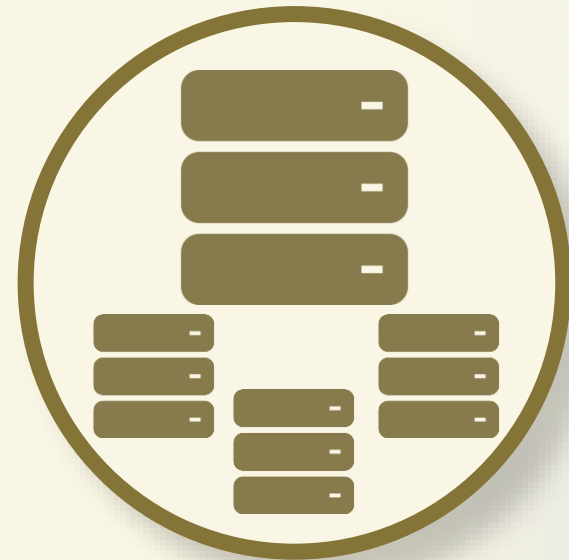
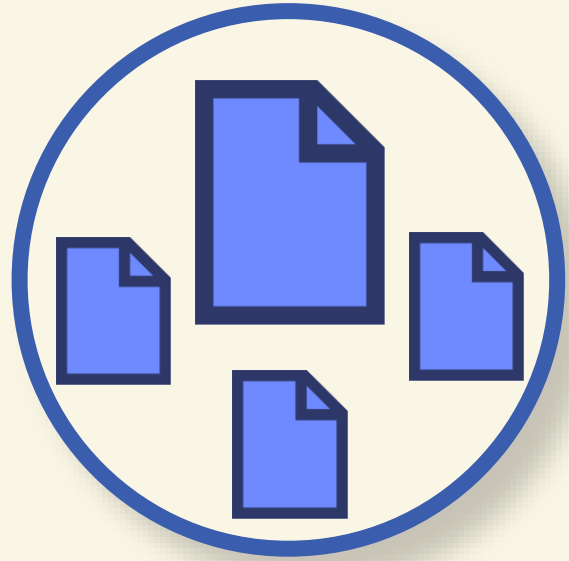


Héctor García-Molina

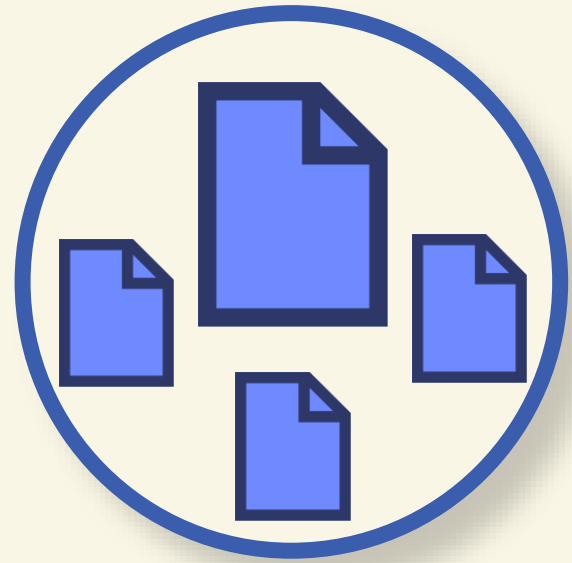


Sergey Brin Larry Page

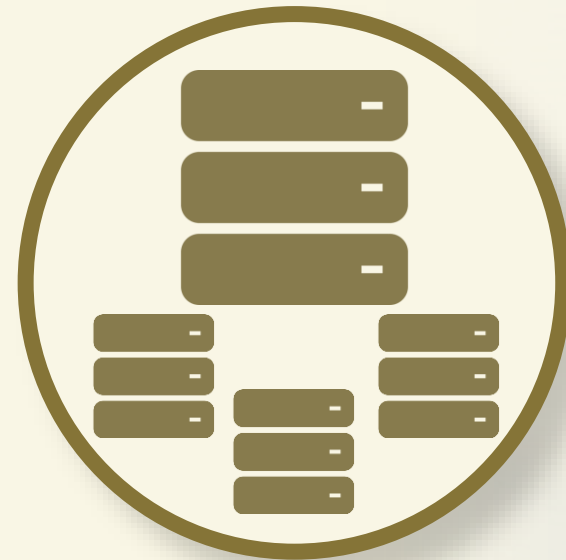
Web-Scale Search



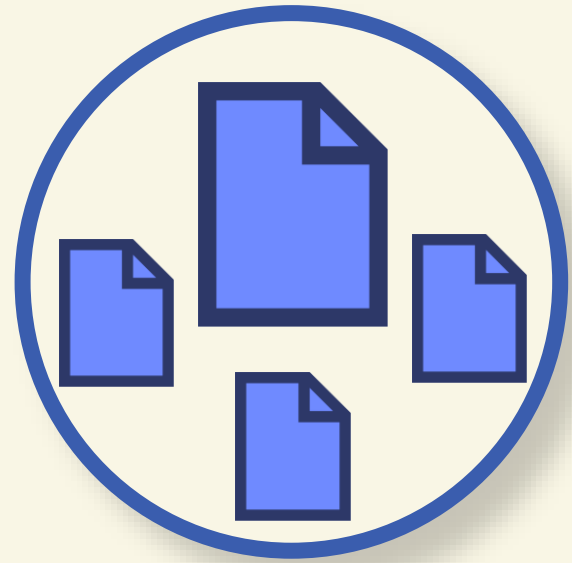
Web-Scale Search



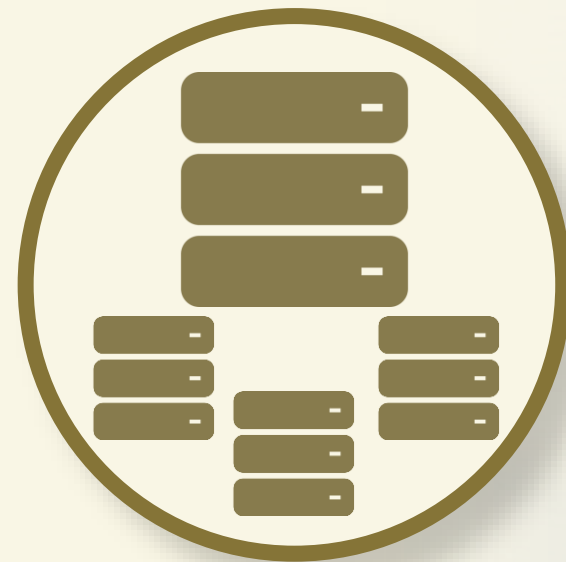
Sharding



Web-Scale Search



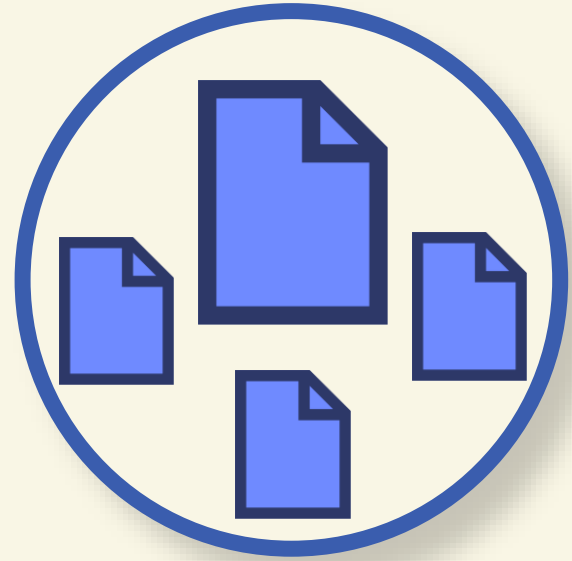
Sharding



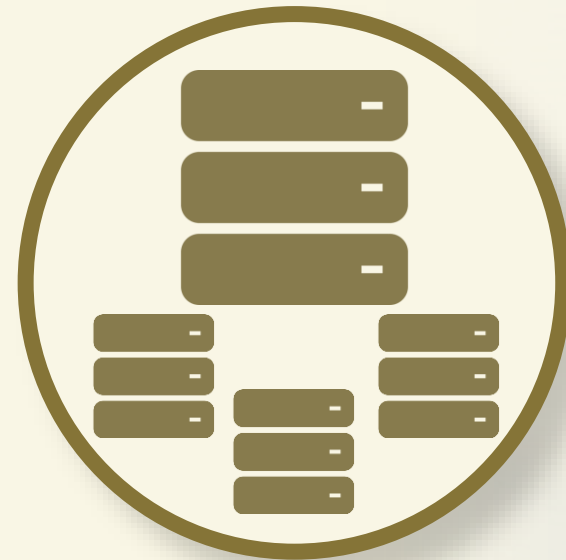
Replication



Web-Scale Search



Sharding



Replication



Web-Scale Inverted Index

Distributed Inverted Index



Word-Based Sharding

Shards (subset of Words)	Replicas
Shard 1 (Words 1...100)	Servers A, B
Shard 2 (Words 101 ... 200)	Servers C, D
...	...

Document Ranking



**Relevant
Ranking**



**Ranking
Algorithm**



**Indexing
and Ranking**



Document Ranking



**Relevant
Ranking**

Documents Prioritized by
Relevance



**Ranking
Algorithm**



**Indexing
and Ranking**

Document Ranking



**Relevant
Ranking**

Documents Prioritized by
Relevance



**Ranking
Algorithm**

PageRank or
Machine Learning



**Indexing
and Ranking**

Document Ranking



**Relevant
Ranking**

Documents Prioritized by
Relevance



**Ranking
Algorithm**

PageRank or
Machine Learning



**Indexing
and Ranking**

Inverted Index
Narrows Down

The Future

Insert full-screen video descriptive of the language.

1. <https://www.gettyimages.com/detail/video/filing-documents-computer-file-digitization-file-folder-stock-footage/2149401020?adppopup=true>
2. <https://www.gettyimages.com/detail/video/business-stock-market-charts-and-data-numbers-stock-footage/1777547787?adppopup=true>
3. <https://www.gettyimages.com/detail/video/values-stock-footage/150487436?adppopup=true>



Inverted Index



**Inverted
Index**

**Web-Scale
Search**

R-Tree



R-Tree



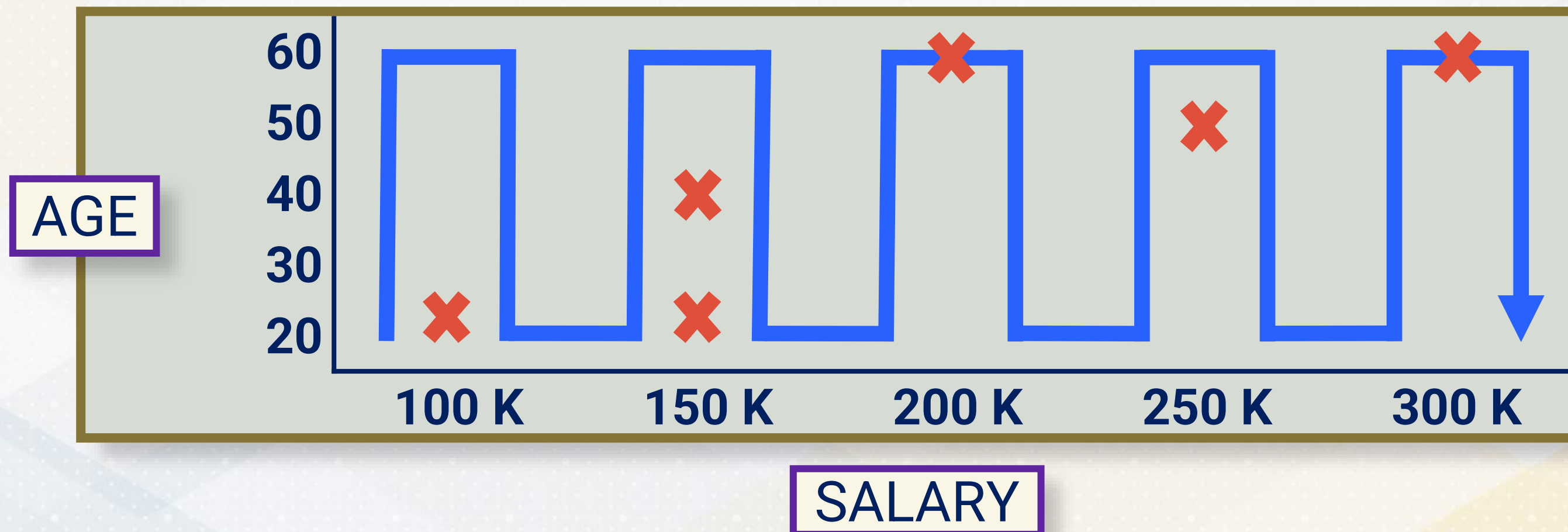
R-Tree

**Nearest
Neighbor
Query**

Limitations of B+Tree

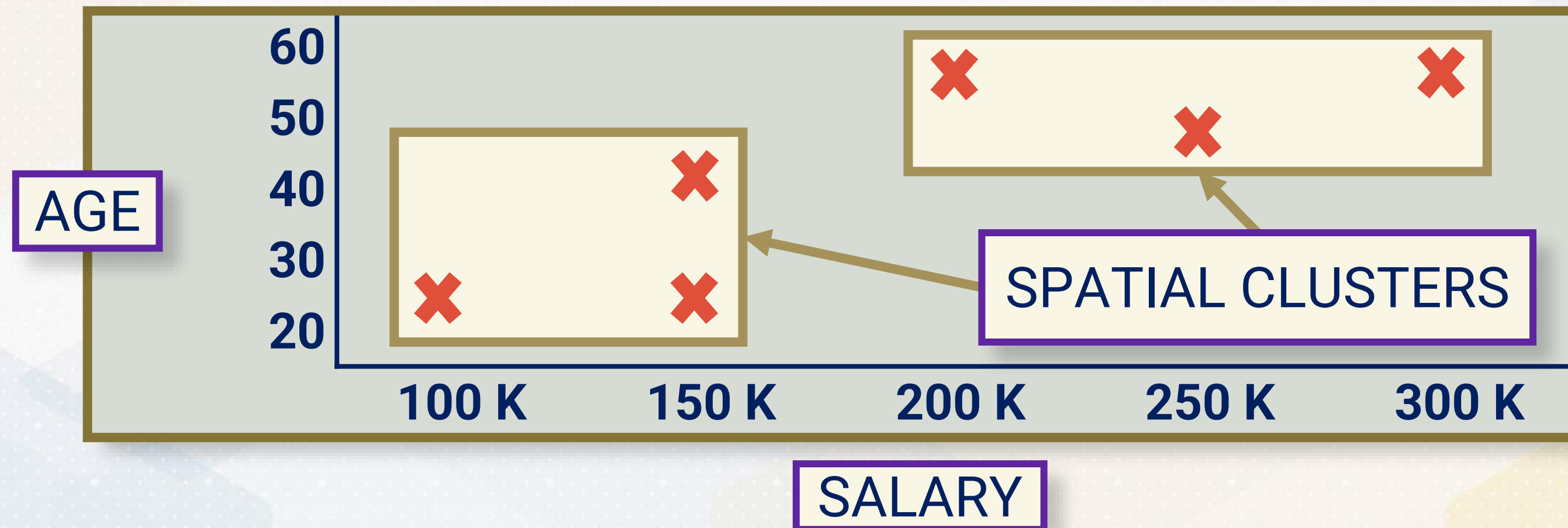


Designed for Single-Dimensional Indexing

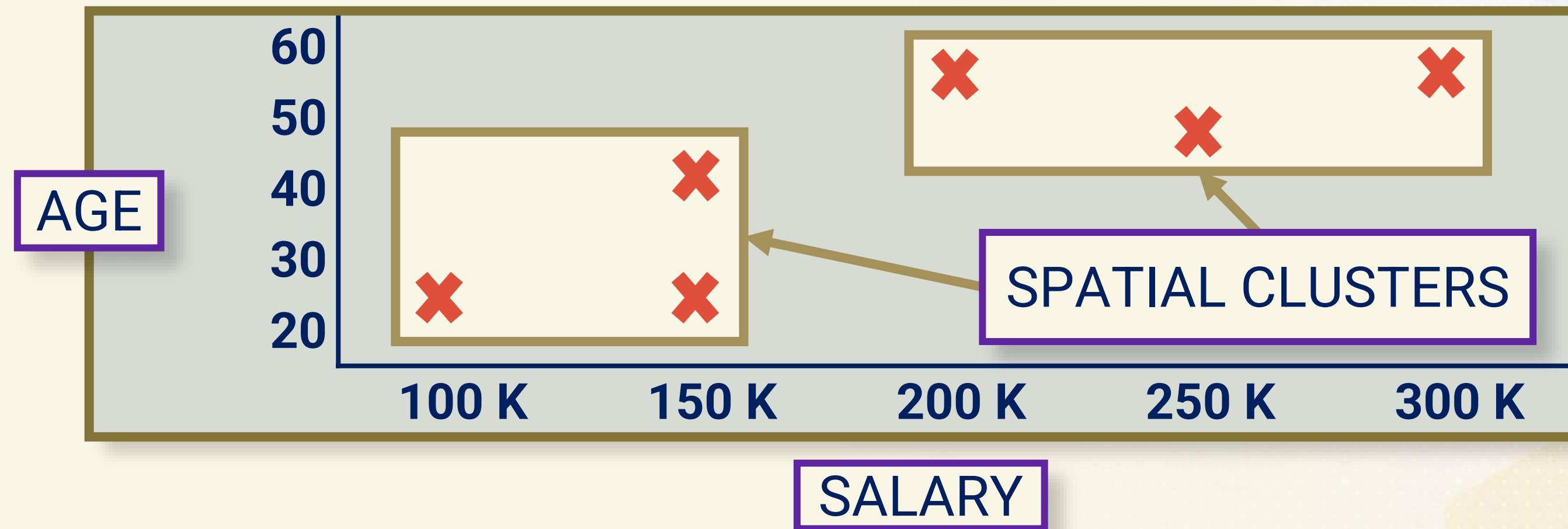


R-Tree: A Multidimensional Index

R-Tree groups multi-dimensional keys based on spatial proximity

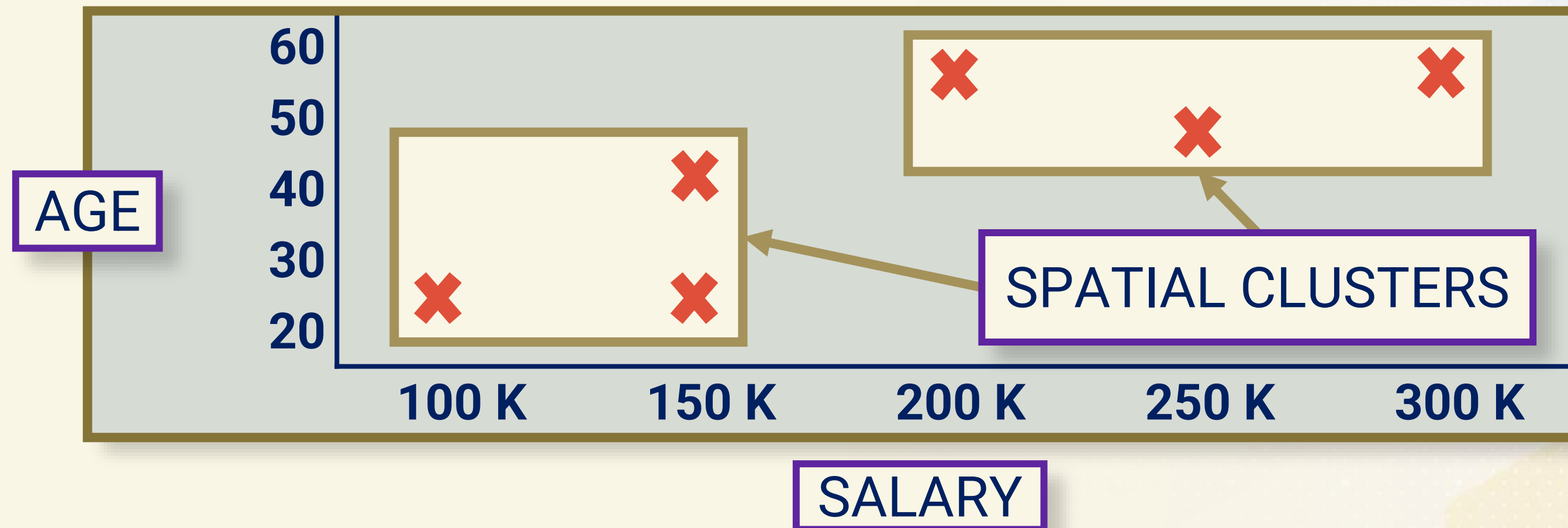


R-Tree: A Multidimensional Index



R-Tree: A Multidimensional Index

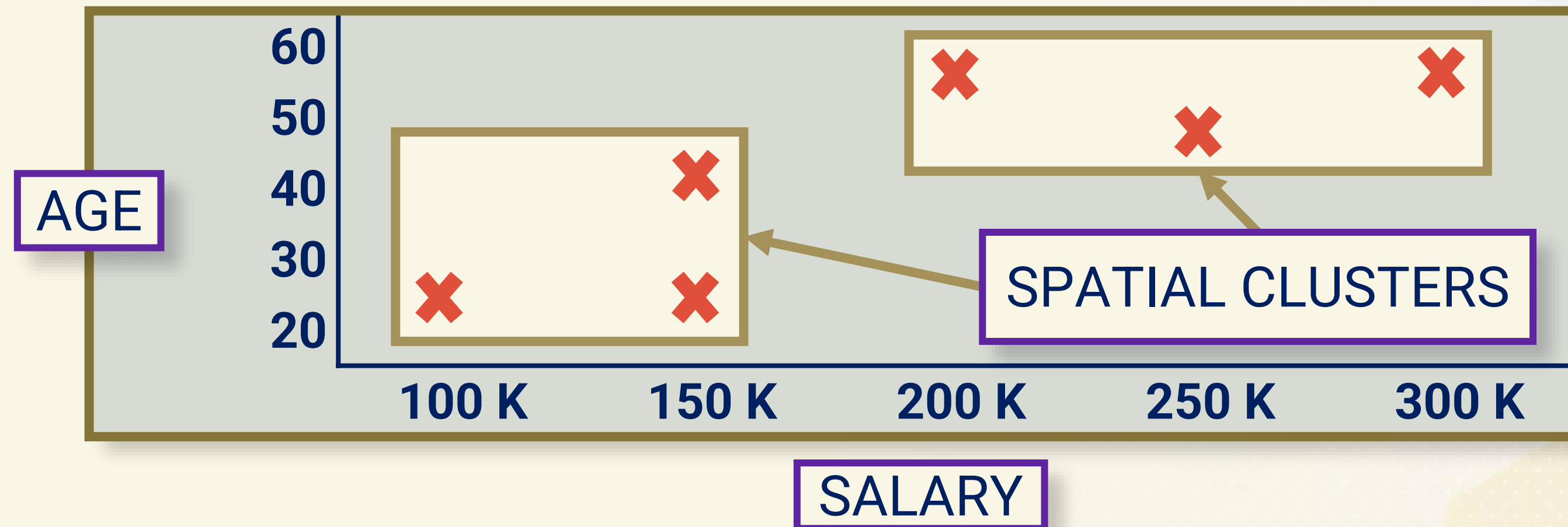
(Salary, Age): Point



R-Tree: A Multidimensional Index

(Salary, Age): Point

Clusters: Bounding Rectangles

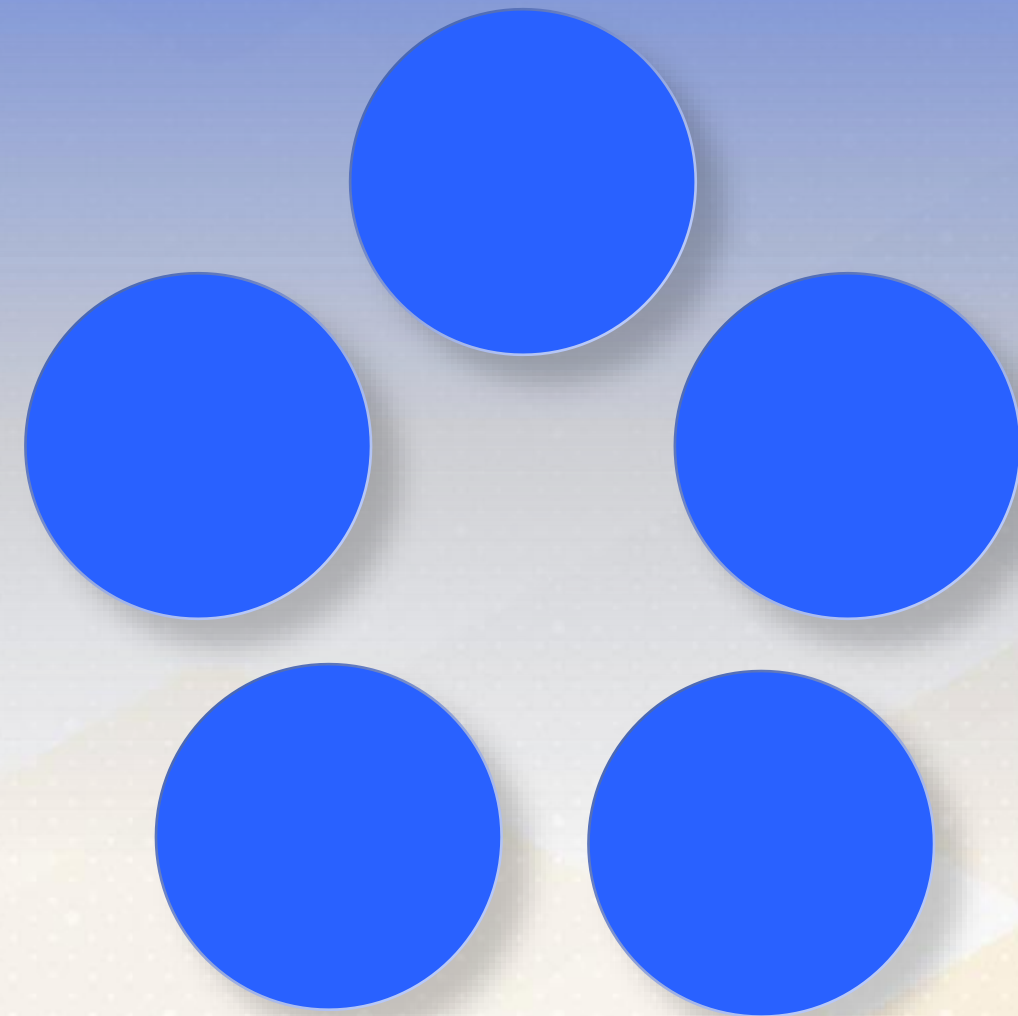


Supported Queries

Spatial Queries



Nearest Neighbor Queries

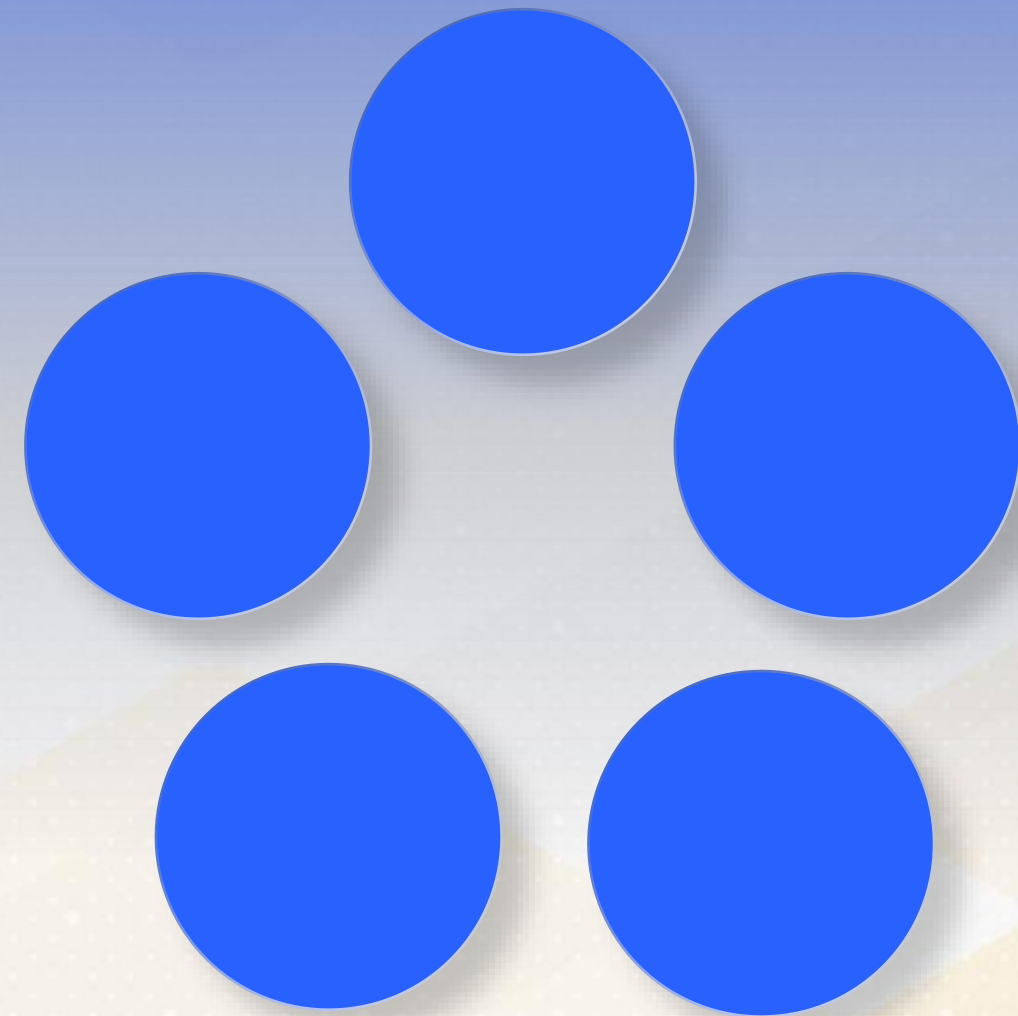


Supported Queries

Spatial Queries



Nearest Neighbor Queries

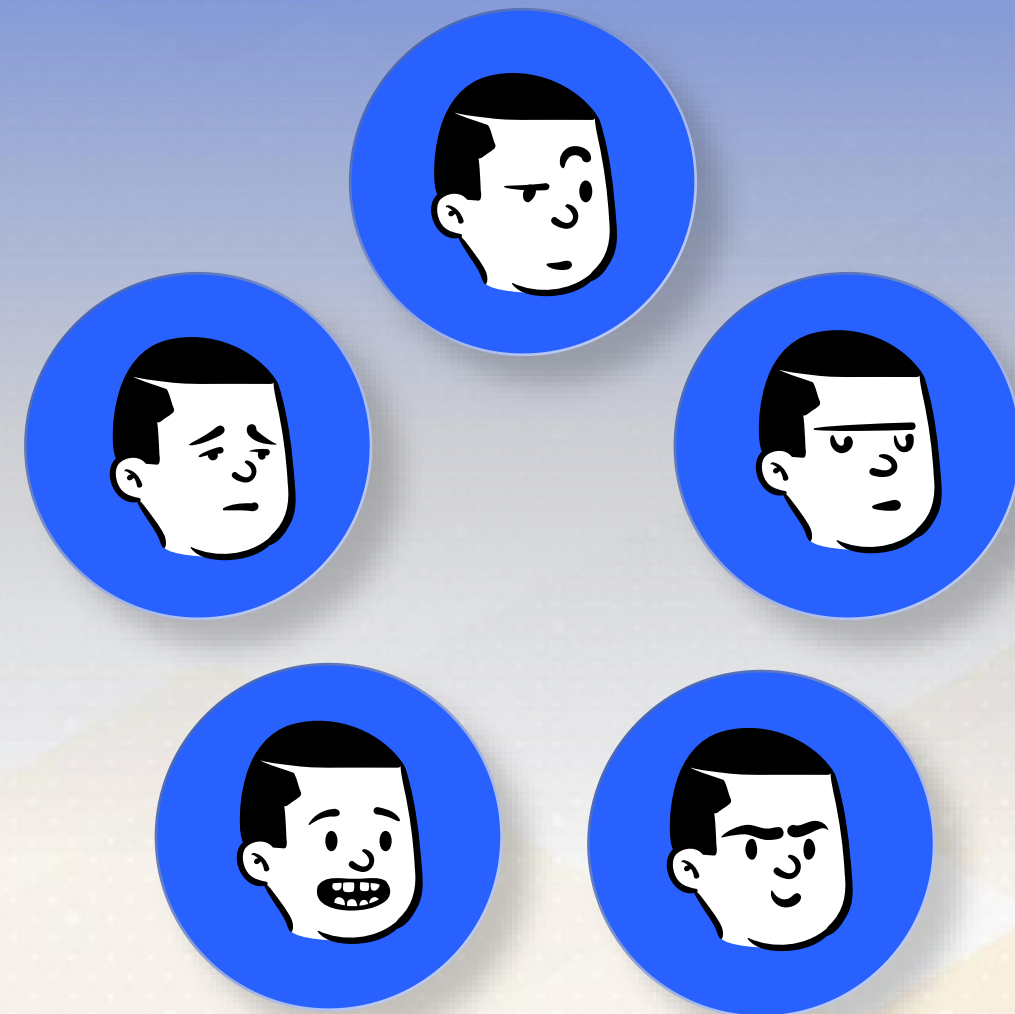


Supported Queries

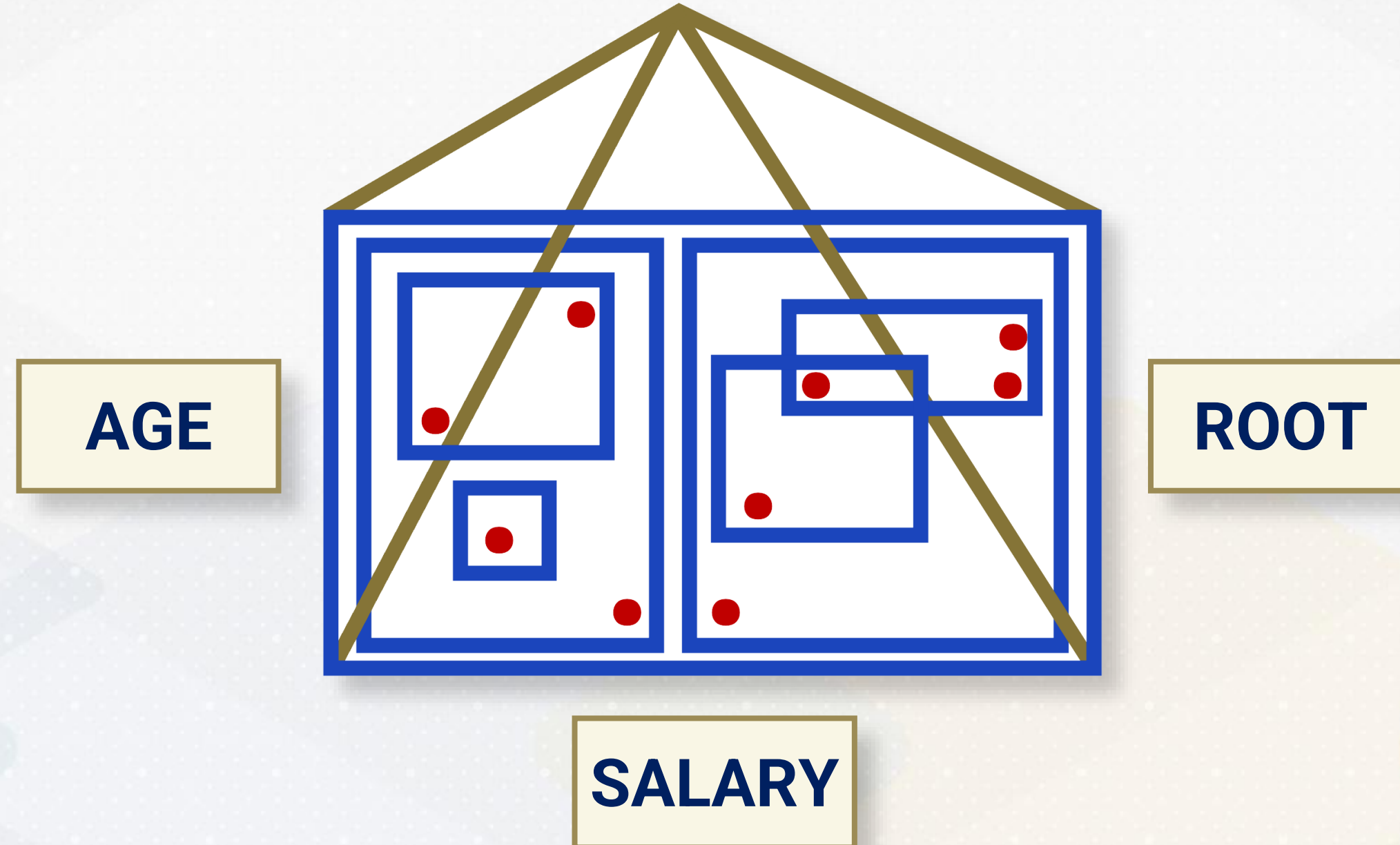
Spatial Queries



Nearest Neighbor Queries



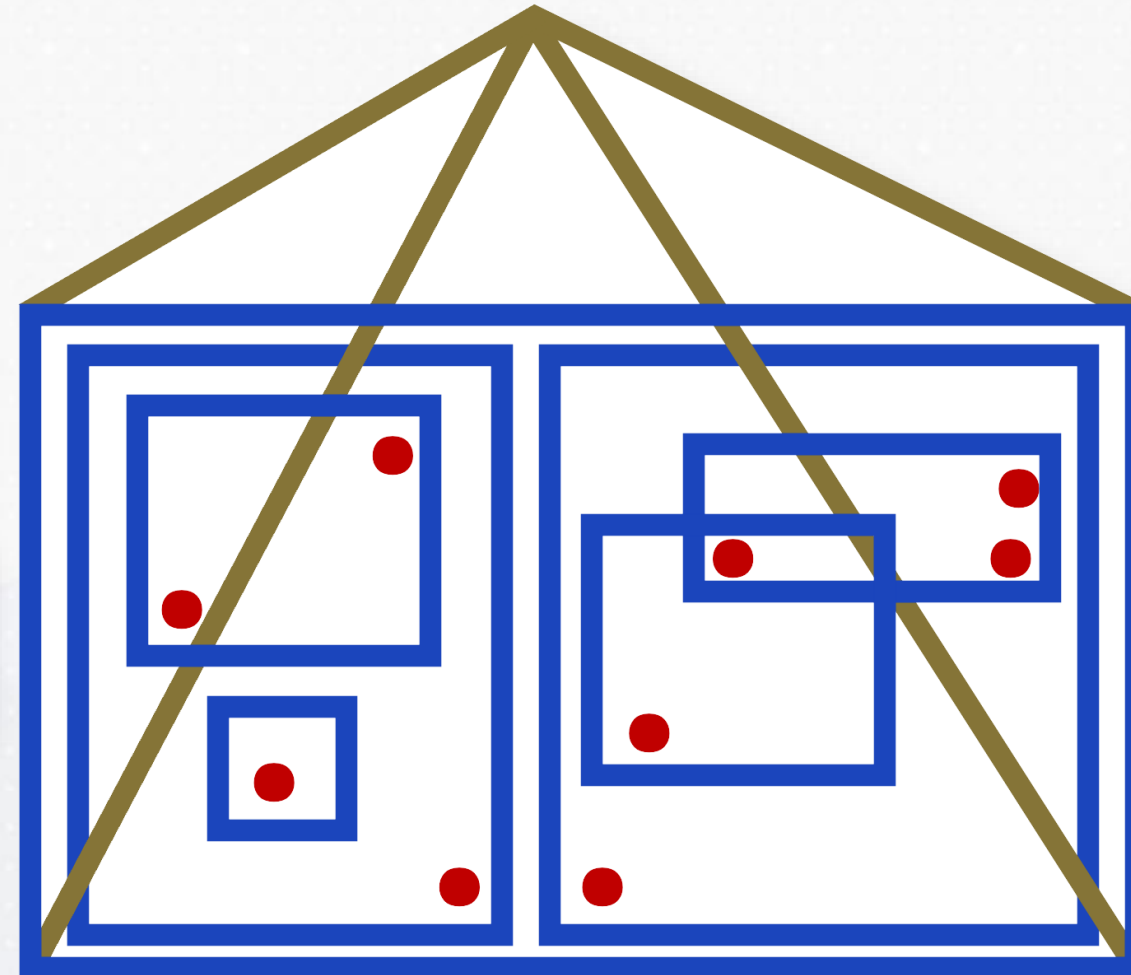
Key Characteristics



Key Characteristics



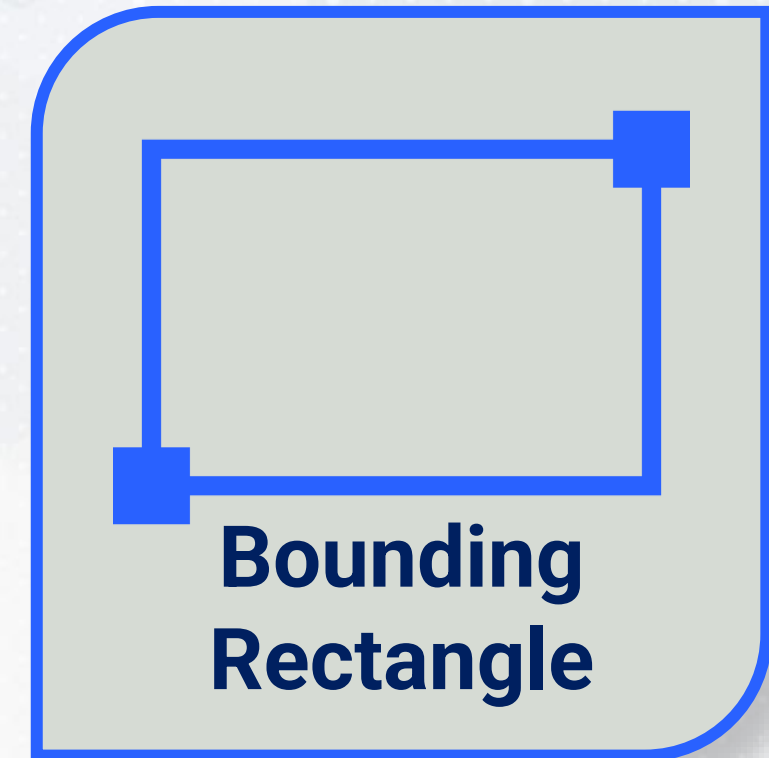
AGE



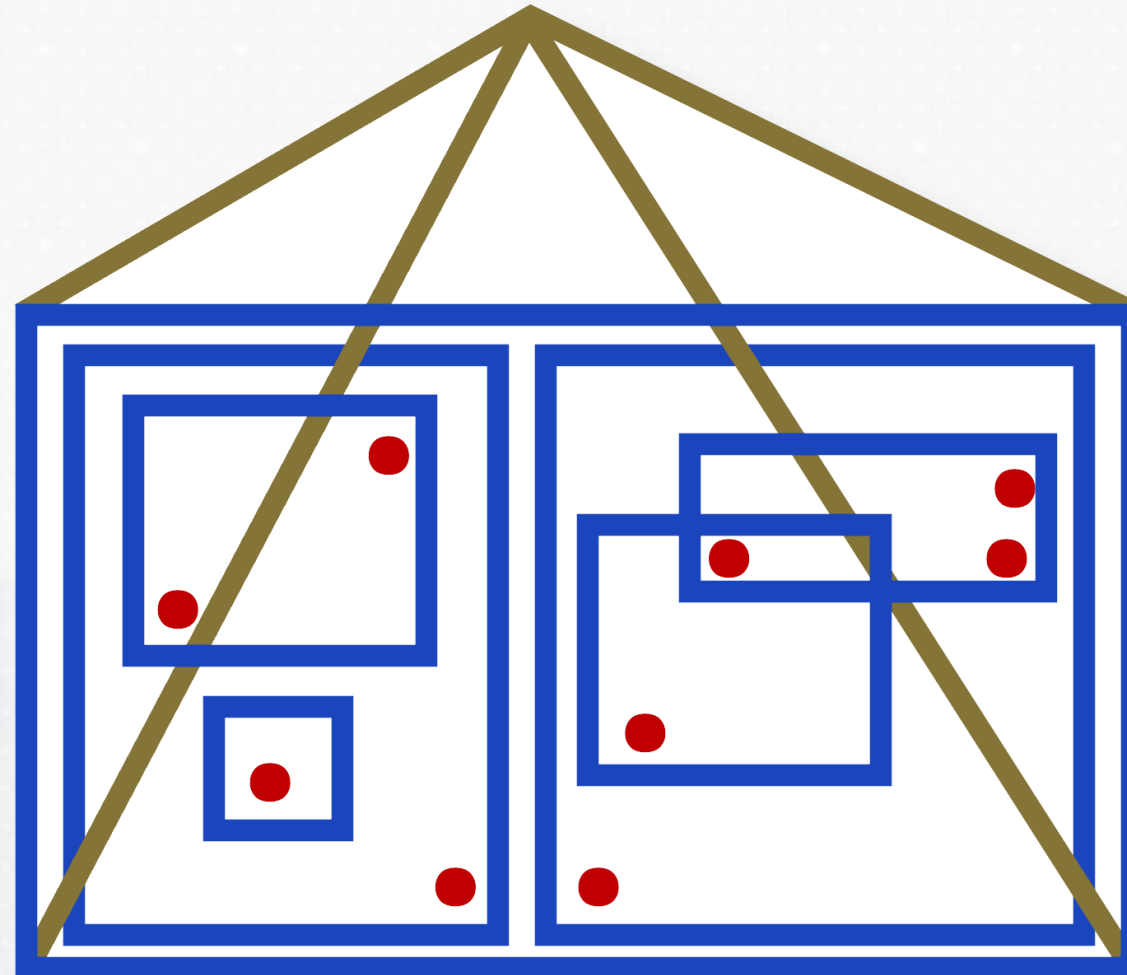
ROOT

SALARY

Key Characteristics



AGE

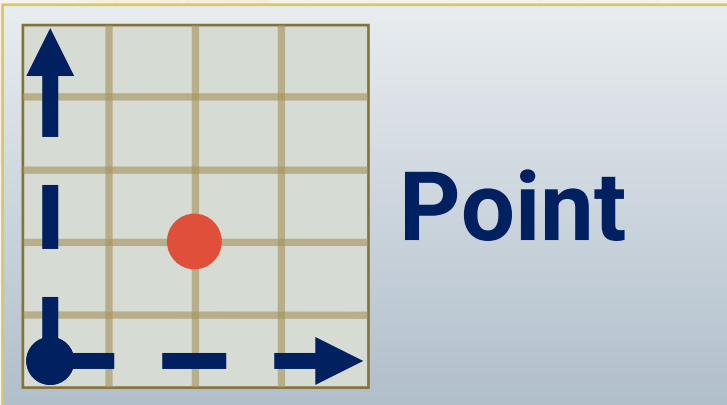


ROOT

SALARY

R-Tree Data Types

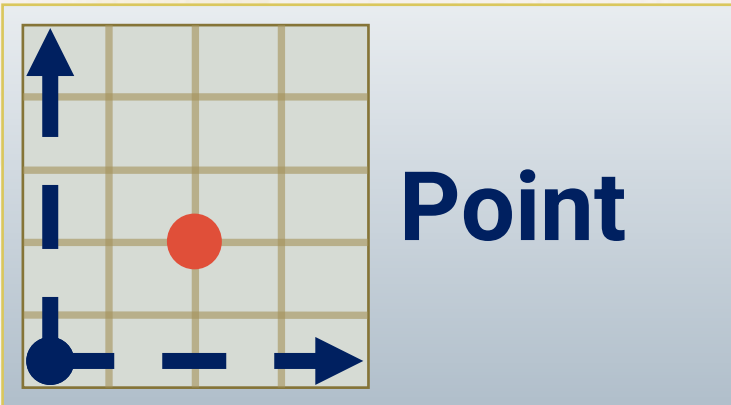
R-Tree Data Types



2D coordinate

Specific
location
in space

R-Tree Data Types



Point

2D coordinate

Specific
location
in space



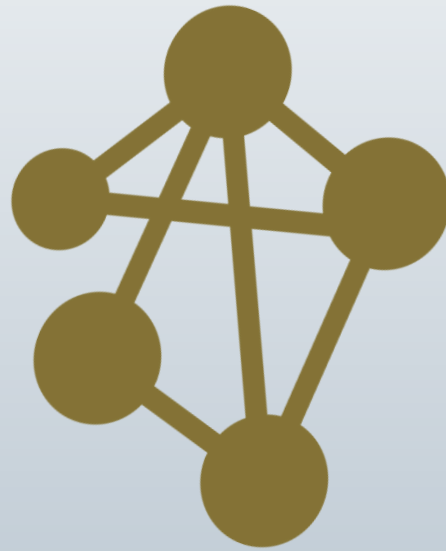
Rectangle

Defines a
bounding box

Encloses points
or other
rectangles

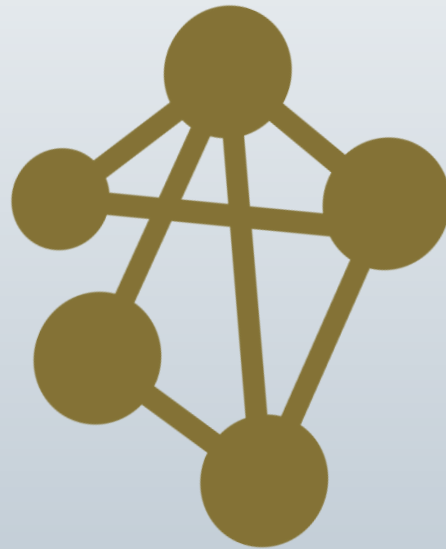
R-Tree Nodes

R-Tree Nodes



**R-Tree
Node**

R-Tree Nodes

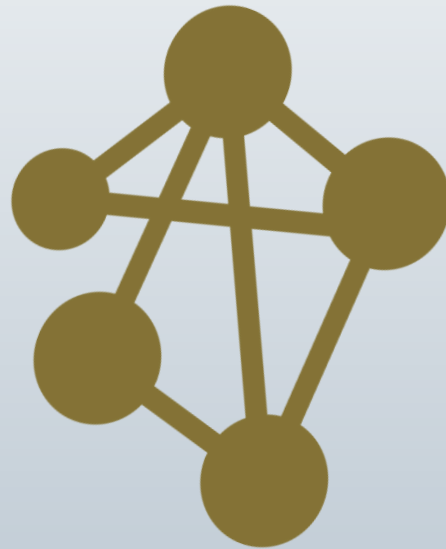


**R-Tree
Node**

Leaf Node

Contains
Points;
Lowest
Level of Tree

R-Tree Nodes



**R-Tree
Node**

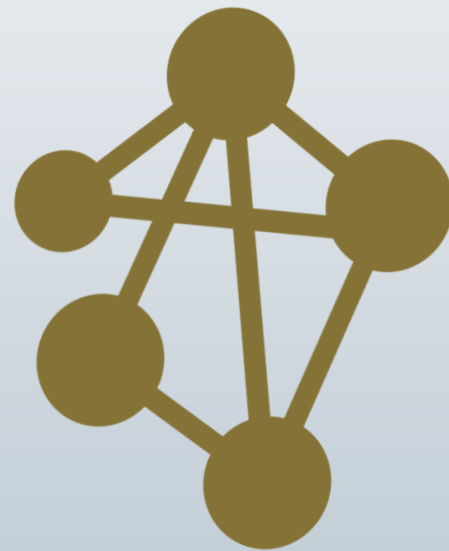
Leaf Node

Contains
Points;
Lowest
Level of Tree

Inner Node

Contains
Child Nodes
and their
Bounding
Rectangles

R-Tree Nodes



**R-Tree
Node**

Leaf Node

Contains
Points;
Lowest
Level of Tree

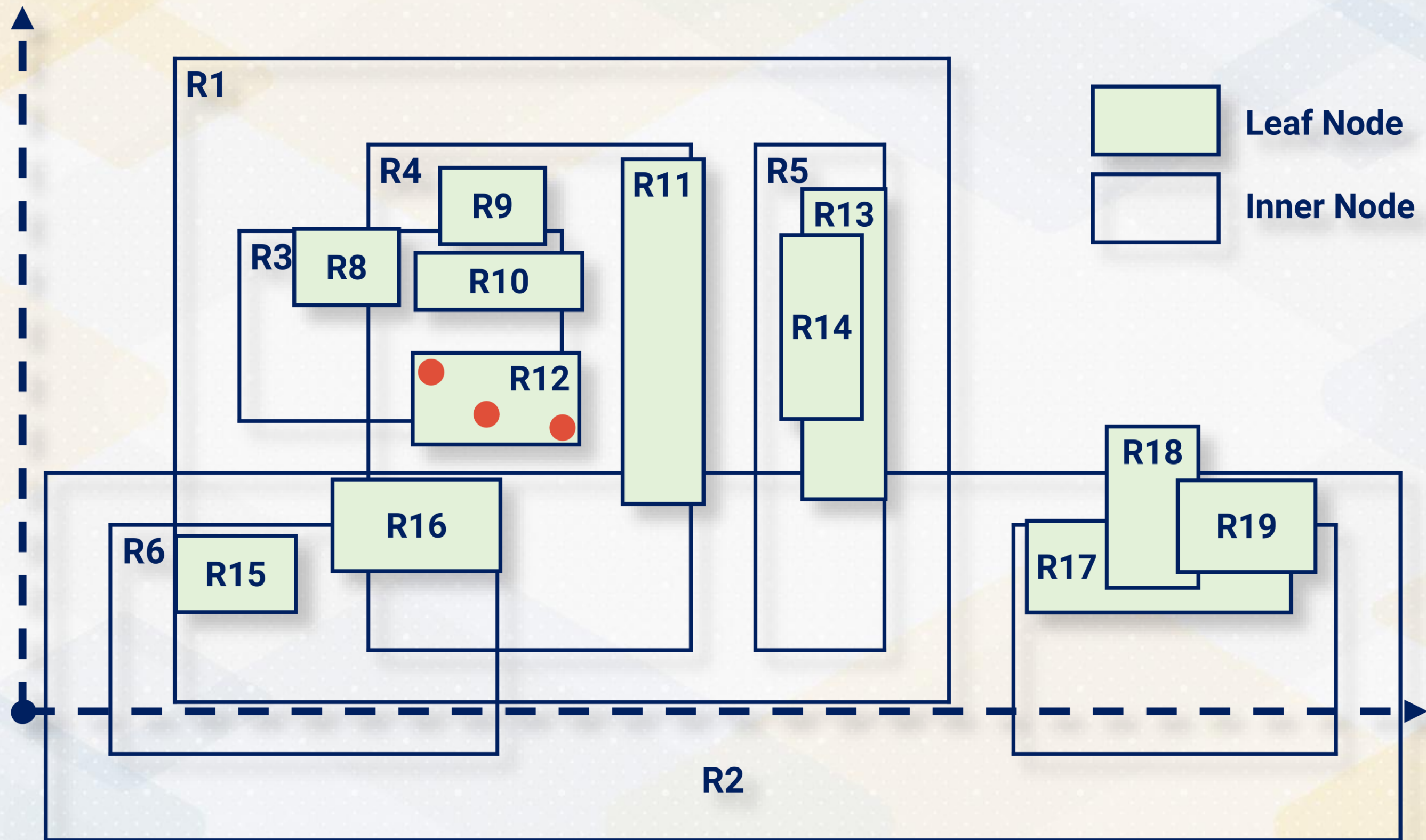
Inner Node

Contains
Child Nodes
and their
Bounding
Rectangles

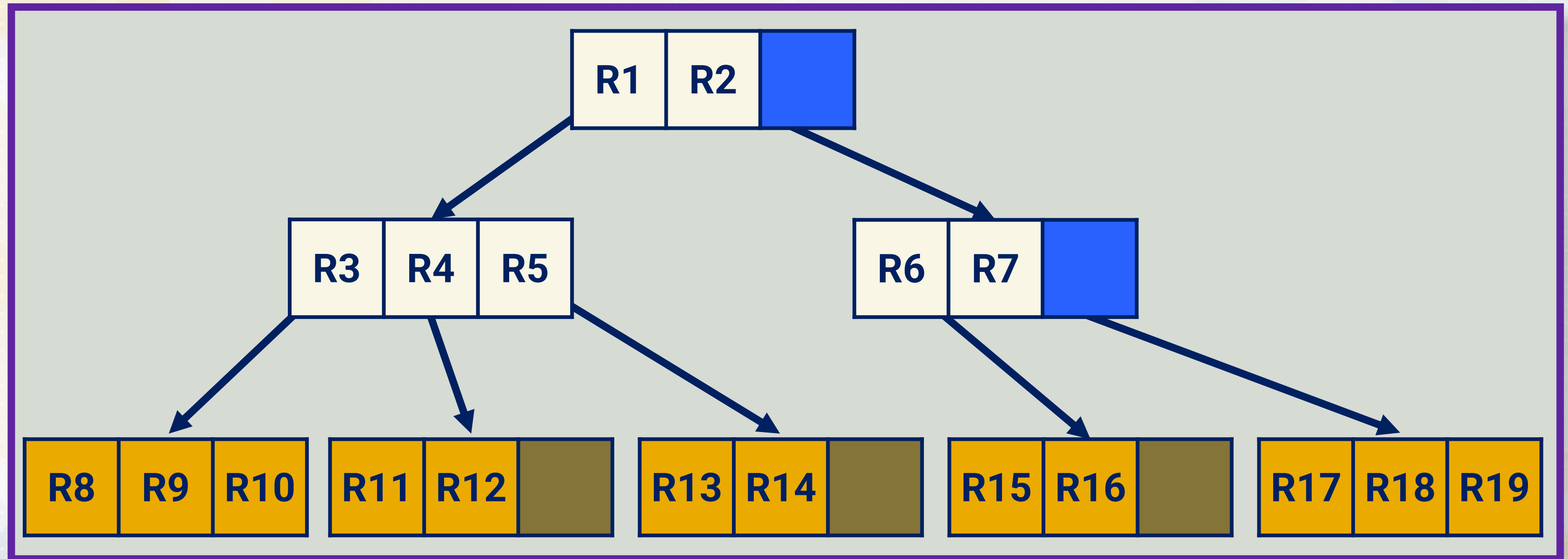
Root Node

Top-Most
Inner Node;
Entry
Point

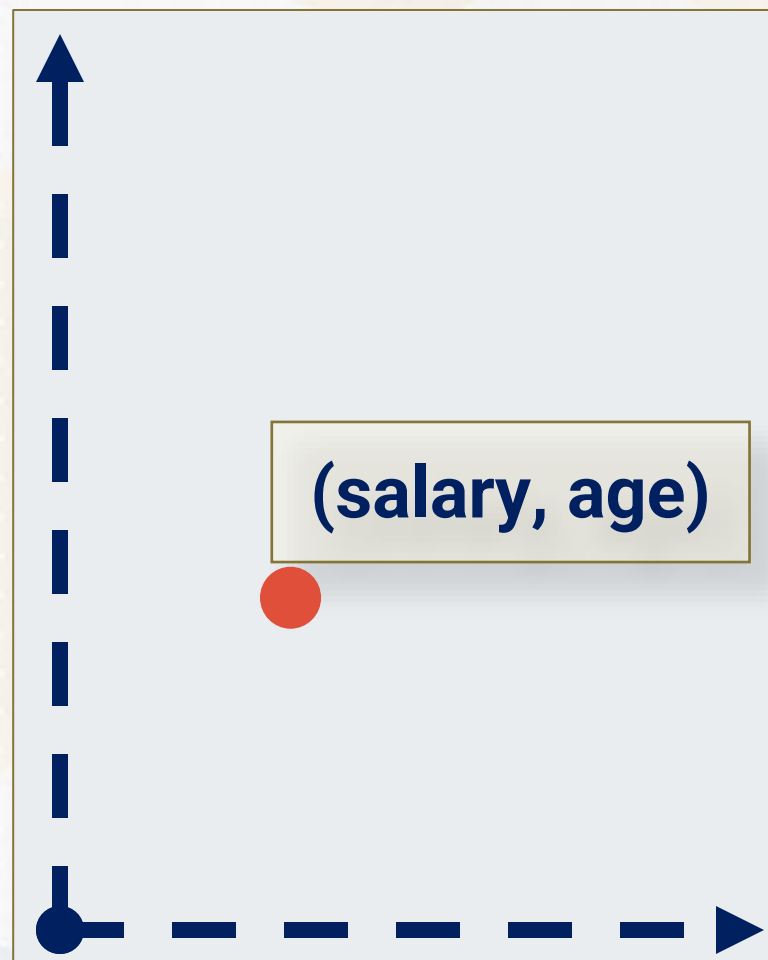
R-Tree Structure



R-Tree Structure



Point

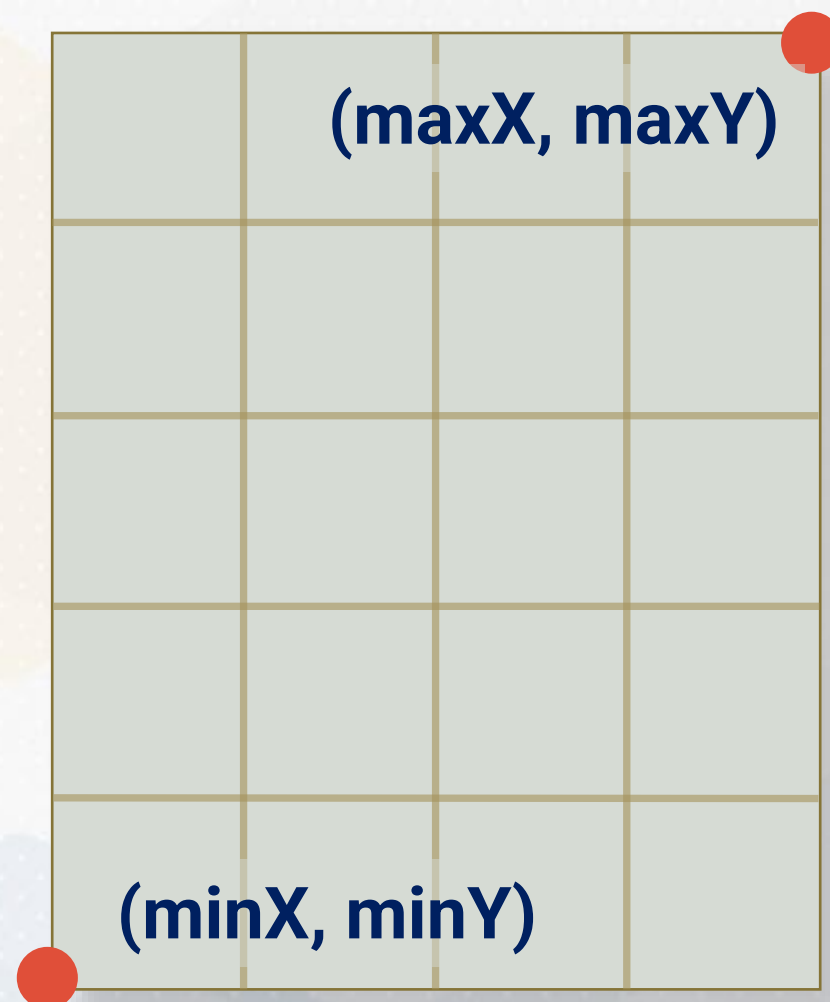


$(x, y) = (\text{salary}, \text{age})$

```
struct Point {  
    float x, y;  
    Point(float x, float y) : x(x), y(y) {}  
};
```


Rectangle

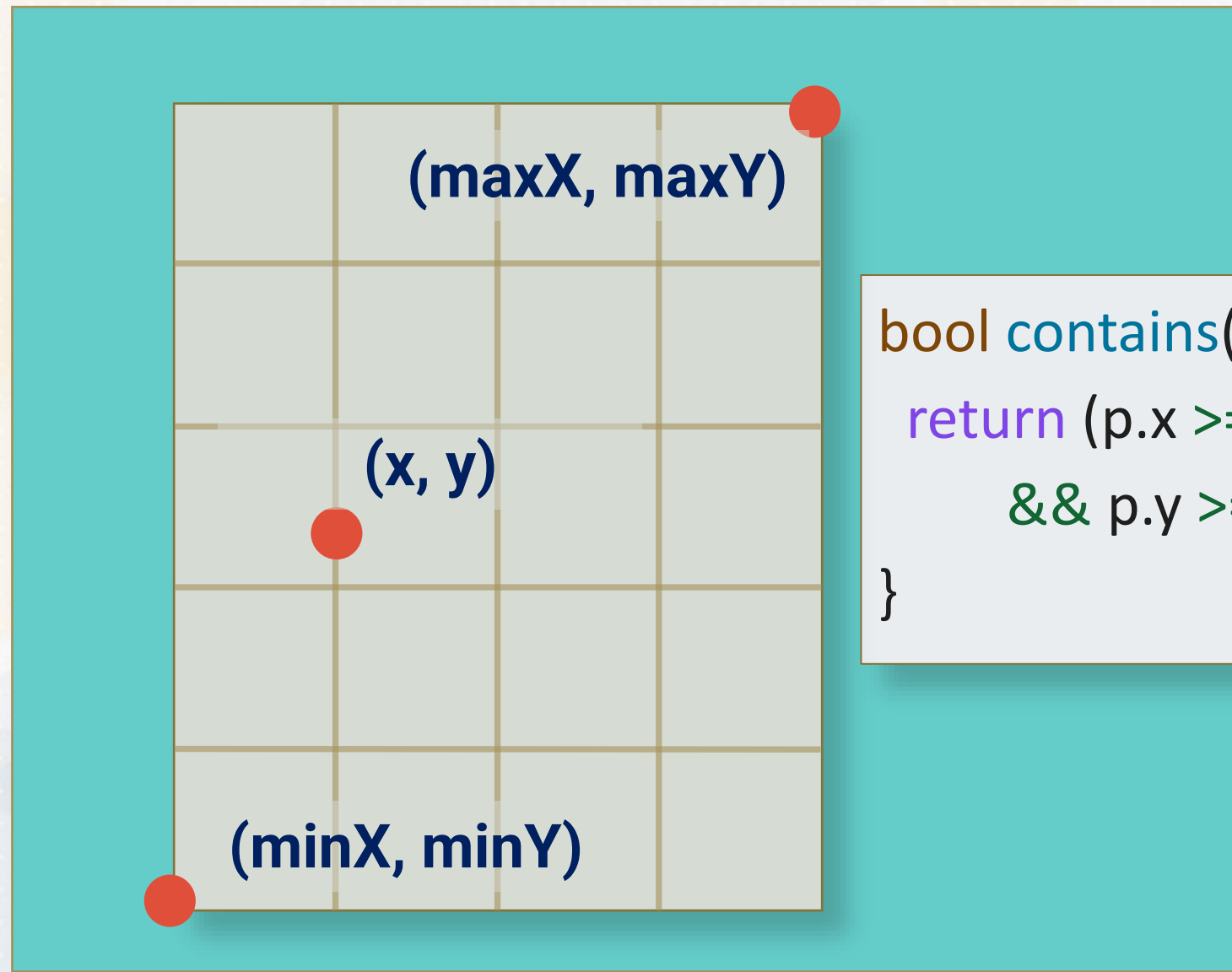
Defines a bounding box using its minimum and maximum coordinates



```
struct Rectangle {  
    float minX, minY, maxX, maxY;  
    Rectangle(float minX, float minY, float maxX, float maxY)  
        : minX(minX), minY(minY), maxX(maxX), maxY(maxY) {}  
  
    bool contains(const Point &p);  
    bool intersects(const Rectangle &other) const;  
};
```

Rectangle: Contains

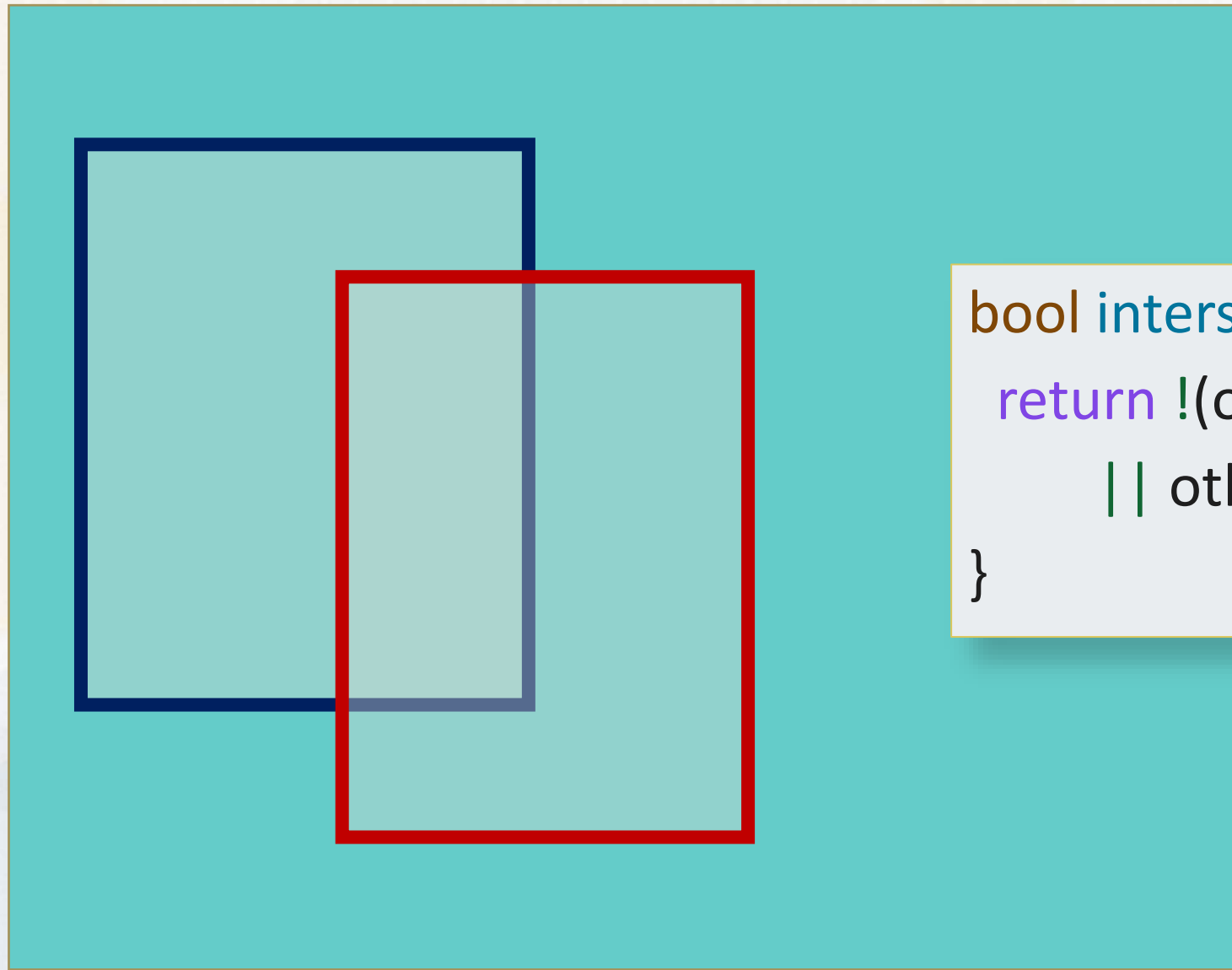
Check if a point lies inside the rectangle



```
bool contains(const Point &p) const {  
    return (p.x >= minX && p.x <= maxX  
            && p.y >= minY && p.y <= maxY);  
}
```


Rectangle: Intersection

Determine if two rectangles overlap



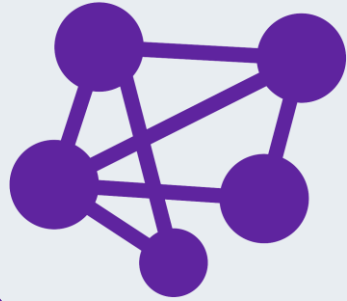
```
bool intersects(const Rectangle &other) const {  
    return !(other.minX > maxX || other.maxX < minX  
            || other.minY > maxY || other.maxY < minY);  
}
```



Inserting Points



Inserting Points into the R-Tree



**Inner
Node**

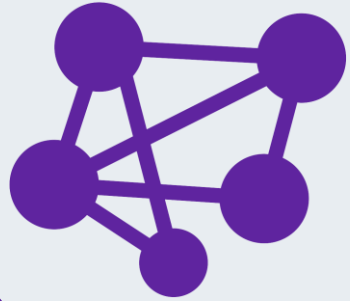
- Recursively finds best child node
- With least enlargement of bounding rectangle

```
void insert(RTreeNode * node, const Point &point, const Rectangle &rect) {  
    ...  
    else {  
        int bestChild = chooseBestChild(node, rect);  
        insert(node->children[bestChild], point, rect);  
        node->childrenRectangles[bestChild].expand(rect);  
    }  
}
```

Best Child with Least Enlargement

```
int chooseBestChild(RTreeNode * node, const Rectangle &rect) {  
    int bestChild = 0;  
    for (size_t i = 0; i < node->children.size(); ++i) {  
        Rectangle enlarged = node->childrenRectangles[i];  
        enlarged.expand(rect);  
        float enlargement = ...  
        if (enlargement < minEnlargement) {  
            minEnlargement = enlargement;  
            bestChild = i;  
        }  
    }  
    return bestChild;  
}
```


Leaf Node Insertion

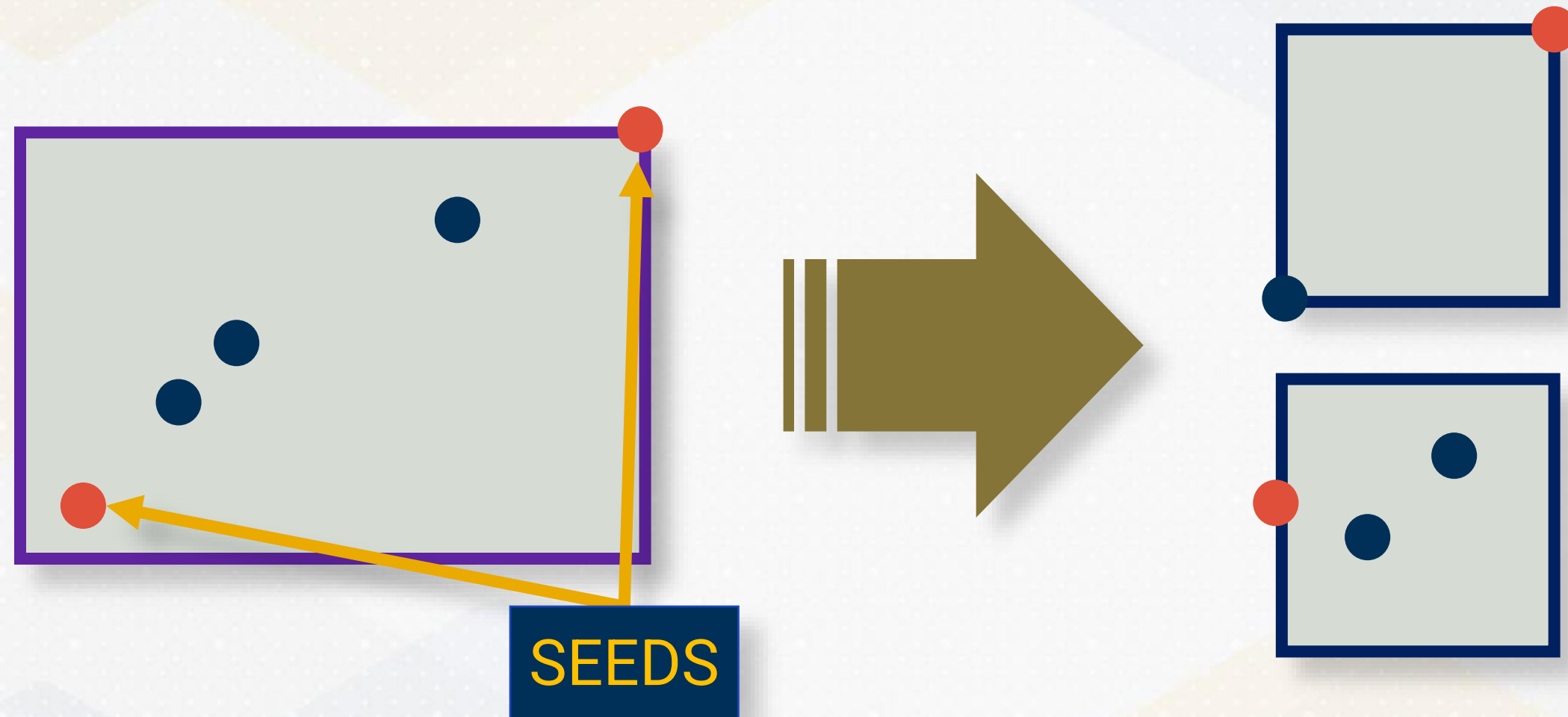


Leaf Node

- Add point to the leaf node
- Split if node exceeds maximum point capacity

```
void insert(RTreeNode * node, const Point &point, const Rectangle &rect) {  
    if (node->isLeaf) {  
        node->points.push_back(point);  
        if (node->points.size() > maxPoints) {  
            split(node); // Split the node if it exceeds max points  
        }  
    }  
    ...  
}
```

Quadratic Node Splitting Algorithm



Quadratic Node Splitting Algorithm

```
void chooseSeeds(const std::vector<Point> &points, int &seed1, int &seed2) {  
    float maxDistance = -1;  
    for (size_t i = 0; i < points.size(); ++i) {  
        for (size_t j = i + 1; j < points.size(); ++j) {  
            float distance = std::sqrt(std::pow(points[i].x - points[j].x, 2) +  
                                       std::pow(points[i].y - points[j].y, 2));  
            if (distance > maxDistance) {  
                maxDistance = distance;  
                seed1 = i;  
                seed2 = j;  
            }  
        }  
    }  
}
```



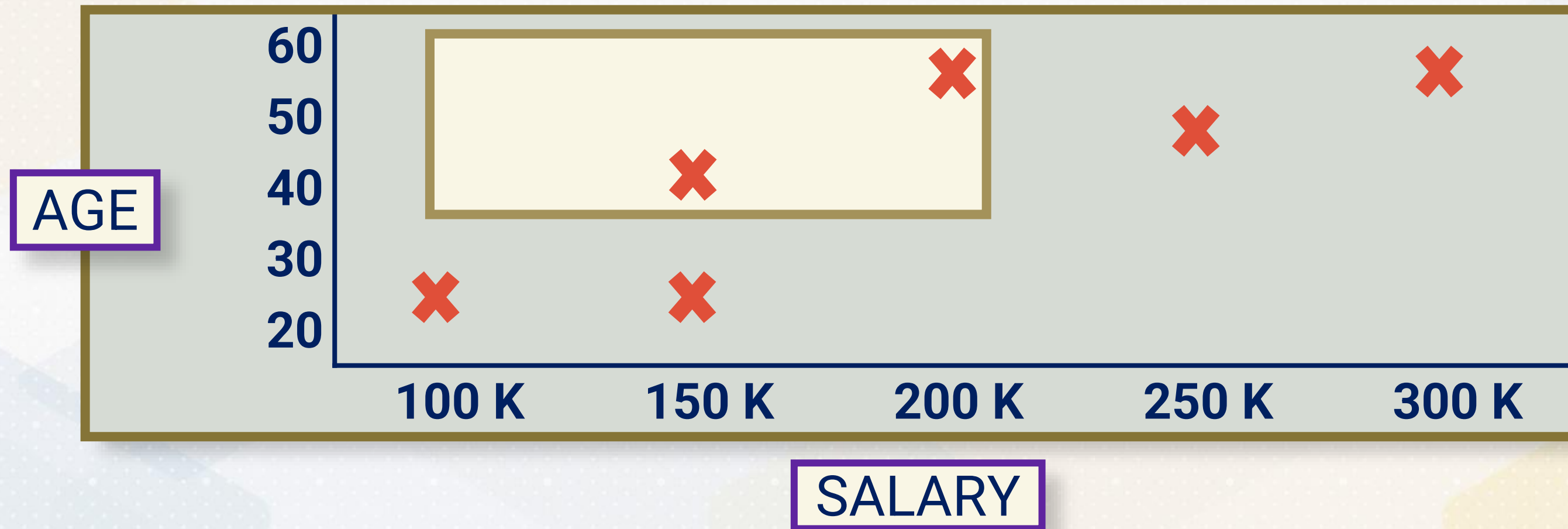
Range & Nearest Neighbor Query



Range Query in R-Tree

$100\text{ K} < \text{Salary} < 200\text{ K}$

$40 < \text{Age} < 60$



Range Query in R-Tree

```
void query(RTreeNode * node, const Rectangle &rect,
          std::vector<Point> &results) {
    if (node->isLeaf) {
        for (const Point &p : node->points) {
            if (rect.contains(p)) {
                results.push_back(p);
            }
        }
    } else {
        for (size_t i = 0; i < node->children.size(); ++i) {
            if (rect.intersects(node->childrenRectangles[i])) {
                query(node->children[i], rect, results);
            }
        }
    }
}
```

Nearest Neighbor Query in R-Tree

Salary ~ 200 K

Age ~ 50



Nearest Neighbor Query: Inner Node

```
void nearestNeighbor(RTreeNode * node, const Point &queryPoint, int k,
                    std::priority_queue<std::pair<float, Point>> &pq) {

    ...
    else {
        std::vector<std::pair<float, R - TreeNode *>> childDistances;
        for (size_t i = 0; i < node->children.size(); ++i) {
            float distance = node->childrenRectangles[i].minDistance(queryPoint);
            childDistances.push_back({distance, node->children[i]});
        }
        std::sort(childDistances.begin(), childDistances.end());
        for (const auto &child : childDistances) {
            nearestNeighbor(child.second, queryPoint, k, pq);
        }
    }
}
```

Nearest Neighbor Query: Leaf Node

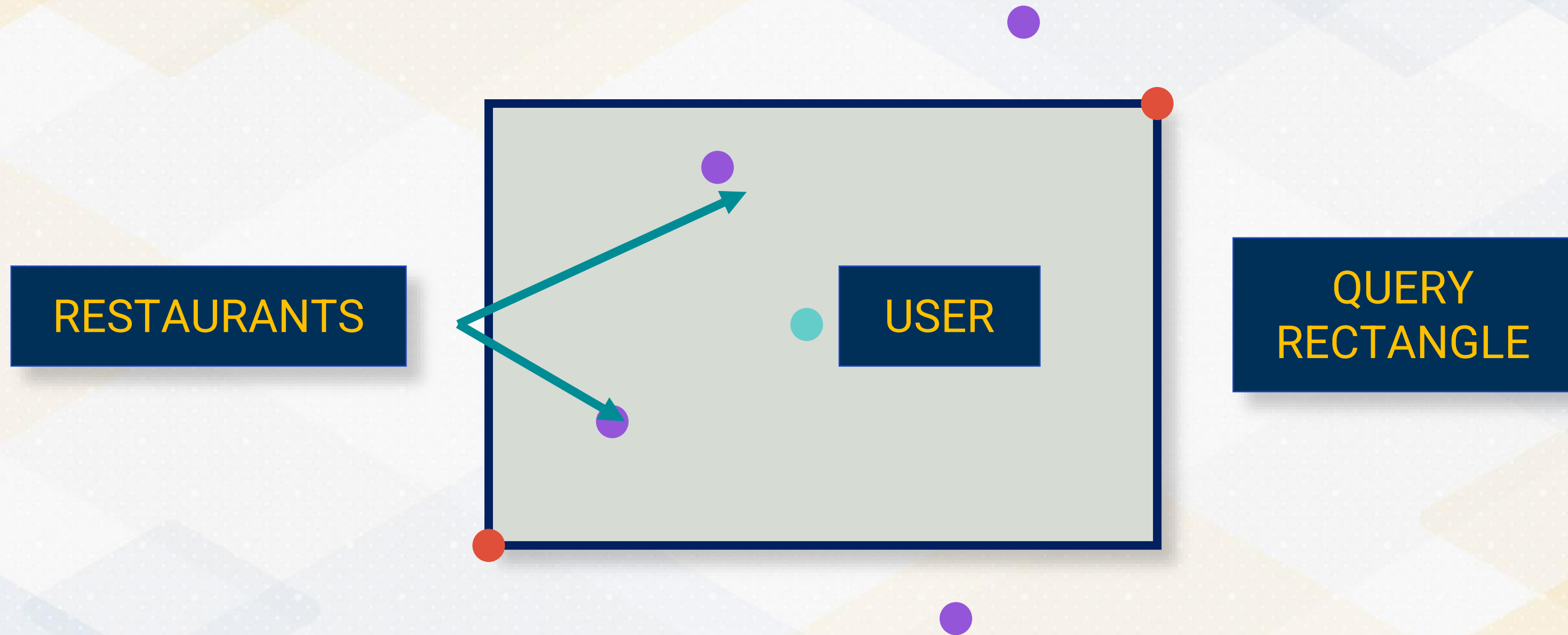
```
void nearestNeighbor(RTreeNode * node, const Point &queryPoint, int k,  
                    std::priority_queue<std::pair<float, Point>> &pq) {  
    if (node->isLeaf) {  
        for (const Point &p : node->points) {  
            float distance = std::sqrt(std::pow(p.x - queryPoint.x, 2) +  
                                       std::pow(p.y - queryPoint.y, 2));  
            pq.push(std::make_pair(distance, p));  
            if (pq.size() > k)  
                pq.pop();  
        }  
    }  
    ...  
}
```



Applications



Restaurant Search on Maps



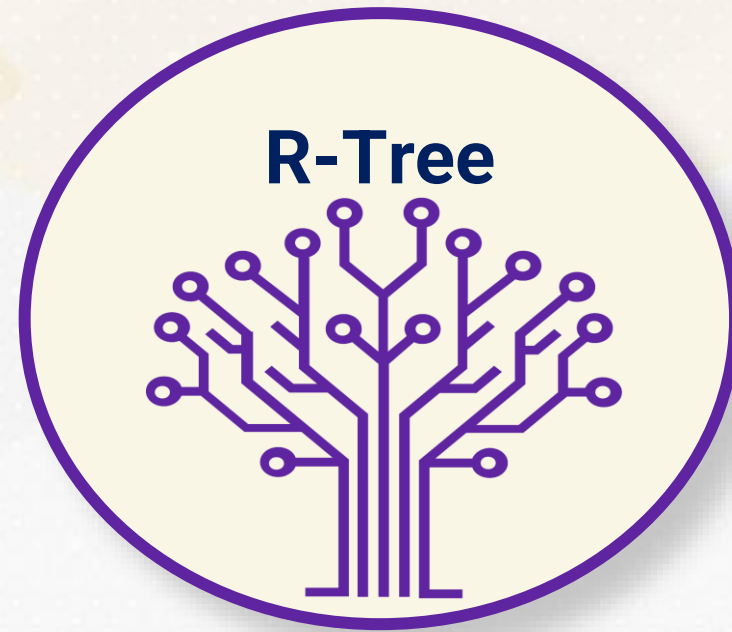
Ride Matching



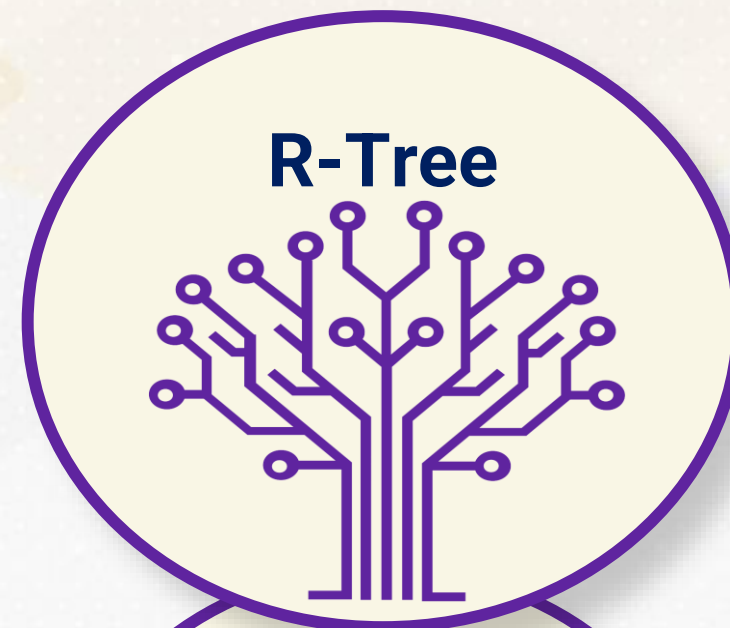
```
Point driver({latitude, longitude}, "DriverID");  
tree.insert(driver);
```

```
Point riderLocation({rider_latitude, rider_longitude}, "RiderLocation");  
int k = 10; // We want the nearest drivers  
std::vector<Point> nearestDrivers = tree.nearestNeighbor(riderLocation, k);
```


Limitation of R-Tree



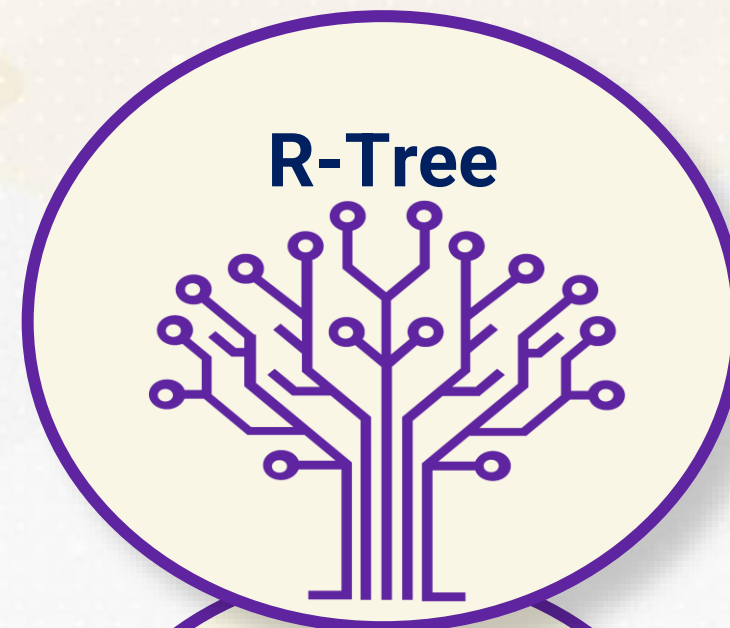
Limitation of R-Tree



**Only
2D
Indexing**



Limitation of R-Tree

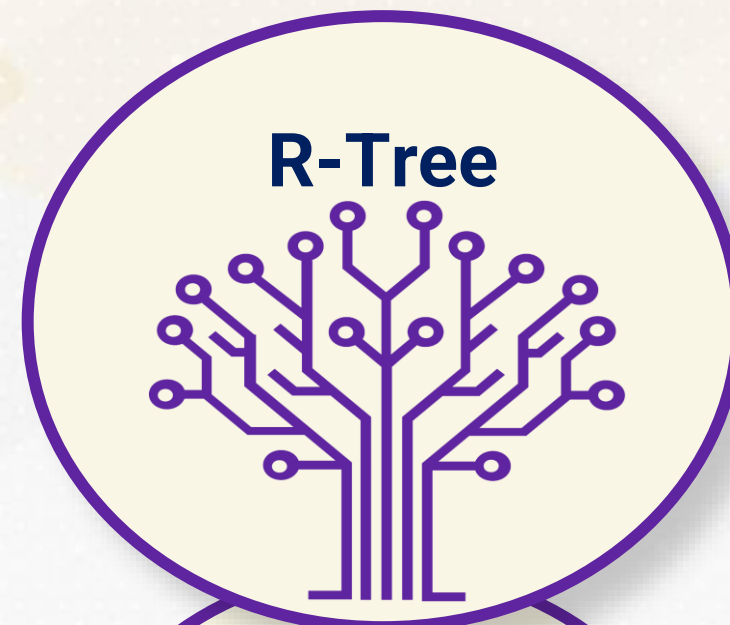


**Only
2D
Indexing**

**Cannot
Support
Higher
Dimensions**



Limitation of R-Tree



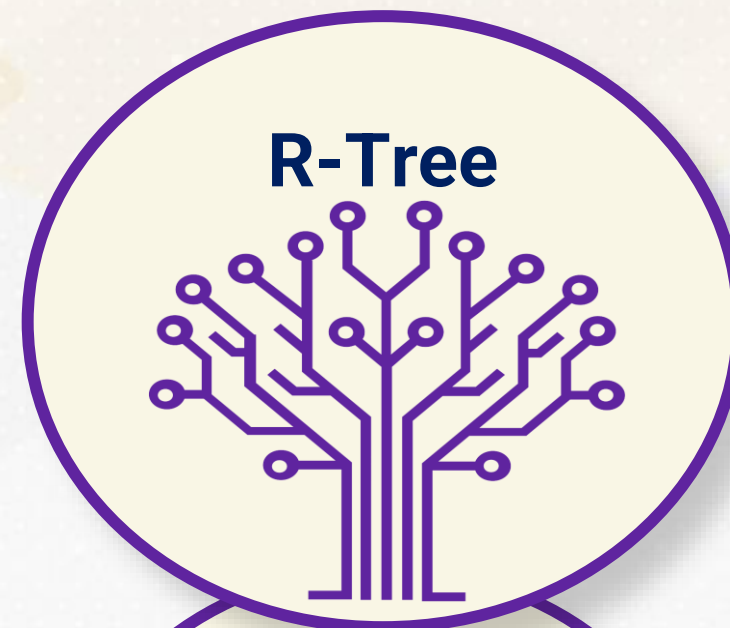
**Only
2D
Indexing**

**Cannot
Support
Higher
Dimensions**



**N
Dimensions**

Limitation of R-Tree



**Only
2D
Indexing**

**Cannot
Support
Higher
Dimensions**



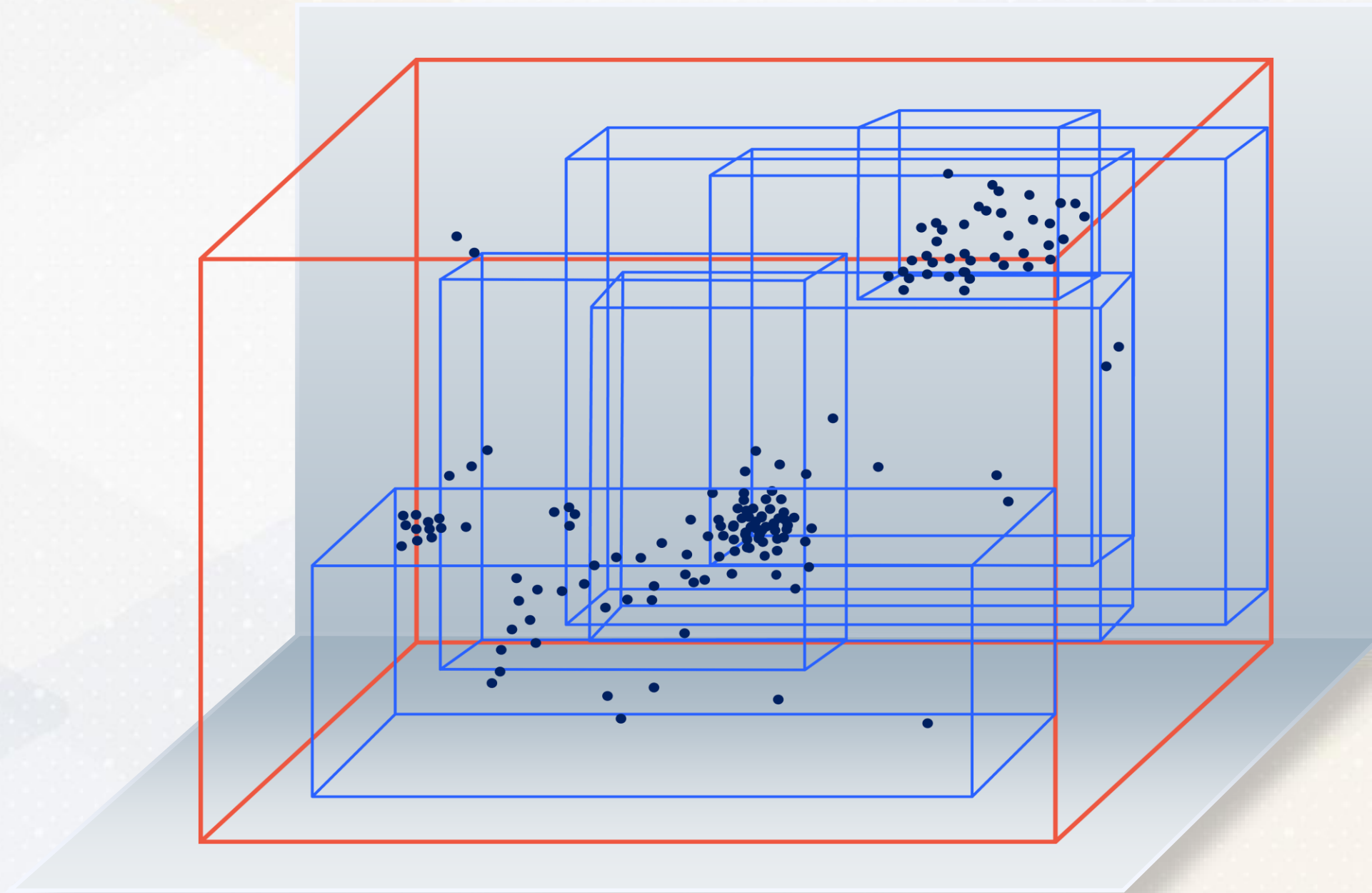
**N
Dimensions**

**Higher
Dimensional
Applications**

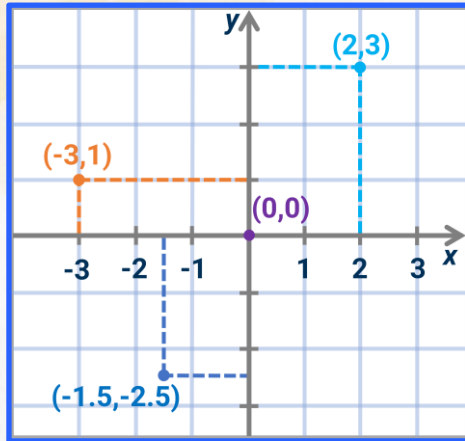
ND R-Tree

ND R-Tree: A Multidimensional Index

Efficiently manage data in tens of dimensions

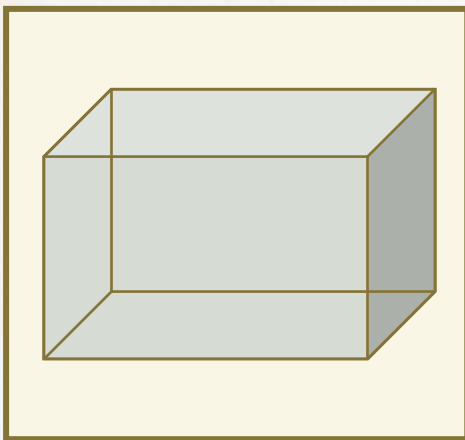


ND R-Tree Structure



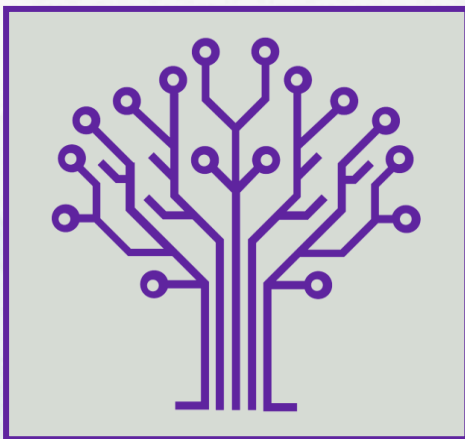
Point

N-dimensional Point



Hyper-
Rectangle

N-dimensional Rectangle



Nodes

Leaf Node

Inner Node

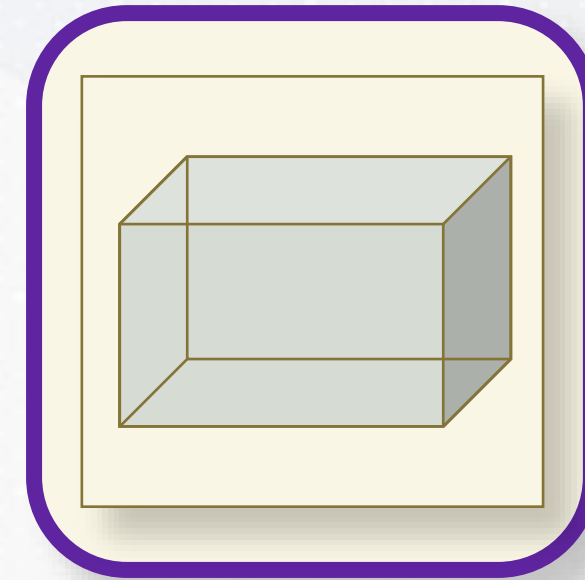
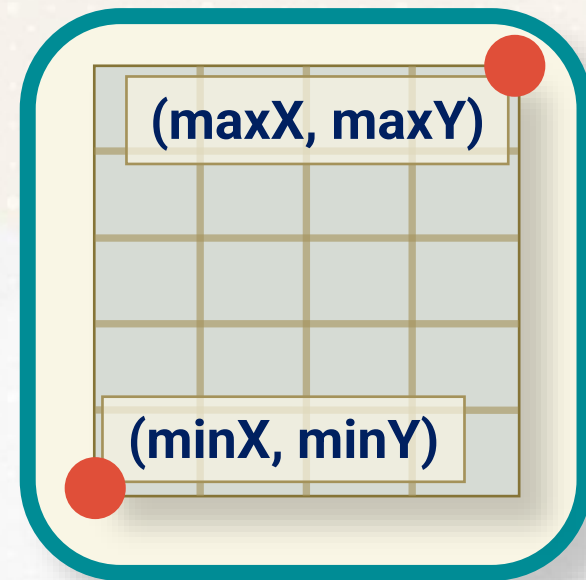
Root Node

N-Dimensional Point

Example: 512-dimensional point with image features

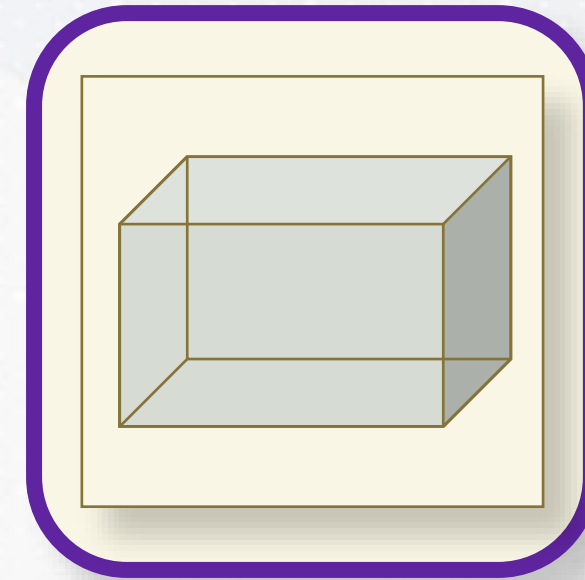
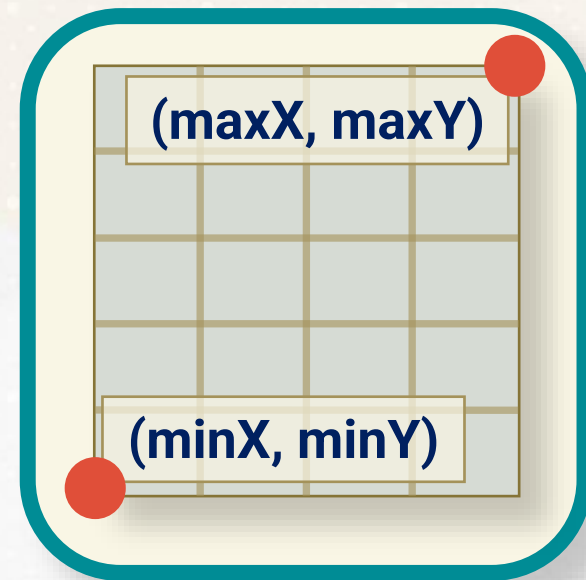
```
struct Point {  
    std::vector<float> coordinates;  
    std::string label;  
};
```

N-Dimensional Hyperrectangle



```
struct Rectangle {  
    std::vector<float> minCoords, maxCoords;  
    bool contains(const Point &p) const;  
};
```


N-Dimensional Hyperrectangle



```
struct Rectangle {  
    std::vector<float> minCoords, maxCoords;  
    bool contains(const Point &p) const;  
};
```

Game Engine



```
Point gameObject({x, y, z}, "GameObjectID");  
tree.insert(gameObject);
```

```
Rectangle collisionArea({minX, minY, minZ}, {maxX, maxY, maxZ});  
std::vector<Point> nearbyObjects = tree.query(collisionArea);
```

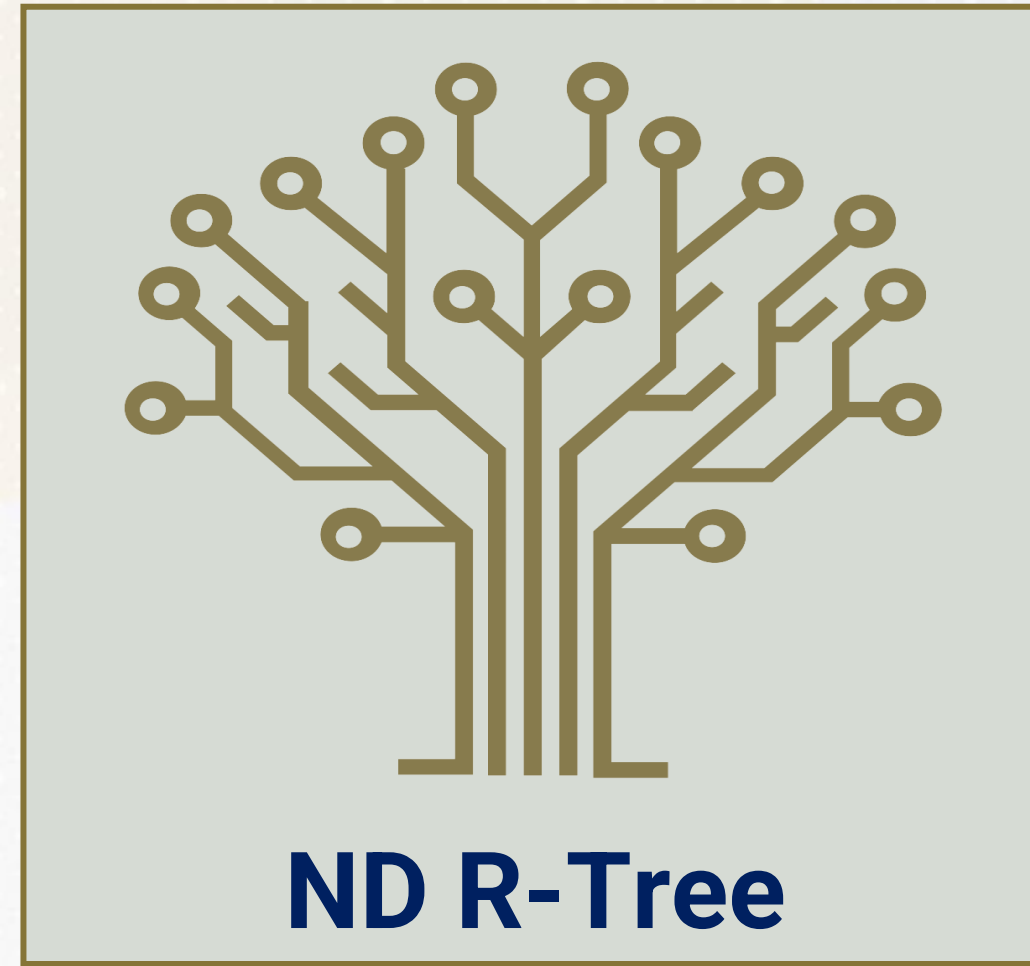

Game Engine



```
Point gameObject({x, y, z}, "GameObjectID");  
tree.insert(gameObject);
```

```
Rectangle collisionArea({minX, minY, minZ}, {maxX, maxY, maxZ});  
std::vector<Point> nearbyObjects = tree.query(collisionArea);
```

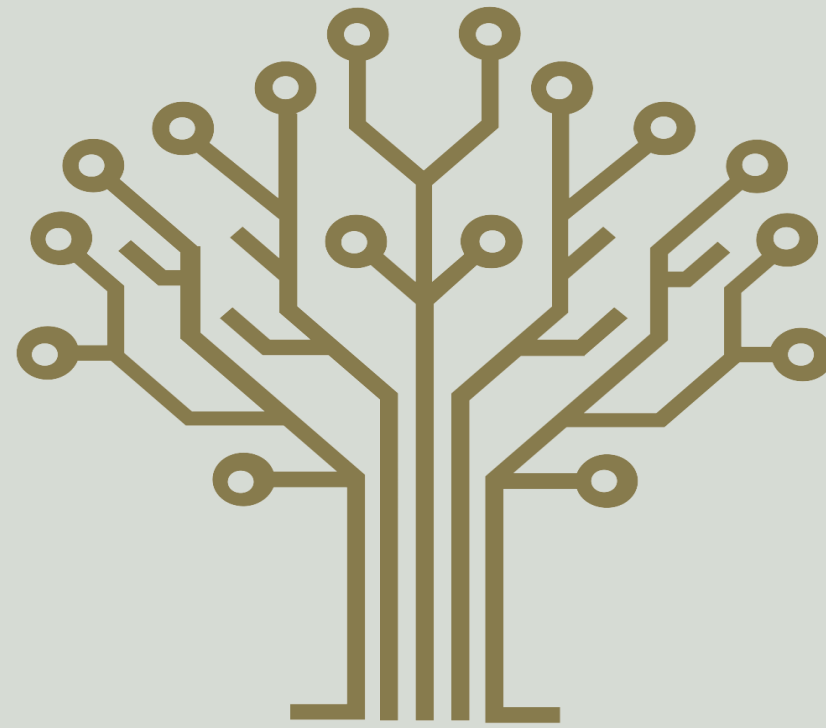
Challenge



Challenge



**Balanced
Efficient Queries**

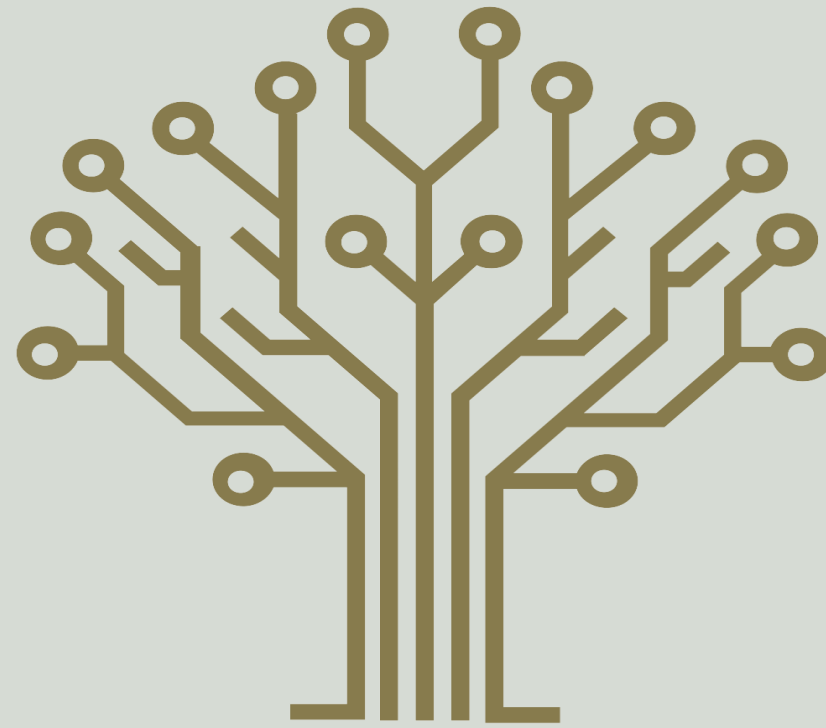


ND R-Tree

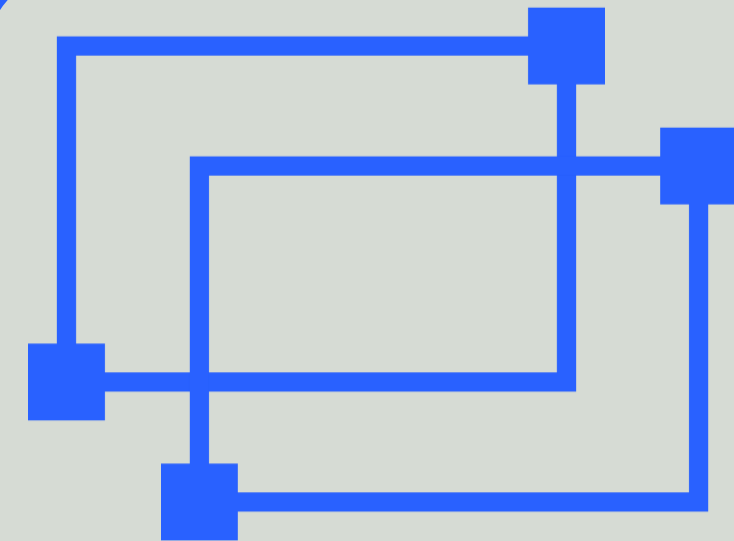
Challenge



Balanced
Efficient Queries



ND R-Tree



Rectangle Overlap
Fewer Subtrees



R-Tree



R-Tree

**Nearest
Neighbor
Query**

ND RTree





NDRTree





NDRTree

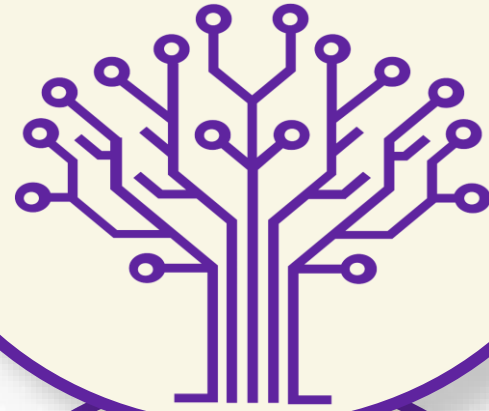
**Nearest
Neighbor
Search**

Limitations of R-Tree



Limitations of R-Tree

ND R-Tree



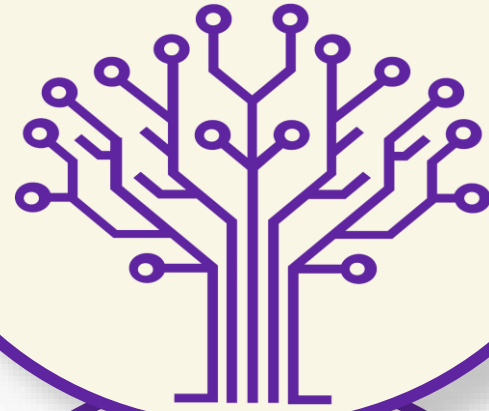
**Extension
of R-Tree**

Applications



Limitations of R-Tree

ND R-Tree



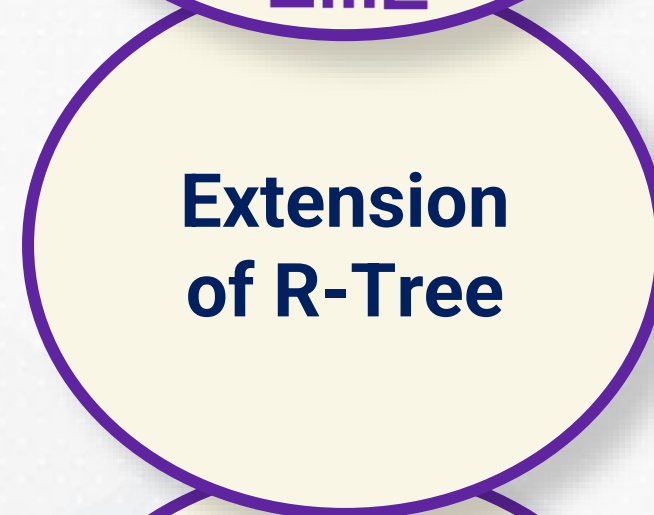
**Extension
of R-Tree**

**Supports
Arbitrary
Indexing**

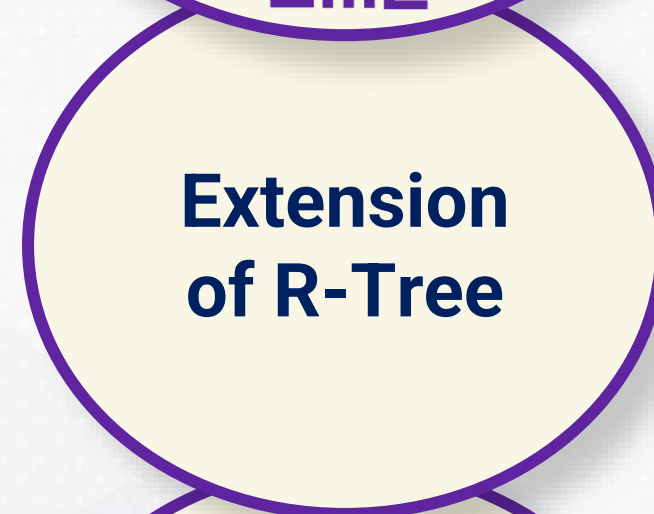
Applications



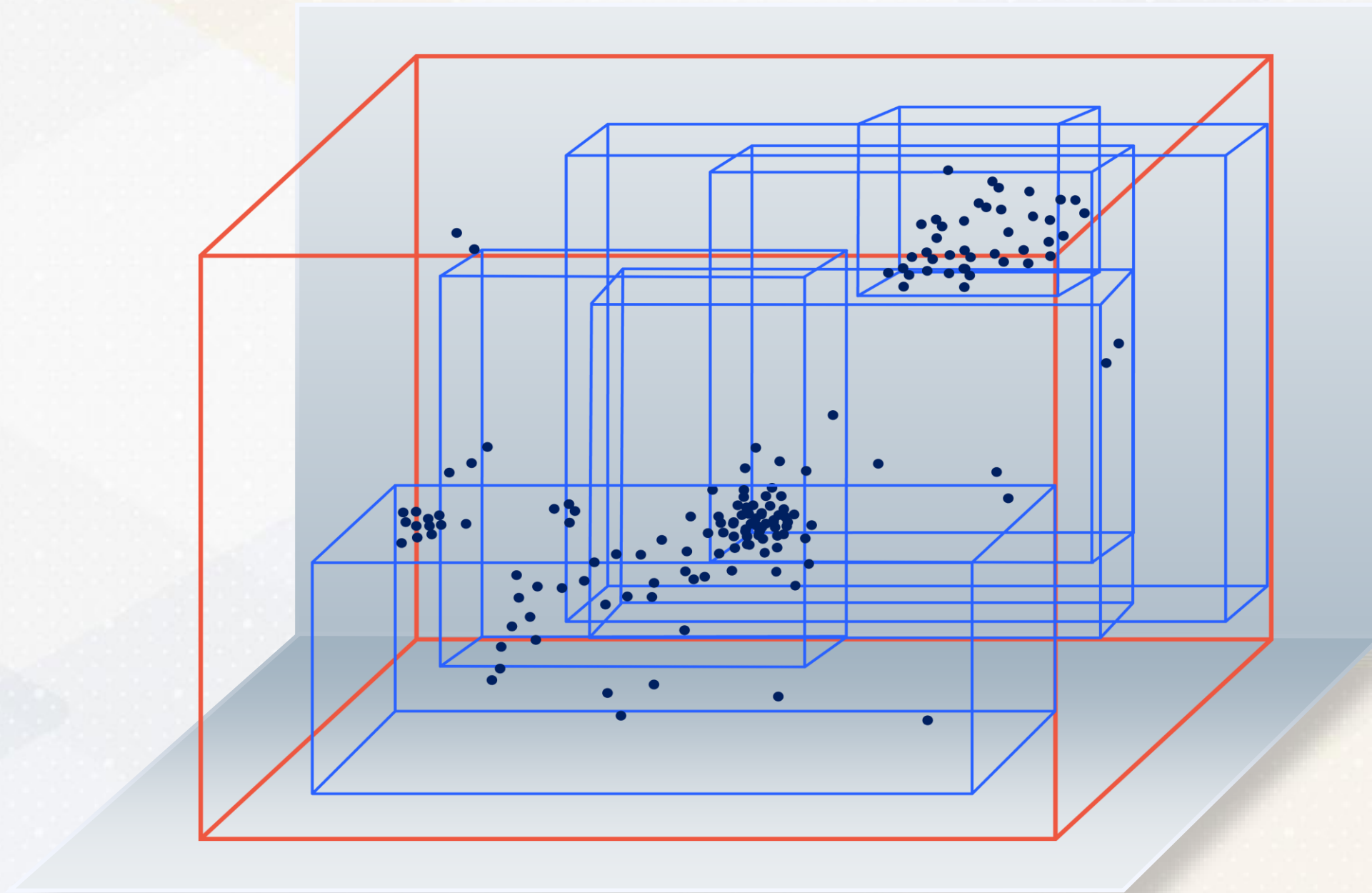
Limitations of R-Tree



Limitations of R-Tree

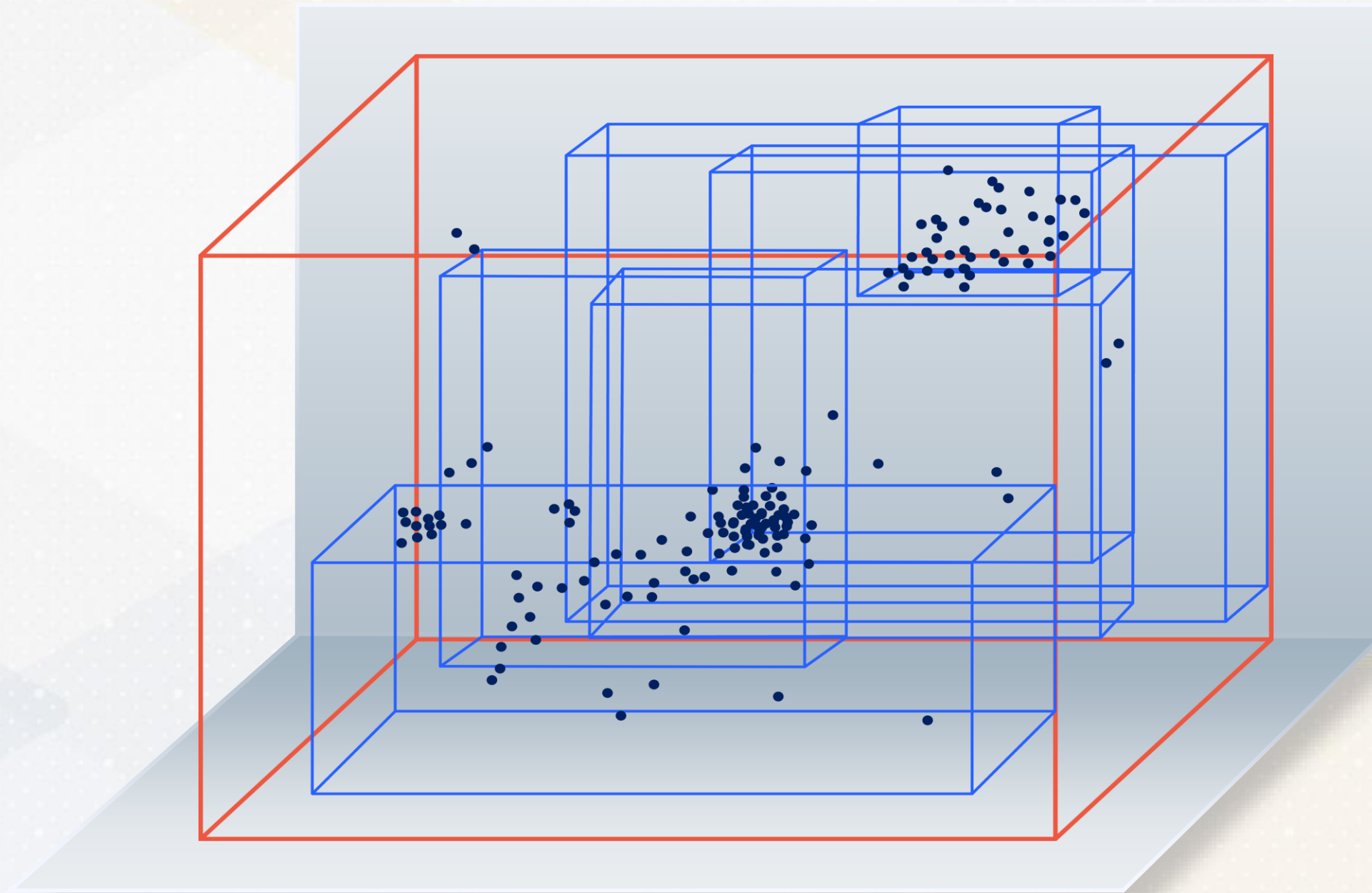


ND R-Tree: A Multidimensional Index

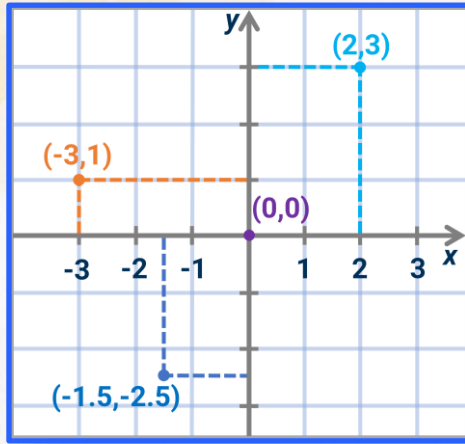


ND R-Tree: A Multidimensional Index

Efficiently Manages Data in Hundreds of Dimensions

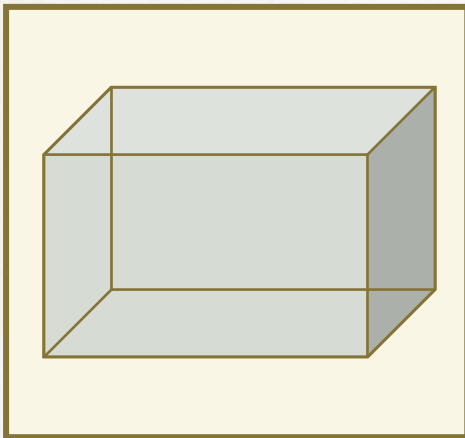


ND R-Tree Structure



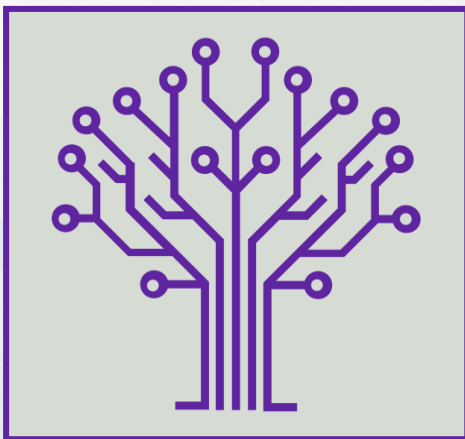
Point

Coordinate Position in N-dimensional Space



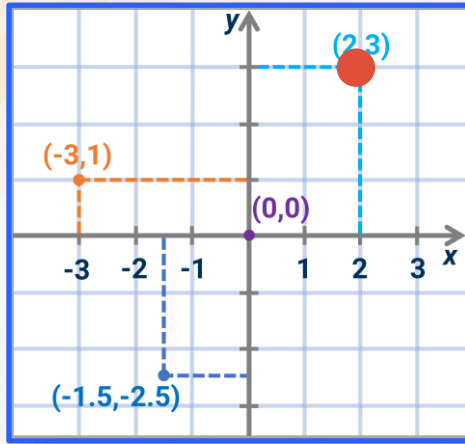
Hyper-Rectangle

N-dimensional Shape Enclosing Points or Rectangles



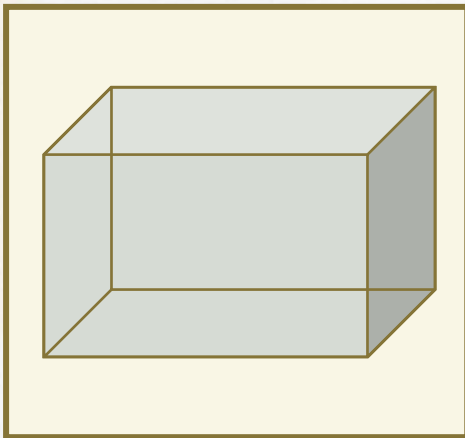
R-Tree
Node

ND R-Tree Structure



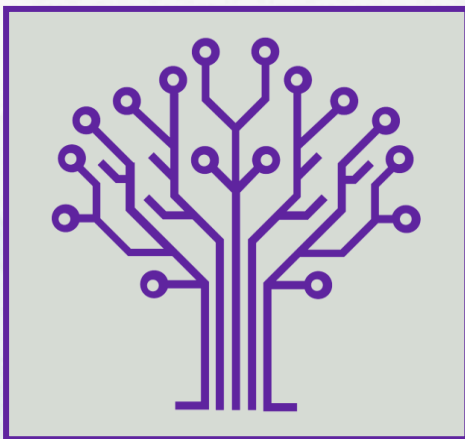
Point

Coordinate Position in N-dimensional Space



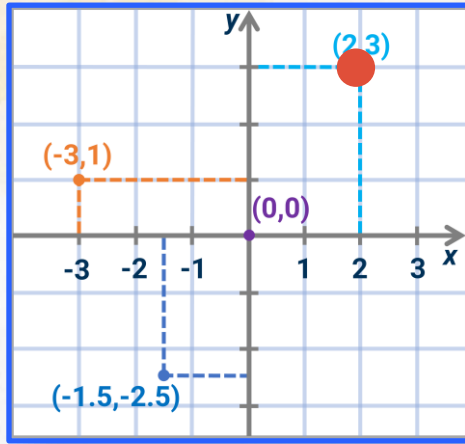
Hyper-
Rectangle

N-dimensional Shape Enclosing Points or Rectangles



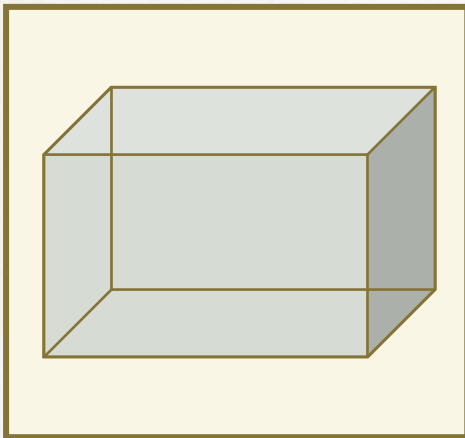
R-Tree
Node

ND R-Tree Structure



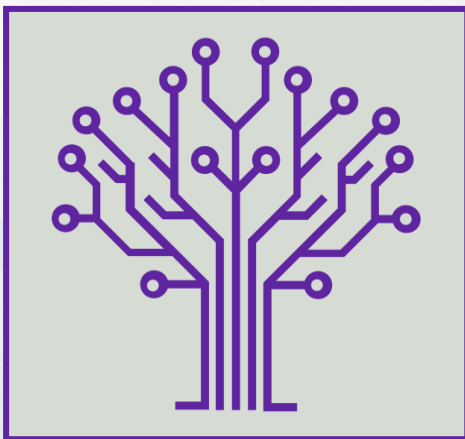
Point

Coordinate Position in N-dimensional Space



Hyper-Rectangle

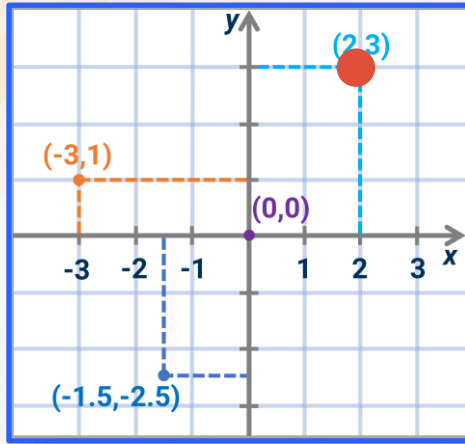
N-dimensional Shape Enclosing Points or Rectangles



R-Tree Node

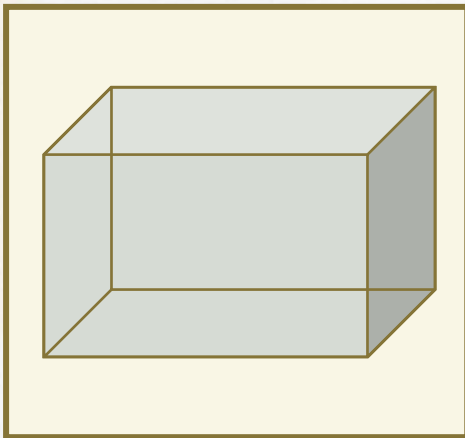
Leaf Node

ND R-Tree Structure



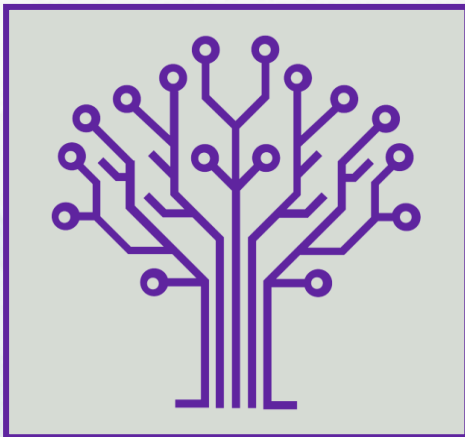
Point

Coordinate Position in N-dimensional Space



Hyper-Rectangle

N-dimensional Shape Enclosing Points or Rectangles

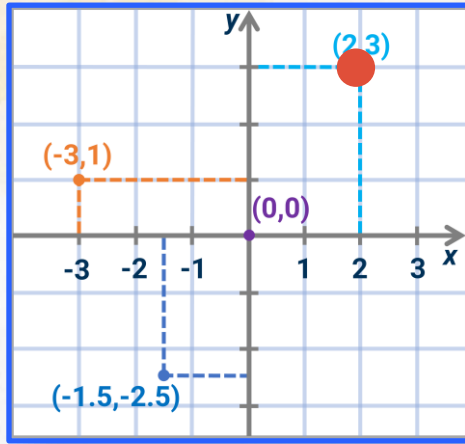


R-Tree Node

Leaf Node

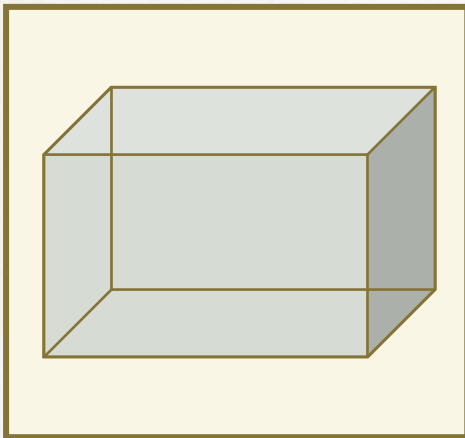
Inner Node

ND R-Tree Structure



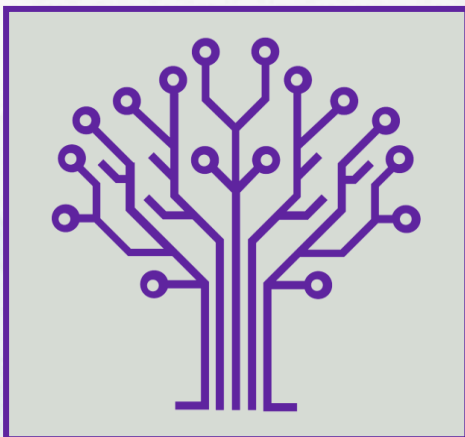
Point

Coordinate Position in N-dimensional Space



Hyper-Rectangle

N-dimensional Shape Enclosing Points or Rectangles



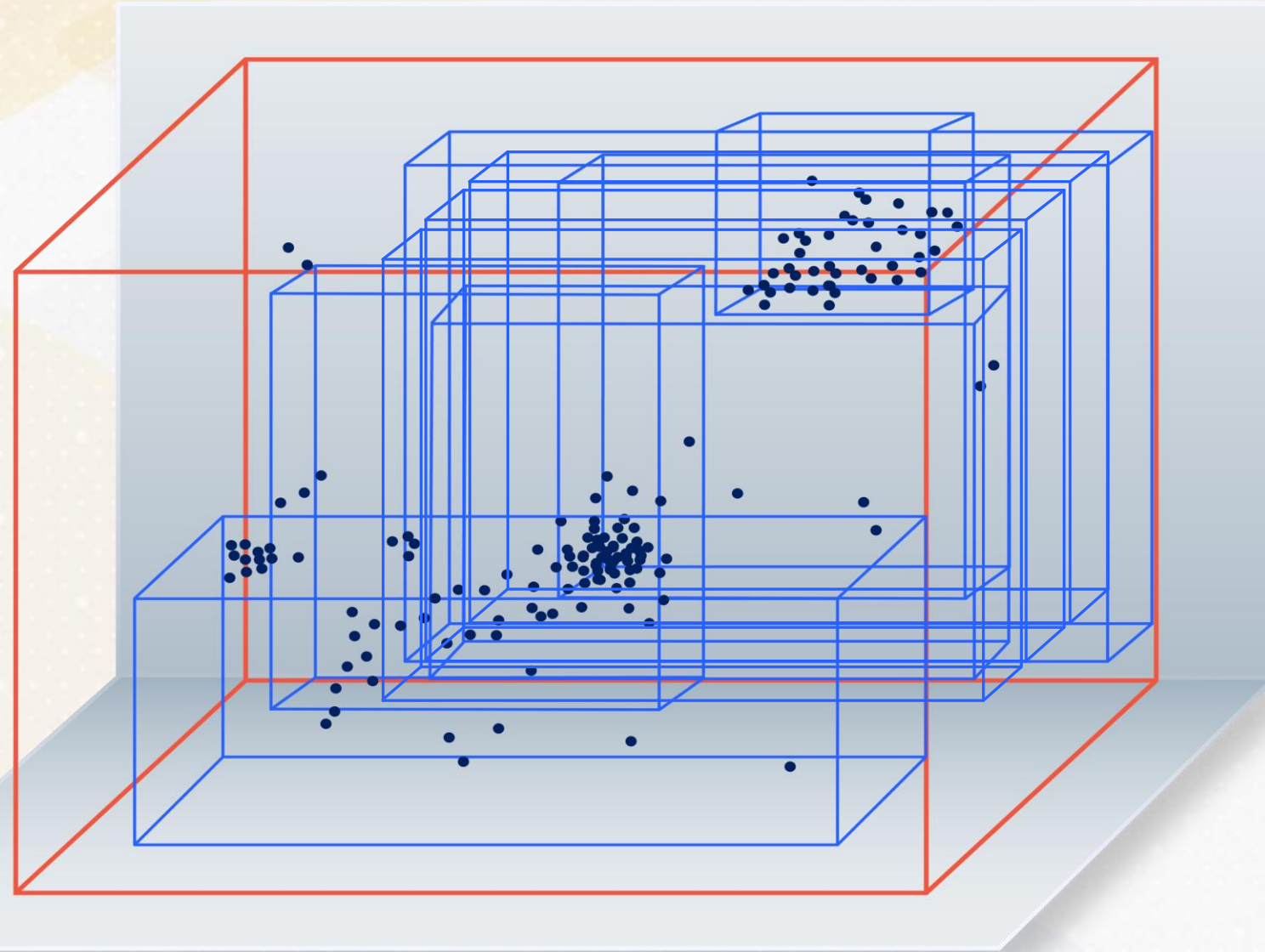
R-Tree Node

Leaf Node

Inner Node

Root Node

Point



```
struct Point {  
    std::vector<float> coordinates;  
    std::string label;  
};
```

- High-dimensional points (e.g., 512 dimensions).
- **Example:** A feature vector representing an image with 512 dimensions, where each value is a descriptor of the image content.

Rectangle

- A bounding box that represents the minimum and maximum feature values for a cluster of points in a high-dimensional space.

```
struct Rectangle {  
    std::vector<float> minCoords, maxCoords;  
    bool contains(const Point& p) const;  
};
```

Nodes

- Internal nodes use bounding hyper-rectangles to efficiently index child nodes.

```
struct RTreeNode {  
    bool isLeaf;  
    std::vector<Point> points;  
    std::vector<Rectangle> childrenRectangles;  
    std::vector<RTreeNode*> children;  
};
```


Inserting Points into the ND R-Tree

- Points are inserted into the ND R-Tree by navigating down the tree to find the best-fit leaf node.

```
void insert(RTreeNode* node, const Point& point, const Rectangle& rect) {  
    ...  
    else {  
        int bestChild = chooseBestChild(node, rect);  
        insert(node->children[bestChild], point, rect);  
        node->childrenRectangles[bestChild].expand(rect); // Update bounding rectangle  
    }  
}
```

Best Child with Least Enlargement

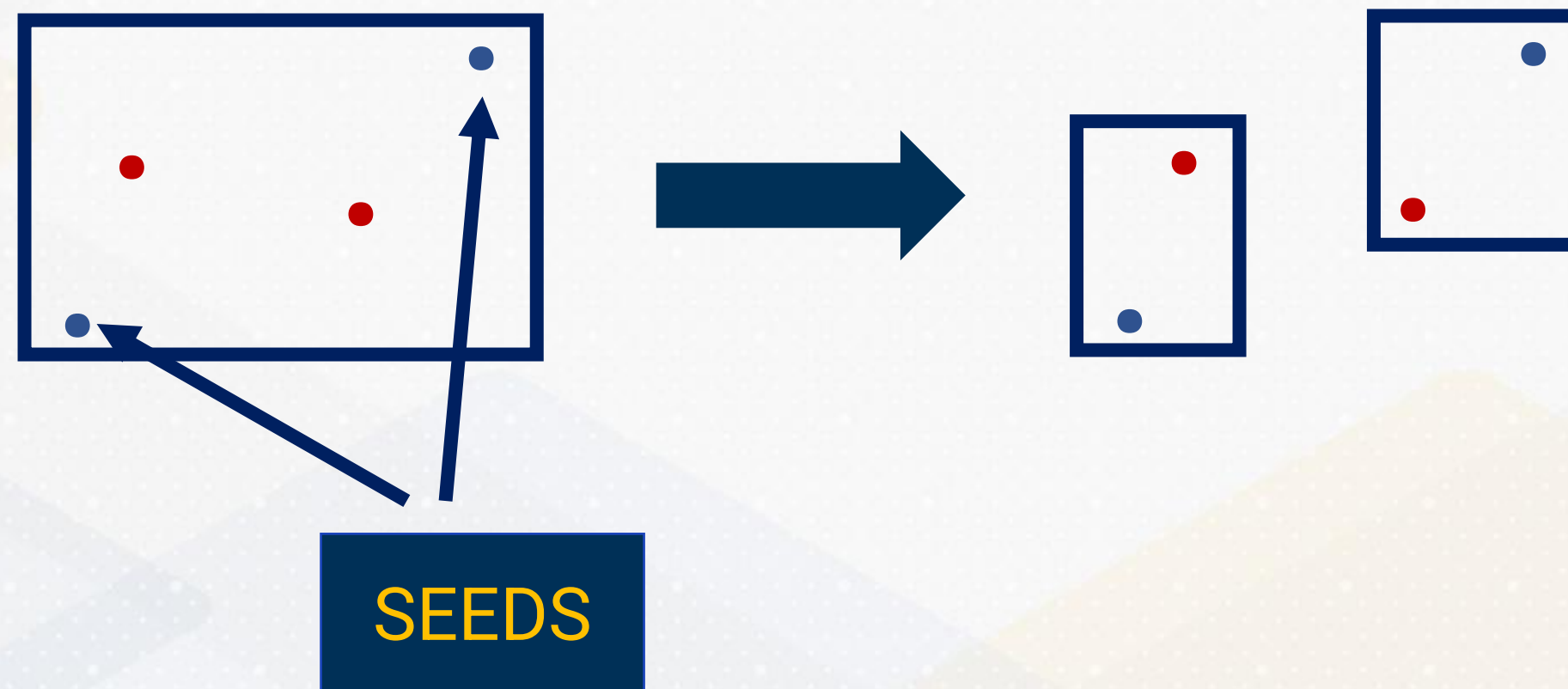
- Minimizes bounding hyperrectangle enlargement to maintain spatial locality.

```
int chooseBestChild(RTreeNode* node, const Rectangle& rect) {  
    int bestChild = 0;  
    float minEnlargement = std::numeric_limits<float>::max();  
    for (size_t i = 0; i < node->children.size(); ++i) {  
        Rectangle enlarged = node->childrenRectangles[i];  
        enlarged.expand(rect);  
        float enlargement = enlarged.area() - node->childrenRectangles[i].area();  
        if (enlargement < minEnlargement) {  
            minEnlargement = enlargement;  
            bestChild = i;  
        }  
    }  
    return bestChild;  
}
```


Node Splitting in ND R-Tree

- If a node overflows, it is split using a quadratic algorithm that aims to minimize bounding box overlap.
- **Quadratic Split:** Selects the two most distant points (seeds) and distributes the remaining entries to minimize area enlargement.

Node Splitting in ND R-Tree



Node Splitting in ND R-Tree

```
void chooseSeeds(const std::vector<Point>& points, int& seed1,
int& seed2) {
    float maxDistance = -1;
    for (size_t i = 0; i < points.size(); ++i) {
        for (size_t j = i + 1; j < points.size(); ++j) {
            float distance = std::sqrt(std::pow(points[i].x - points[j].x, 2)
+
                                std::pow(points[i].y - points[j].y, 2));
            if (distance > maxDistance) {
                maxDistance = distance;
                seed1 = i;
                seed2 = j;
            }
        }
    }
}
```

Nearest Neighbor Search

- **Purpose:** Finds the k nearest neighbors of a point.
- **Recursive Search Approach:** Traverses nodes in increasing order of distance from the query point, using a priority queue.

Nearest Neighbor Search (Internal Node)

```
void nearestNeighbor(RTreeNode* node, const Point& queryPoint, int k,
                    std::priority_queue<std::pair<float, Point>>& pq) {
    if (node->isLeaf) { ..}
    else {
        std::vector<std::pair<float, RTreeNode*>> childDistances;
        for (size_t i = 0; i < node->children.size(); ++i) {
            float distance = node->childrenRectangles[i].minDistance(queryPoint);
            childDistances.push_back(std::make_pair(distance, node->children[i]));
        }
        std::sort(childDistances.begin(), childDistances.end());
        for (const auto& child : childDistances) {
            nearestNeighbor(child.second, queryPoint, k, pq);
        }
    }
}
```

Nearest Neighbor Search (Leaf Node)

```
void nearestNeighbor(RTreeNode* node, const Point& queryPoint, int k,  
                    std::priority_queue<std::pair<float, Point>>& pq) {  
    if (node->isLeaf) {  
        for (const Point& p : node->points) {  
            float distance = 0.0;  
            for (size_t i = 0; i < p.coordinates.size(); ++i) {  
                distance += std::pow(p.coordinates[i] - queryPoint.coordinates[i], 2);  
            }  
            distance = std::sqrt(distance);  
            pq.push(std::make_pair(distance, p));  
            if (pq.size() > static_cast<size_t>(k)) {  
                pq.pop();  
            }  
        }  
    } ...  
}
```


Range Query in ND R-Tree

- ND R-Tree can efficiently perform **multidimensional range queries**, which are common in high-dimensional datasets.
- The algorithm recursively traverses nodes whose bounding rectangles intersect with the query region.
- For leaf nodes, it returns all points contained within the rectangle.
- For internal nodes, it recursively explores child nodes that may intersect the query.

Range Query in R-Tree

```
void query(RTreeNode* node, const Rectangle& rect, std::vector<Point>&
results) {
    if (node->isLeaf) {
        for (const Point& p : node->points) {
            if (rect.contains(p)) {
                results.push_back(p);
            }
        }
    } else {
        for (size_t i = 0; i < node->children.size(); ++i) {
            if (rect.intersects(node->childrenRectangles[i])) {
                query(node->children[i], rect, results);
            }
        }
    }
}
```


Challenges

- The key difficulty of ND R-tree is to build an efficient tree that:
 - On one hand is balanced (so the leaf nodes are at the same height)
 - On the other hand the rectangles do not cover too much empty space and do not overlap too much (so that during search, fewer subtrees need to be processed).





NDRTree





NDRTree

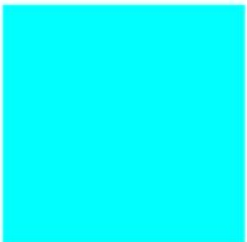
**Nearest
Neighbor
Search**

CMYK-Based Tertiaries

	IMPACT PURPLE <i>(dark purple)</i> PMS: 267 C RGB: 95, 36, 159 HEX: #5F249F CMYK: 81, 99, 0, 0		ELECTRIC BLUE <i>(light green-blue)</i> PMS: 325 C RGB: 100 204 201 HEX: #64CCC9 CMYK: 54, 0, 20, 0		RAT CAP <i>(warm yellow)</i> PMS: 116 C RGB: 255, 205, 0 HEX: #FFCD00 CMYK: 0, 10, 98, 0
	BOLD BLUE <i>(medium blue)</i> PMS: 7455 C RGB: 58, 93, 174 HEX: #3A5DAE CMYK: 86, 66, 0, 0		CANOPY LIME <i>(light warm green)</i> PMS: 2299 C RGB: 164, 210, 51 HEX: #A4D233 CMYK: 38, 0, 94, 0		NEW HORIZON <i>(orange-red)</i> PMS: 7417 C RGB: 224, 79, 57 HEX: #E04F39 CMYK: 0, 82, 82, 0
	OLYMPIC TEAL <i>(medium green-blue)</i> PMS: 321 C RGB: 0, 140, 149 HEX: #008C95 CMYK: 100, 0, 37, 10				

RGB-Based Tertiaries ("Brights")

The tertiary palette also includes a separate suite of brighter, web-only colors. These color standards maximize visibility and contrast on screens and should not be applied to print projects.

	BRIGHT PURPLE <i>(violet)</i> RGB: 120, 0, 255 HEX: #7800FF		BRIGHT ELECTRIC <i>(light teal blue)</i> RGB: 0, 255, 255 HEX: #00FFFF		BRIGHT BUZZ <i>(warm yellow)</i> RGB: 255, 204, 0 HEX: #FFCC00
	BRIGHT BLUE <i>(medium blue)</i> RGB: 41, 97, 255 HEX: #2961FF		BRIGHT CANOPY <i>(cool green)</i> RGB: 0, 236, 156 HEX: #00EC9C		BRIGHT HORIZON <i>(red-orange)</i> RGB: 255, 100, 15 HEX: #FF640F

Accessible Tech Gold

Use accessible color combinations to enhance legibility and ensure that Georgia Tech's content is available to a diverse audience. **Text in Tech Gold on a white background is not accessible.** When using Tech Gold for text on a website or other digital media, substitute Tech Medium Gold for large text and Tech Dark Gold for small bold text.

	TECH MEDIUM GOLD HEX: #A4925A RGB: 164, 146, 90 For use in large headline text in any digital media.		TECH DARK GOLD HEX: #857437 RGB: 133, 116, 55 For use in small bold text in any digital media.
---	--	---	--

	GRAY MATTER <i>(neutral dark gray)</i> PMS: 425 HEX: #54585A RGB: 84, 88, 90 HSL: 200, 3, 34 HSB: 200, 7, 35 CMYK: 48, 29, 26, 76		PI MILE <i>(light warm gray)</i> PMS: 427 HEX: #D6DBD4 RGB: 214, 219, 212 HSL: 103, 9, 85 HSB: 103, 3, 86 CMYK: 9, 4, 10, 8
	DIPLOMA <i>(light ivory yellow)</i> PMS: 7499 HEX: #F9F6E5 RGB: 249, 246, 229 HSL: 51, 63, 94 HSB: 51, 8, 98 CMYK: 1, 1, 10, 1		BUZZ GOLD PMS: 124 HEX: #EAAA00 RGB: 234, 170, 0 HSL: 44, 100, 46 HSB: 44, 100, 92 CMYK: 0, 30, 100, 0



Learned Index







Learned Index Using Regression

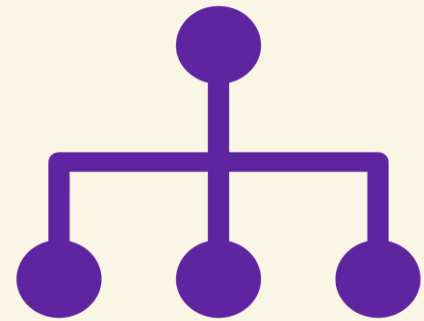


**Learned Index
Using
Regression**

**Learned Index
Using Neural
Network**

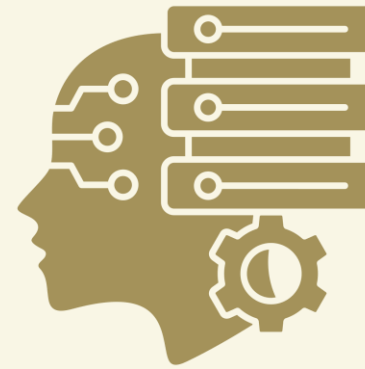
Limitations of B+Tree

Fixed Structure



Hierarchical nodes require multiple levels of traversal ($\log(n)$ comparisons), leading to higher CPU and memory costs.

Memory Overhead



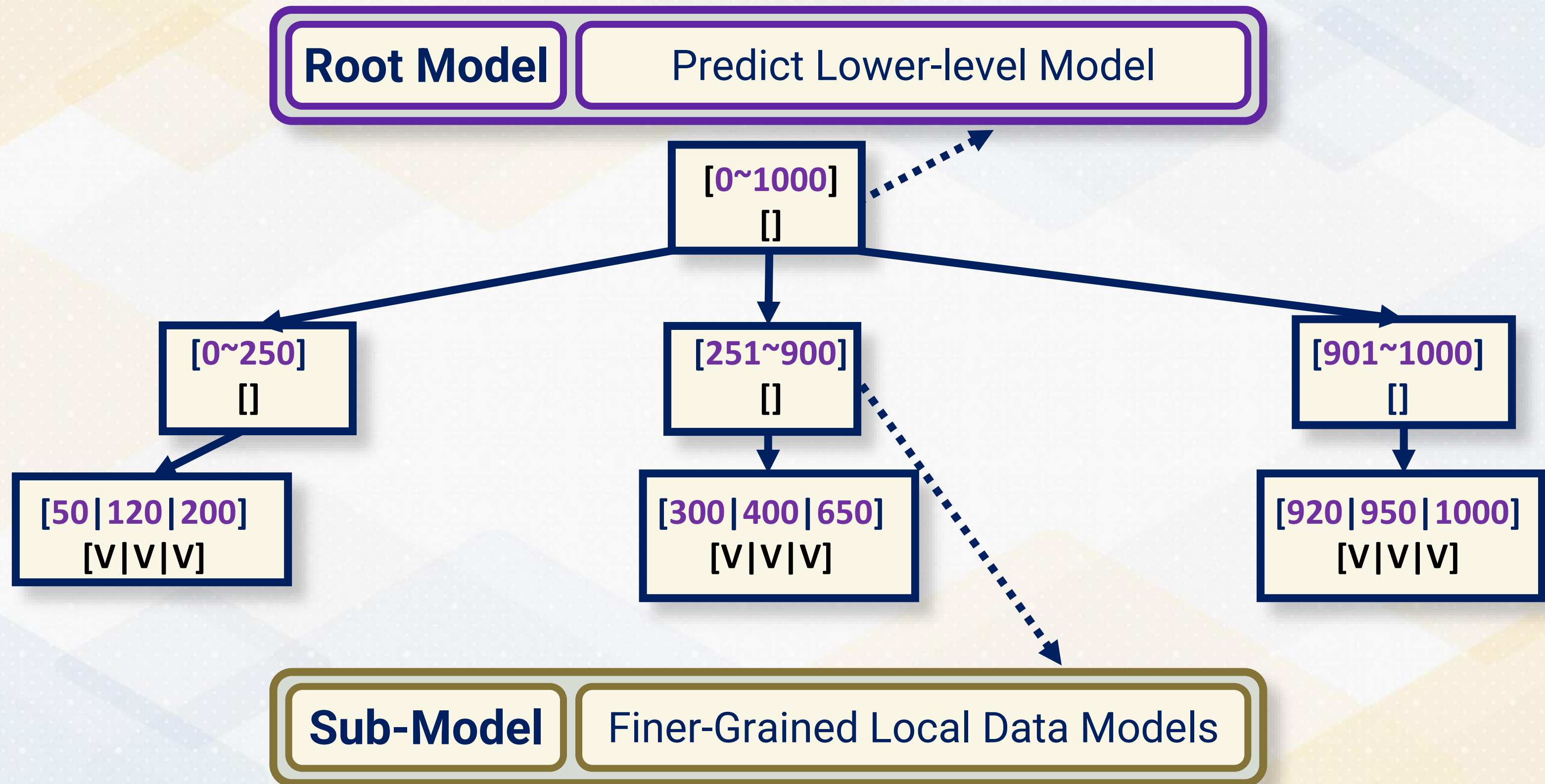
Each node holds pointers, using significant memory for large datasets.

Rigid Partitioning



B+ Trees don't adapt to predictable key patterns, which can cause unnecessary disk I/O and slower access.

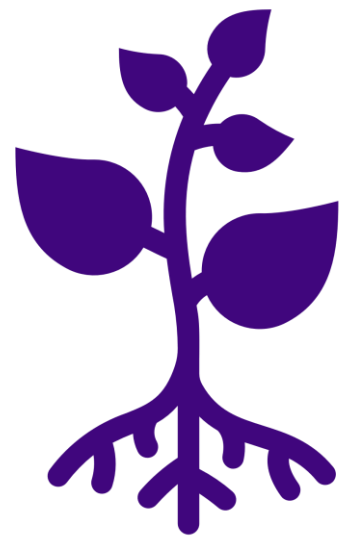
Learned Index



Learned Index

The background of the slide features a white grid pattern. Overlaid on this grid are several semi-transparent squares in two colors: a warm yellow and a cool blue. These squares are arranged in a staggered, overlapping fashion, creating a sense of depth and geometric complexity. The squares vary in size and are positioned across the entire slide, with a higher concentration towards the top and bottom edges.

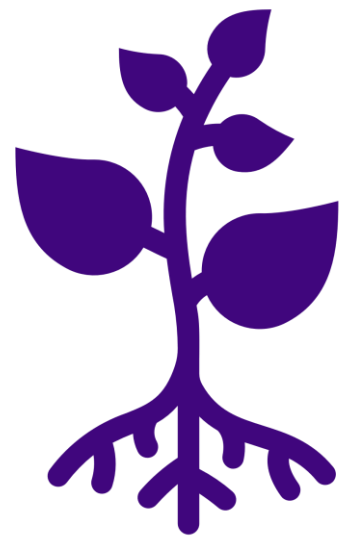
Learned Index



Root Model

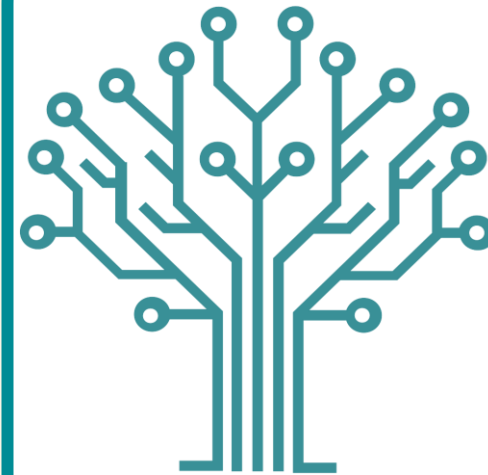
- Sub-Model Access
- Tree Elimination

Learned Index



Root Model

- Sub-Model Access
- Tree Elimination



Data Adaptive

- Memory Usage
- Faster Lookups

History of Learned Index



Machine Learning

Place a video on the right half of the screen.

See link:

https://v.ftcdn.net/06/14/04/53/700_F_614045399_UChvaRpxL5HpChISNXNQQKnV1Vr7KUUV_ST.mp4

History of Learned Index



Machine Learning

Predict Data Positions

Reduce B+ Tree Reliance

Place a video on the right half of the screen.

See link:

https://v.ftcdn.net/06/14/04/53/700_F_614045399_UChvaRpxL5HpChlSNXNQQKnV1Vr7KUUV_ST.mp4

History of Learned Index



Machine Learning

History of Learned Index



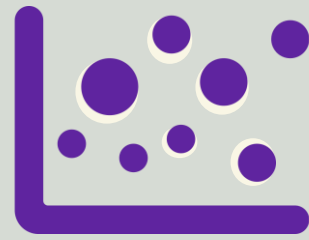
Machine Learning

Predict Data Positions

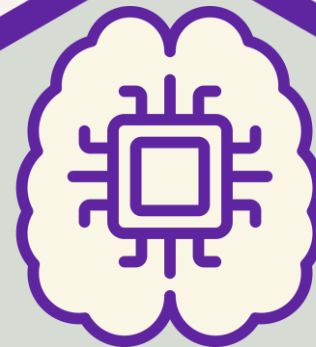
Reduce B+ Tree Reliance

Key Idea

Key Idea



**Data Distribution
Awareness**



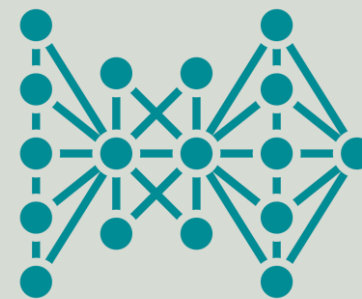
**Machine
Learning**



**Key Position
Mapping**



**Predictive
Power**



**Data
Patterns**



**Learned
Index**

B+Tree vs Learned Index

	B+ Tree	Learned Index
Structure	More rigid and hierarchical	Data-adaptive and predictive
Search Cost	Log(n) comparisons	Direct prediction w/ neighborhood search
Memory Usage	High, with pointers per node	Lower, fewer nodes needed

Structure of a Learned Index

```
struct Model {  
    double slope = 0.0;  
    double intercept = 0.0;  
    void train(const std::vector<int> &keys,  
               const std::vector<size_t> &positions);  
    size_t predict(int key) const;  
    void print() const;  
};
```



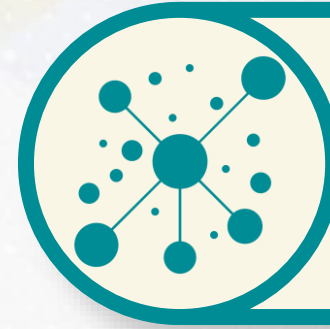

Training Root and Sub-Models



Training the Root and Sub-models



Root Model
Rough Key Mapping



Sub Model
Localized Predictions

```
void train(const std::vector<int> &keys, const std::vector<size_t> &positions) {  
    // Calculate mean  
    for (size_t i = 0; i < keys.size(); ++i) {  
        mean_x += keys[i];  
        mean_y += positions[i];  
    }  
    // Compute slope and intercept  
    slope = num / denom;  
    intercept = mean_y - slope * mean_x;  
}
```


Regression

Regression

position(key) =

slope * key

+ intercept

$$y(x) = m * x + b$$

Training Regression Model

$$y = mx + b$$

position	0	1	2	3	4
key	50	100	120	200	250

Training Regression Model

$$y = mx + b$$

**Slope-Intercept Form
Defines Linear
Relationships**

position	0	1	2	3	4
key	50	100	120	200	250

Training Regression Model



```
{  
    double num = 0.0, denom = 0.0;  
    for (size_t i = 0; i < keys.size(); ++i) {  
        double x_diff = keys[i] - mean_x;  
        num += x_diff * (positions[i] - mean_y);  
        denom += x_diff * x_diff;  
    }  
}  
  
slope = num / denom;  
intercept = mean_y - slope * mean_x;
```

Training Regression Model

Mean

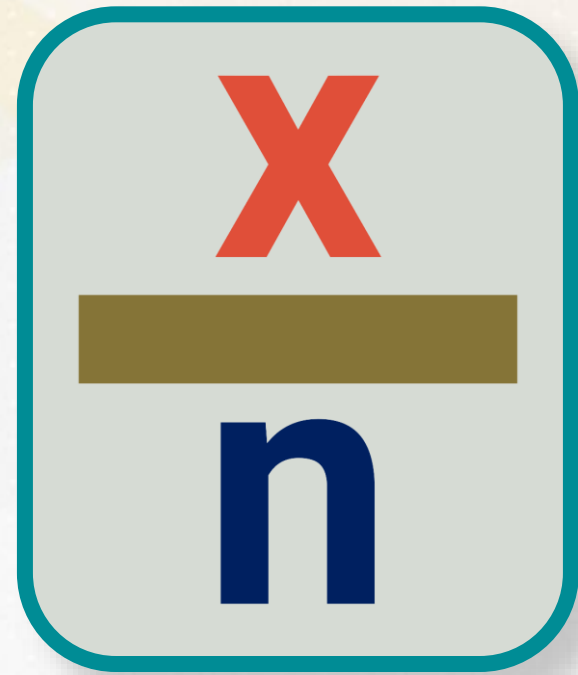
Slope

Intercept



```
{  
    double num = 0.0, denom = 0.0;  
    for (size_t i = 0; i < keys.size(); ++i) {  
        double x_diff = keys[i] - mean_x;  
        num += x_diff * (positions[i] - mean_y);  
        denom += x_diff * x_diff;  
    }  
}  
  
slope = num / denom;  
intercept = mean_y - slope * mean_x;
```


Training Regression Model

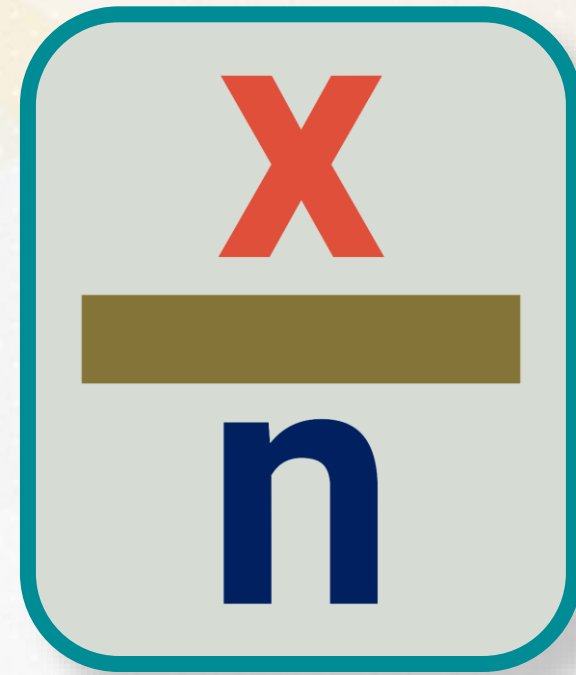


$$\text{slope} = \text{num} / \text{denom} = 500 / 20000 = 0.025$$

$$\text{intercept} = \text{mean}_y - (\text{slope} \times \text{mean}_x) = 2 - (0.025 \times 150) = -1.75$$

$$y = 0.025 \times x - 1.75$$

Training Regression Model

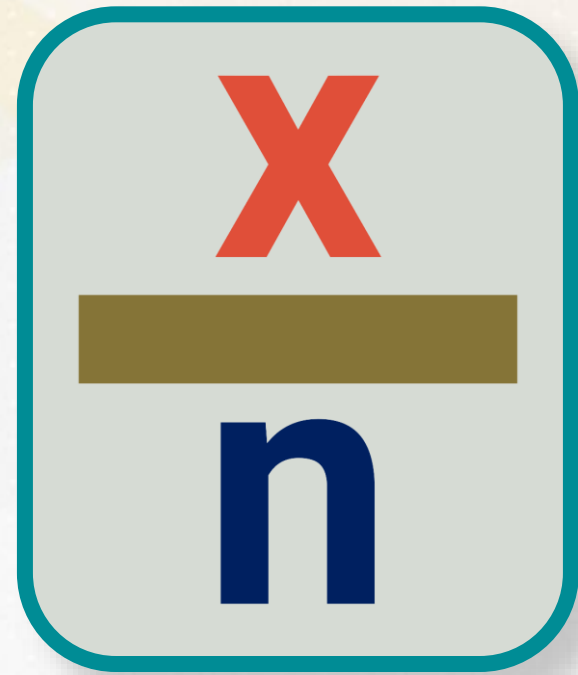


← Accumulates Covariance

$$\begin{aligned}\text{slope} &= \text{num} / \text{denom} = 500 / 20000 = 0.025 \\ \text{intercept} &= \text{mean_y} - (\text{slope} \times \text{mean_x}) = 2 - (0.025 \times 150) = -1.75\end{aligned}$$

$$y = 0.025 \times x - 1.75$$

Training Regression Model



Accumulates Covariance

Accumulates Variance of Keys

$$\text{slope} = \text{num} / \text{denom} = 500 / 20000 = 0.025$$
$$\text{intercept} = \text{mean_y} - (\text{slope} \times \text{mean_x}) = 2 - (0.025 \times 150) = -1.75$$

$$y = 0.025 \times x - 1.75$$

Training Regression Model

$$y = 0.025 \times x - 1.75$$

position	0	1	2	3	4
key	50	100	120	200	250
y	-0.5	0.75	2	3.25	4.5

Training Sub-Models

```
for (int i = 0; i < num_sub_models; ++i) {  
    cluster_keys.clear();  
    cluster_positions.clear();  
    for (size_t j = start; j < end; ++j) {  
        cluster_keys.push_back(sorted_data[j].first);  
        cluster_positions.push_back(j);  
        leaves[i].push_back(sorted_data[j]);  
    }  
    sub_models[i].train(cluster_keys, cluster_positions);  
}
```



Training Root and Sub-Models

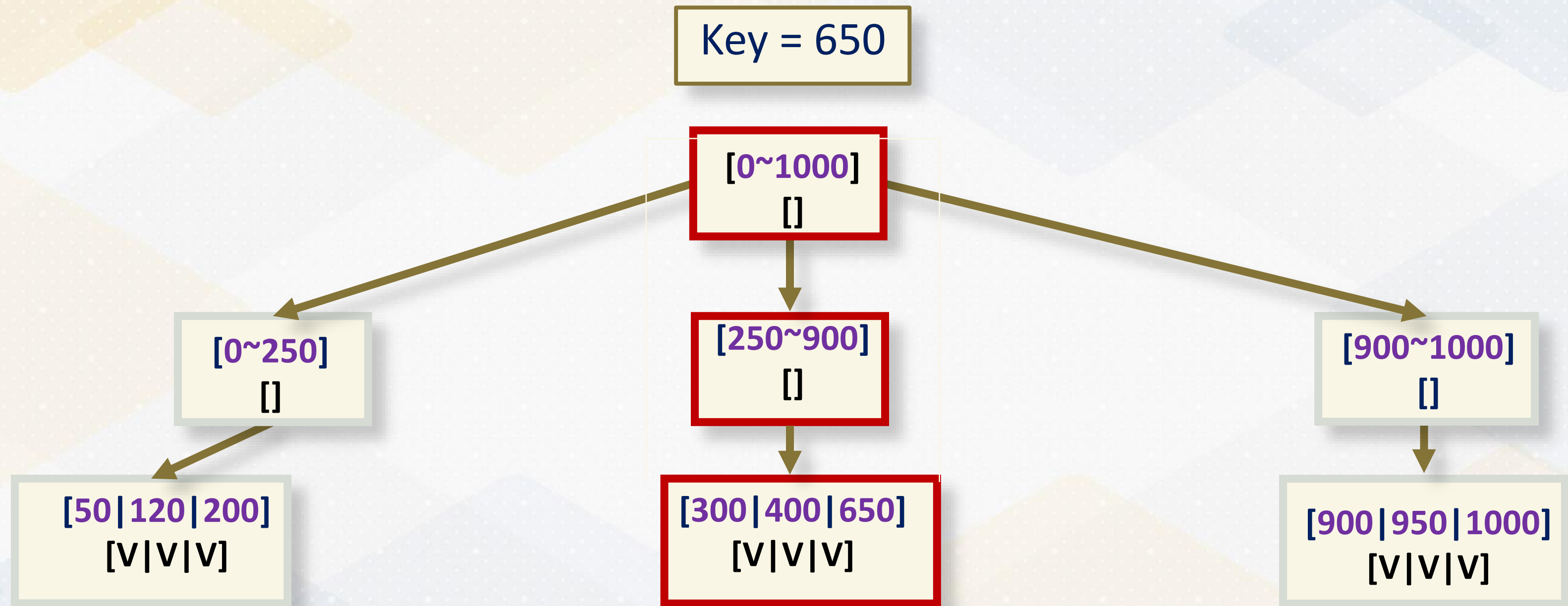


Position Prediction Process

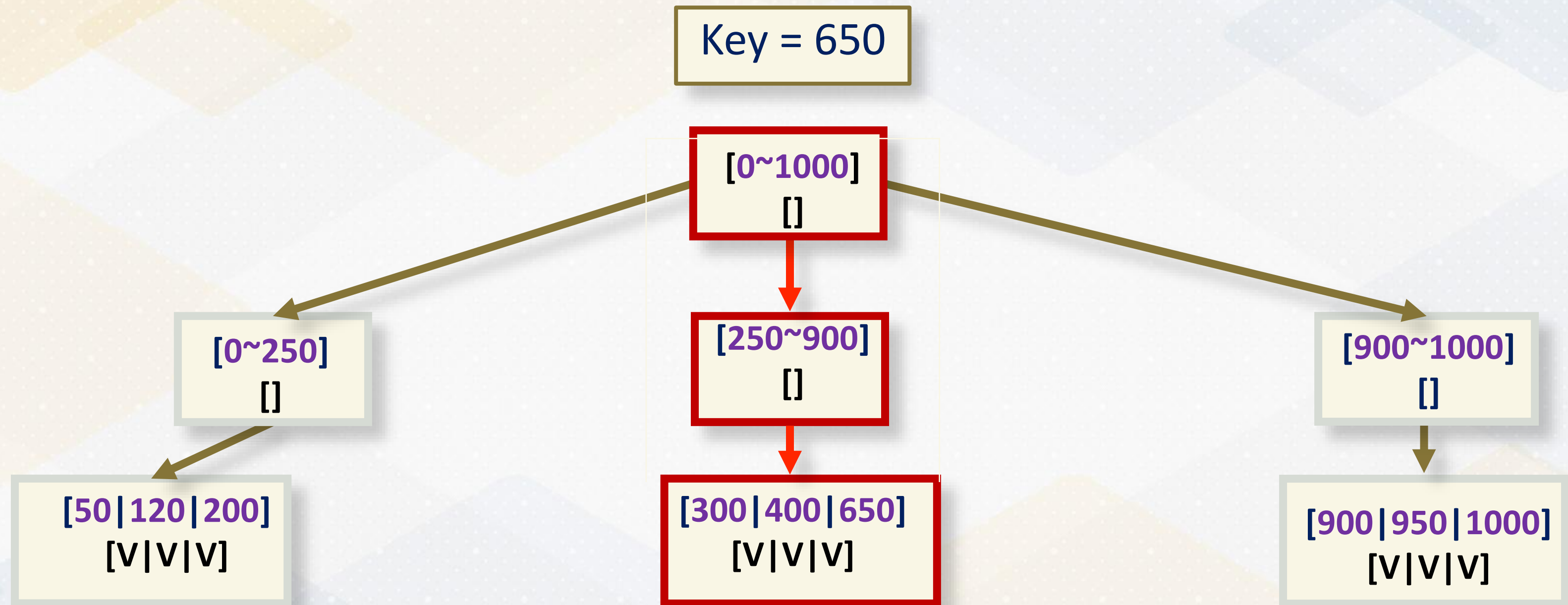


```
int sub_model_idx = root_model.predict(key);  
size_t pos = sub_models[sub_model_idx].predict(key);
```

Position Prediction Process



Position Prediction Process



Using Trained Regression Model

```
size_t predict(int key) const {  
    return static_cast<size_t>(std::max(0.0, slope * key + intercept));  
}
```


Neighborhood Search Around Predicted Position

```
int search_radius = 5;
int start = std::max(0, int(pos) - search_radius);
int end =
    std::min(int(leaves[sub_model_idx].size() - 1), int(pos) + search_radius);

for (int i = start; i < end; ++i) {
    if (key == leaves[sub_model_idx][i].first) {
        return leaves[sub_model_idx][i].second;
    }
}
```

Neighborhood Search Around Predicted Position

```
int search_radius = 5;
int start = std::max(0, int(pos) - search_radius);
int end =
    std::min(int(leaves[sub_model_idx].size() - 1), int(pos) + search_radius);

for (int i = start; i < end; ++i) {
    if (key == leaves[sub_model_idx][i].first) {
        return leaves[sub_model_idx][i].second;
    }
}
```


Example of Neighborhood Search

Predicted position = 10

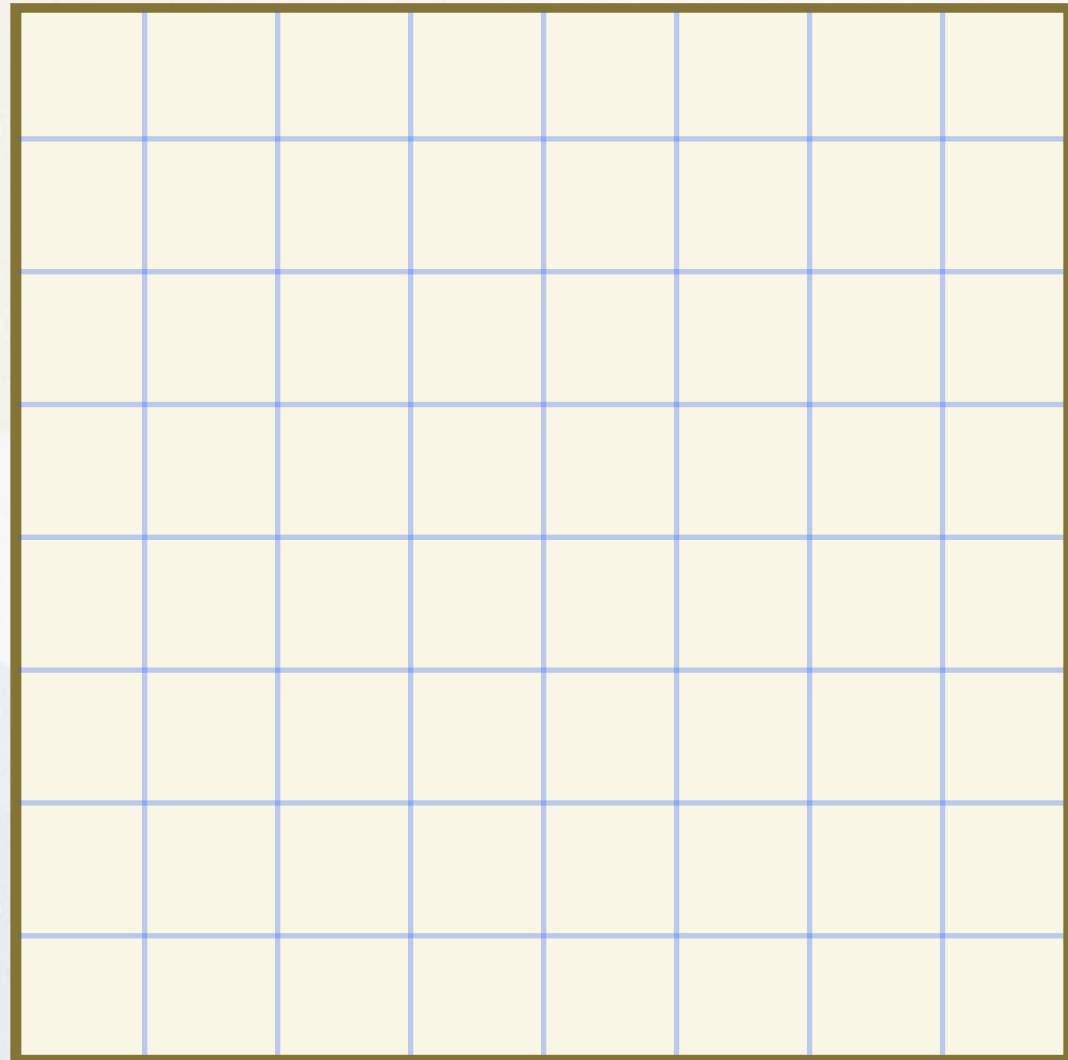
Search radius = 2

8	9	10	11	12
3	7	15	25	32

Practical Considerations for Neighborhood Search



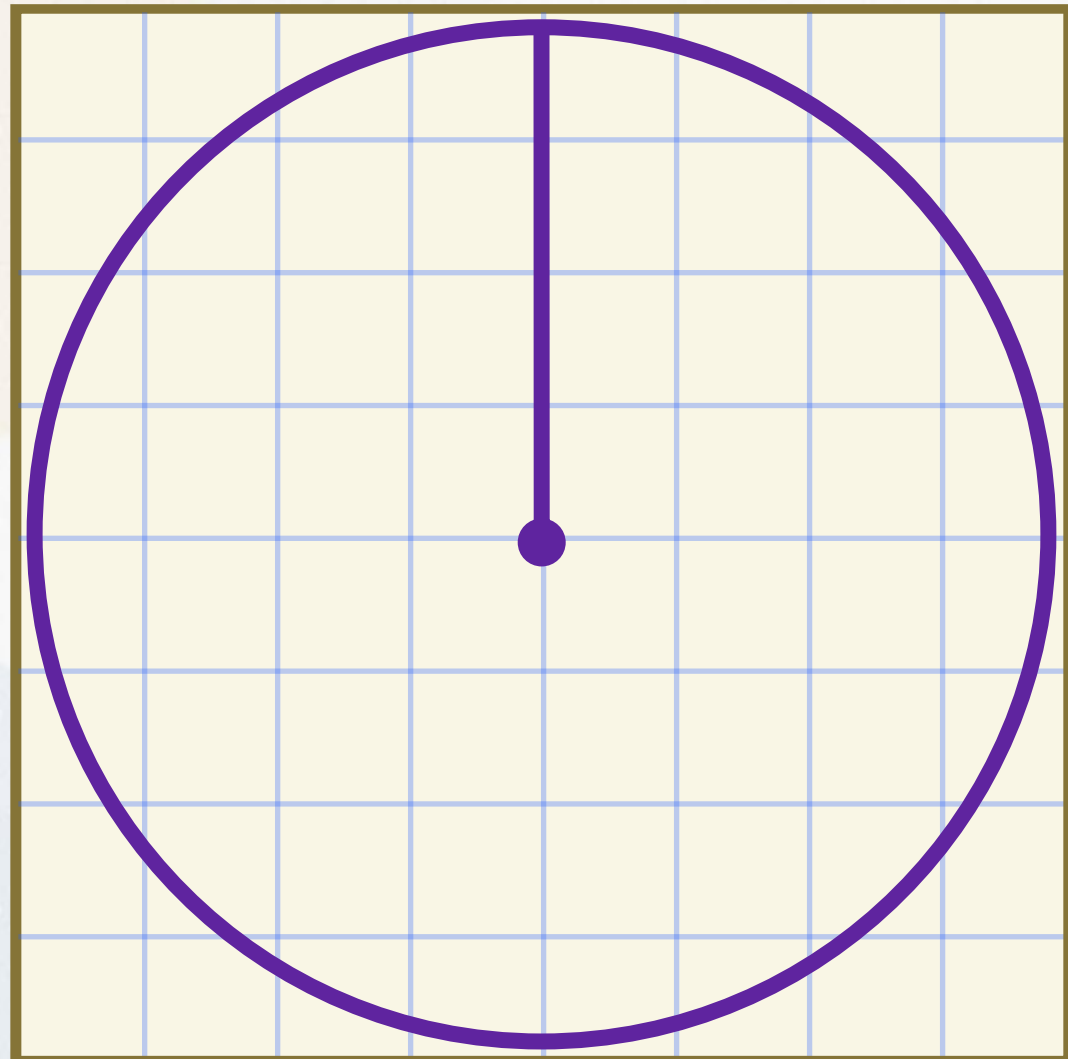
Radius Selection



Practical Considerations for Neighborhood Search



Radius Selection



Large Radius = Slower Search

Data Distribution Optimization

Radius Adjustment for Skewed Data

Testing the Learned Index

```
for (auto search_key : search_keys) {  
    std::cout << "Search for key " << search_key << " : "  
        << index.search(search_key) << std::endl;  
}
```


Benefits and Limitations of Learned Indexes

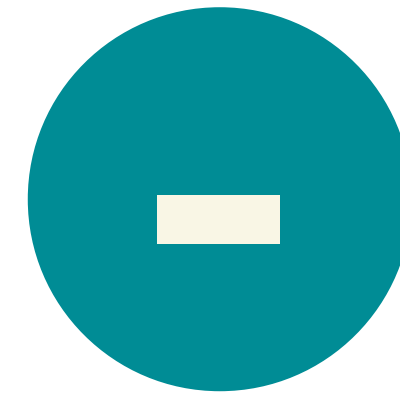


Benefits

Fast Lookups

Efficient Memory

Flexibility



Limitations

**Approximate
Predictions**

**Inaccurate for
Scattered Data**

Evaluation Metrics

The background of the slide features a white grid pattern. Overlaid on this grid are several semi-transparent squares in two colors: a warm yellow and a cool blue. These squares are arranged in a staggered, overlapping fashion, creating a layered, geometric effect. The yellow squares are primarily located on the left and bottom-right areas, while the blue squares are more prominent on the right and bottom-left areas.

Evaluation Metrics

Lookup Speed



Memory Usage



Accuracy



LEARNED INDEX USING NEURAL NETWORK

Limitation of Learned Index Using Regression

0	1	2	3	4	5	6	7	8
10	20	45	100	200	350	600	1000	1500

Non-Linearity

The background of the slide features a white grid pattern. Overlaid on this grid are several semi-transparent squares in two colors: a warm yellow and a cool blue. These squares are arranged in a staggered, overlapping fashion, creating a sense of depth and geometric complexity. The overall aesthetic is clean and modern.

Non-Linearity



Customer Behaviors



Seasonal Patterns



Geographical Distributions

Neural Network

Use a full-screen video of an AI neural network.

See the video:

https://v.ftcdn.net/03/45/31/52/700_F_345315215_XAM28vDcKdvy1rQuFBd7HKVlkoh0JiKg_ST.mp4

Neural Network

The background of the slide is an abstract composition of overlapping squares in various shades of yellow, light blue, and pale green. These squares are arranged in a way that creates a sense of depth and movement, with some squares appearing more prominent than others. The overall effect is a clean, modern, and visually appealing design.

Abstract Model Class

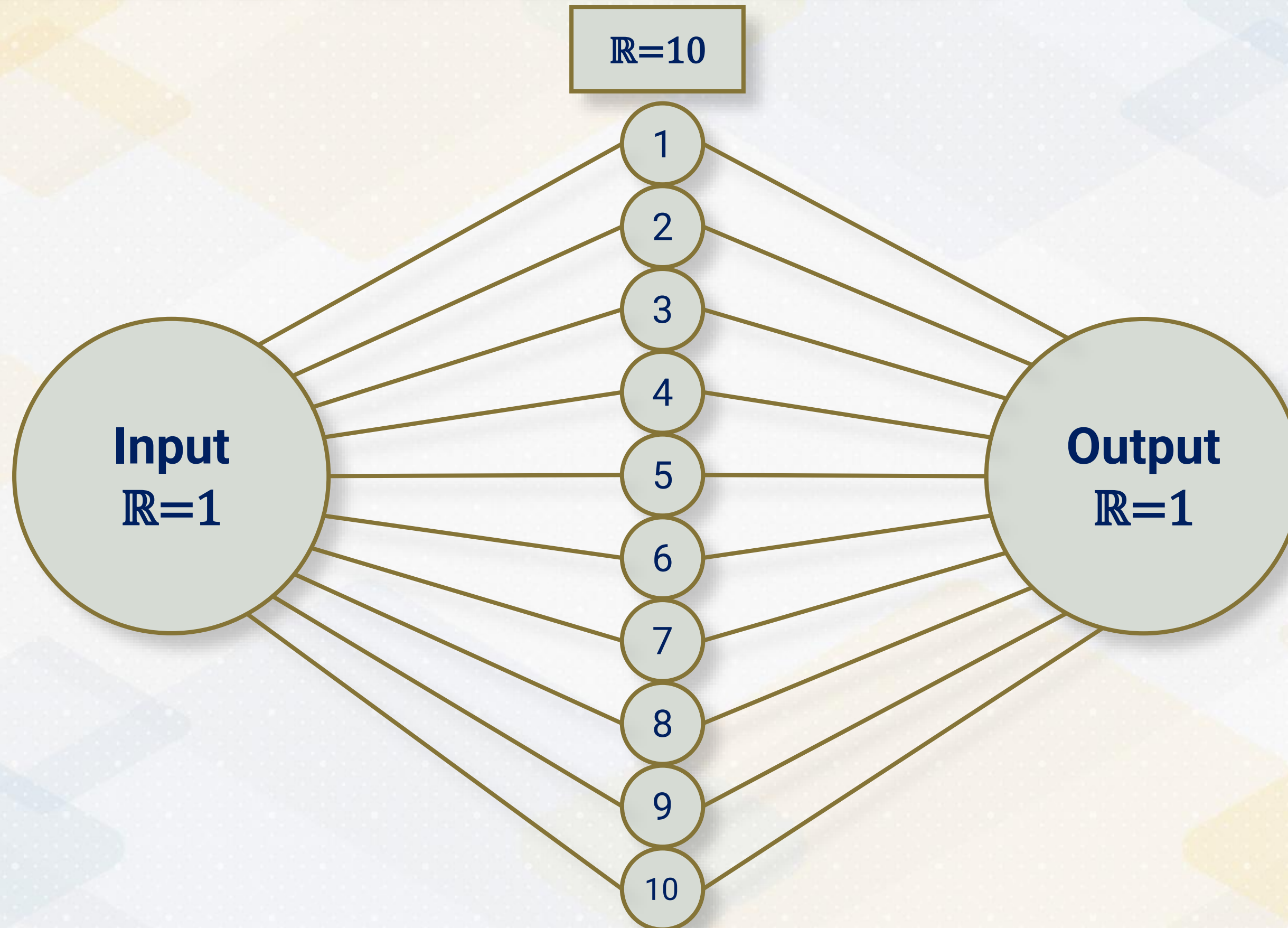
Defines a General Model Interface with **Train**, **Predict**, and **Print** Methods

```
struct Model {  
    virtual ~Model() {}  
    virtual void train(const std::vector<int> &inputs,  
                      const std::vector<double> &targets) = 0;  
    virtual double predict(int x) const = 0;  
    virtual void print() const = 0;  
};
```


SimpleNeuralNetwork Class

```
class SimpleNeuralNetwork : public Model {  
    std::vector<double> input_weights;  
    std::vector<double> output_weights;  
    double input_bias, output_bias;  
    int hidden_neurons;  
    // other variables and helper functions...  
};
```

Simple Neural Network Structure





TRAINING THE NEURAL NETWORK



Training the Neural Network for Learned Index

```
void train(const std::vector<int> &inputs, const std::vector<double> &targets) {  
    for (int epoch = 0; epoch < epochs; ++epoch) {  
        for (size_t i = 0; i < inputs.size(); ++i) {  
            // Forward and backward pass...  
            output_bias += learning_rate * delta_output;  
            input_bias += learning_rate * delta_hidden;  
        }  
    }  
}
```


Training the Neural Network for Learned Index

Training the Neural Network for Learned Index



Forward Pass

Calculate Hidden
Layer Outputs

Sum Outputs for
Prediction

Training the Neural Network for Learned Index



Forward Pass

Calculate Hidden
Layer Outputs

Sum Outputs for
Prediction



Backward Pass

Adjust Weights and
Biases

Error and Actual
Differences

Training the Neural Network for Learned Index



Forward Pass

Calculate Hidden
Layer Outputs

Sum Outputs for
Prediction



Backward Pass

Adjust Weights and
Biases

Error and Actual
Differences



Epochs & Learning Rate

Fine Tuned Learning
Rate

Increased Accuracy

Training the Neural Network for Learned Index



Normalize Each Key by Dividing by Maximum Key

Each Hidden Neuron Computes Its Output Using:

$$h_j = \text{sigmoid} (x \cdot w_{ij} + b_j)$$

Training the Neural Network for Learned Index

Final Predicted Position

$$y = \text{sum} (h_j \cdot w_{ij}) + b_o$$

Loss

$$\text{loss} = (y - y_{\text{actual}})^2 / n$$

Example Iteration

key = 10, position = 0

Normalize key: $10/1500 = 0.0067$

Initial weights and biases: 0.1

$h_1 = \text{sigmoid}(0.0067 \cdot 0.1 + 0.1) \approx 0.525$

Repeat for each hidden neuron; Compute predicted position y

Predicted position for key 10: 0.5

Loss = $(0.5 - 0)^2 = 0.25$

Backpropagation to adjust weights and biases for improved

Recursive Model Index

```
void initModels(bool use_neural_network) {  
    if (use_neural_network) {  
        root_model = std::make_unique<SimpleNeuralNetwork>();  
        for (int i = 0; i < 4; ++i) {  
            sub_models.push_back(std::make_unique<SimpleNeuralNetwork>());  
        }  
    }  
}
```


Benefits and Limitations of Neural Networks

Benefits

Flexibility in Capturing Non-Linear Data Patterns

Consideration

Tuning Required for Hidden Neuron Count, Learning Rate, and Epochs

Consideration

Network Weights Leads to Higher Memory Usage





Learned Index Using Regression



**Learned Index
Using
Regression**

**Learned Index
Using Neural
Network**